

A Journey to Core Python

*Experience the Applications of Tuples,
Dictionary, Lists, Operators, Loops,
Indexing, Slicing, and Matrices*

Mr. Girish Kumar

Dr. Ajay Shriram Kushwah

Ms. Raji Ramakrishnan Nair

Ms. Subhashiny G



www.bpbonline.com

FIRST EDITION 2022

Copyright © BPB Publications, India

ISBN: 978-93-55511-249

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.

Distributors:

BPB PUBLICATIONS

20, Ansari Road, Darya Ganj
New Delhi-110002
Ph: 23254990/23254991

DECCAN AGENCIES

4-3-329, Bank Street,
Hyderabad-500195
Ph: 24756967/24756400

MICRO MEDIA

Shop No. 5, Mahendra Chambers,
150 DN Rd. Next to Capital Cinema,
V.T. (C.S.T.) Station, MUMBAI-400 001
Ph: 22078296/22078297

BPB BOOK CENTRE

376 Old Lajpat Rai Market,
Delhi-110006
Ph: 23861747

To View Complete
BPB Publications Catalogue
Scan the QR Code:



Published by Manish Jain for BPB Publications, 20 Ansari Road, Darya Ganj, New Delhi-110002 and Printed by him at Repro India Ltd, Mumbai

Dedicated to

*My beloved parents **Shri. Manjit Raj** and **Shrimati. Asha Rani**
and*

*My wife **Monika** and kids **Vansh & Romil***

— Girish Kumar

*My beloved father **Late. Shriram Kirani Kushwaha**
and My family*

— Dr. Ajay Shriram Kushwaha

*My beloved father **Late M. N. Ramakrishnan Nair,**
My family and My dear students*

— Raji Ramakrishnan Nair

My beloved family and My dear students

— Subhashiny G

About the Authors

- **Girish Kumar** holds a B.Sc. (Computer Science) Degree and PGDCA from MIT and is a Research Scholar currently working as an Assistant Professor at Lovely Professional University. He has more than 18 years of teaching experience. He has one patent to his credit and has published more than 20 research papers in different national as well as international conferences and journals. He has authored three books published by reputed national and international publishers. He is also a Certified Academic Associate by IBM for DB2. He is an active member of IAENG- International Association of Engineers. In the field of programming, he has a good command of Fortran, C, C++, Java, Python, VC++.

LinkedIn profile: <https://www.linkedin.com/in/girish-kumar-21a62a14/>

- **Dr. Ajay Shriram Kushwaha** is an Associate Professor at the School of Computer Science & Information Technology, Jain (Deemed-to-be-University), Bengaluru, India. He has completed his Ph.D. in Computer Science & Technology in OSSDP from RTM Nagpur University, Nagpur, India, in 2017. His research interests include cyber and network security, as well as blockchain security. He has over 30 research papers published in prestigious international journals and IEEE Springer Conferences. He has authored four books published by reputed national and international publishers. He has served as the Editor in Chief in the IJEDR peer-reviewed journal and also as an editorial board member in more than 10 International Journals. He is also the International Journal of Sensors and Sensor Networks' guest editor. In conferences, he has served as the General Chair, Session Chair, and Panelist. He is a life member of the Hong Kong-based International Association of Engineers, the Indian Science Congress, and the IAENG Societies of Computer Science, AI, Data Mining, and Software Engineering. He is now supervising two students, one of whom has been conferred a Ph.D.

- **Raji Ramakrishnan Nair** holds a BCA and MCA degree and is a Research Scholar currently working as an IQAC Coordinator at Yeldo Mar Baselios College Kothamangalam. She has 17 years of teaching experience and believes that teaching is about being a friend, philosopher, and guide to her students. She is an author and philanthropist, actively involved in many social causes, which also made her students engage in relief works in Kerala mega floods and resulted in three houses being built for the flood victims.

LinkedIn profile: <https://www.linkedin.com/in/raji-nair-aa7038132>

- **Subhashiny G** holds a B.Sc.(C.S.) and MCA degree. She is passionate about technology and has 1.5 years of experience as a blockchain consultant. She is currently working as an Assistant Professor in the Department of Computer Applications at Yeldo Mar Baselios College Kothamangalam. She is also working as a research assistant (part-time) at Arab Open University, Muscat, Sultanate of Oman.

LinkedIn profile: <https://www.linkedin.com/in/subhashiny-g-a5a053189/>

About the Reviewer

Prashanth Raghu graduated with a Master's from the National University of Singapore and a Bachelor's from PES Institute of Technology, Bangalore, and has a keen interest in contributing to open source technologies. His interests lie in learning new technologies and their impact on developing scalable solutions. He currently works as a Staff Software Engineer at InMobi Technologies aiming to develop solutions at scale for the advertising domain. He has been pursuing Carnatic music initially under the tutelage of Sri. Vid. S Venkatesh and currently under the guidance of Sri. Vid. H.S. Venugopal.

Acknowledgements

The completion of this book would not have been possible without the guidance and support of so many people. First and foremost, I would like to thank my parents Shri. Manjit Raj and Shrimati. Asha Rani, my better half Monika and kids Vansh and Romil for continuously encouraging me to write this book.

I express my deep gratitude to Mr. Balraj Kumar (Assistant Professor and Assistant Dean, HOD Programming Techniques Domain, School of Computer Application, Lovely Professional University), Miss Divanshi Mehra (Project Engineer at Wipro Ltd), and Miss Sukhmanjeet Kaur (Assistant System Engineer at TCS) for their valuable co-operation and assistance.

-Girish Kumar

There are a few people I would like to thank for the continued and ongoing support they have provided during the writing of this book. First and foremost, I would like to thank my Mrs. Kismati (mother), Bindu (wife) and my daughters Anahita and Anshika, for understanding and encouraging me to write the book. I could have never completed this book without their support. Second, I would like to thank Subhashiny G. and Praveen N Kumar for providing technical support to complete the book.

I am grateful to my colleague Girish Kumar & Guru Dr. S. B. Kishor for his great ideas and also to other outstanding online content and courses provided by top universities and MOOC courses that have been developed across the globe over the past few years.

My gratitude also goes to the team at BPB Publications for being supportive enough and providing us sufficient time to complete the book.

-Dr. Ajay Shriram Kushwaha

I would like to specially acknowledge and thank a few people for their continuous support and encouragement at all stages of writing this book. Firstly, I would like to thank my family, especially my daughter Thanima, for understanding and encouraging me to write the book. I could have never completed this book without their support. I would like to thank Praveen N Kumar, Data Scientist, Django developer for providing technical support to complete the book.-

I am thankful to our beloved Chairman of Yeldo Mar Baselios College Kothamangalam, Chev. Prof. Baby M Varghese who has been motivating and inspiring me throughout.

I am grateful to the excellent online courses provided by top schools across the globe over the past few years.

I am very much thankful to the Technical Reviewer of this book Mr. Prashanth Raghu for his valuable suggestions.

I would also like to express my gratitude to the team at BPB Publications for their support and assistance in creating and publishing this book.

-Raji Ramakrishnan Nair

I feel I have learned a lot from the journey of this book. This will be the great treasure I am going to cherish not only in my future career but in my whole life.

I would like to take this opportunity to express my immense gratitude to the Almighty and all those people who have given their priceless support and assistance.

I would like to thank all my teachers wholeheartedly for helping me reach this stage.

-Subhashiny G

Preface

The approach adopted for learning the programming concepts using core Python is very simple. After discussing general ideas of instructions in every chapter, we explore the same concepts using programming examples. Our main objective is to learn core python programming. So we have written sufficient programs for building programming concepts for the non-programmer. This book contains 16 chapters covering core Python with concepts of programs in detail. This book will help you understand Python from scratch and help you build a career in programming.

The key features of our book are:

1. The focus is provided on the in-depth treatment of the basics of Python Programming.
2. Each programming concept is well described with the help of suitable example programs.
3. The history to the core of Python has been explained well, helping any naïve programmer to understand and learn Python Programming.
4. It covers code reusability concepts in Python.
5. It contains the concepts of File handling, exceptional handling, and run time error.
6. It consists of basic concepts of OOps in Python like Class, constructor, Inheritance, etc.

Chapter 1 focuses on Python's past, present, and future. With this chapter, the reader will understand Python, its history, benefits, features, and various applications.

Chapter 2 focuses on the basic syntax of Python programming. All these syntaxes are well explained with lots of examples, which will boost logical thinking. With this chapter, the base of python programming gets strong in every reader.

Chapter 3 covers the variables and different data types available in python programming. The concept of data types is well explained with many examples, which helps the reader understand the concept effectively.

Chapter 4 covers arithmetic and logical operators used in python programming. All these concepts are well explained with the help of examples, which helps the reader understand the usage of these operators effectively.

Chapter 5 emphasizes decision-making statements available in python programming. Different usage styles of IF statements are well explained with suitable examples, helping students develop their logical skills.

Chapter 6 emphasizes the loop statements used in python programming. All the loop statements are neatly explained with suitable examples, helping students understand which loop statements to use while doing programming tasks.

Chapter 7 unleashes the concept of numbers. The chapter focuses on the number type conversion and various functions such as mathematical function, random number function, trigonometric function, and mathematical constants.

Chapter 8 deals with the concept of strings. The chapter focuses on the operators used with strings, various string operations, and string formatting.

Chapter 9 deals with the concept of lists and various operations that can be performed with this data structure.

Chapter 10 deals with the concept of tuples and various operations that can be performed with this data structure.

Chapter 11 covers the concept of dictionaries and various operations that can be performed with this data structure.

Chapter 12 provides a detailed explanation of defining and calling functions. It also deals with important concepts such as pass by reference v/s value, recursion, function arguments, and anonymous functions.

Chapter 13 focuses on the concept of Python's highest-level program organization unit. It also deals with the importing function.

Chapter 14 lays special focus on the concept of files. It deals with the various operations performed in files, such as opening and closing files, reading and writing files, renaming and deleting files, and other file methods.

Chapter 15 elucidates the concepts of exception handling that can be used to make your programs robust. Concepts such as raising and handling exceptions, user-defined exceptions, etc., used for handling exceptions are demonstrated in this chapter.

Chapter 16 deals with OOPS concept implementation in Python programming. It mainly focuses on various built-in attributes, types of inheritance, overriding methods, etc.

We expect fruitful learning through our book **Core Python Programming**.

Code Bundle and Coloured Images

Please follow the link to download the *Code Bundle* and the *Coloured Images* of the book:

<https://rebrand.ly/5d390>

The code bundle for the book is also hosted on GitHub at **<https://github.com/bpbpublications/link>**. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at **<https://github.com/bpbpublications>**. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At **www.bpbonline.com**, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at **business@bpbonline.com** with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit **www.bpbonline.com**. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit **www.bpbonline.com**.

Table of Contents

1. Introduction	1
Introduction.....	1
Structure.....	1
Objectives.....	1
History of Python	2
Features of Python.....	2
Benefits of Python.....	3
Applications of Python	4
Conclusion	5
Key terms	5
Questions	5
2. Basic Syntax.....	7
Introduction.....	7
Structure.....	7
Objectives.....	8
First Python program	8
Program output and print statement.....	9
Script mode programming	9
Python identifiers	10
<i>Rules for naming identifiers</i>	<i>10</i>
Keywords.....	11
Lines and indentation	11
Multi-line statements	12
Quotations used in Python.....	12
Comments in Python.....	13
Multiple statements on a single line	13
Conclusion	13
Questions	14

3. Variable Types.....	15
Introduction.....	15
Structure.....	15
Objectives.....	16
Variables and assignment.....	16
Multiple assignments.....	16
Built-in data types.....	17
<i>Numbers</i>	17
<i>Strings</i>	18
<i>Lists</i>	18
<i>Tuples</i>	19
<i>Dictionary</i>	20
Data type conversion	21
Conclusion	22
Questions	22
 4. Basic Operators	23
Introduction.....	23
Structure.....	23
Objectives.....	24
Arithmetic operators	24
Relational operators	26
Python assignment operators	27
Bitwise operators	29
Logical operators	31
Membership operators.....	32
Operators precedence	34
Programs	35
Conclusion	39
Questions	39
 5. Decision Making	41
Introduction.....	41
Structure.....	42
Objectives.....	42

if statement	42
if..else statement.....	43
elif statements.....	44
Nested if statements	46
Single statement suites.....	47
Programs	47
Conclusion	50
Key terms	50
6. Repeating Code Using Loops.....	51
Introduction.....	51
Structure	52
Objective.....	52
While loop.....	52
For loop	53
Range() function.....	54
Break, continue, pass, and the loop else	55
Break.....	56
<i>Continue</i>	56
<i>Pass</i>	57
<i>Loop else</i>	58
Nested loops.....	58
Infinite loop	59
Iterator and generator	60
Conclusion	68
Multiple Choice Questions.....	68
7. Numbers.....	71
Introduction.....	71
Structure	71
Objectives.....	71
Types of numbers.....	72
Mathematical functions	73
<i>Number abs() method</i>	73
<i>Number ceil() method</i>	74

<i>Number exp() method</i>	74
<i>Number fabs() method</i>	75
<i>Number floor() method</i>	76
<i>Number log() method</i>	77
<i>Number log10() method</i>	77
<i>Number modf() method</i>	78
<i>Number pow() method</i>	79
<i>Number sqrt() method</i>	79
<i>Number round() method</i>	80
Random number functions	81
<i>Number choice() method</i>	81
<i>Number randrange() method</i>	82
<i>Number random() method</i>	83
<i>Number seed() method</i>	83
<i>Number shuffle() method</i>	84
<i>Number uniform() method</i>	85
Trigonometric functions	85
<i>Number acos() method</i>	86
<i>Number asin() method</i>	87
<i>Number atan() method</i>	87
<i>Number atan2() method</i>	88
<i>Number cos() method</i>	88
<i>Number hypot() method</i>	89
<i>Number degrees() method</i>	89
<i>Number radians() method</i>	90
Mathematical constants	90
Conclusion	91
Key terms	91
8. Strings	93
Introduction	93
Structure	93
Objectives	94
Creating a string	94
Accessing values in a string	94

Changing or deleting a string	96
Escape characters	96
String operations.....	97
Concatenation	97
Repetition.....	98
Membership	98
Raw string	98
String formatting.....	99
Triple quote code	99
Unicode string	100
Built-in string methods	100
String capitalize() method	100
String center() method.....	100
str.center(width[, fillchar])	100
String count() method	101
String decode() method	101
String encode() method	102
String endswith() method.....	103
String expandtabs() method	104
String find() method	104
String index() method.....	105
String isalnum() method	106
String isalpha() method	106
String islower() method	107
String isnumeric() method	107
String isspace() method	108
String istitle() method	109
String isupper() method.....	109
String join() method	110
String len() method.....	110
String ljust() method	111
String lower() method.....	111
String lstrip() method	112
String maketrans() method.....	112

<i>String max()</i> method.....	113
<i>String min()</i> method	113
<i>String replace()</i> method	114
<i>String rfind()</i> method.....	114
<i>String rindex()</i> method	115
<i>String rjust()</i> method.....	116
<i>String rstrip()</i> method	116
<i>String split()</i> method	117
<i>String splitlines()</i> method.....	117
<i>String startswith()</i> method	118
<i>String strip()</i> method.....	118
<i>String swapcase()</i> method.....	119
<i>String title()</i> method	119
<i>String translate()</i> method	120
<i>String upper()</i> method	121
<i>String zfill()</i> method	121
<i>String isdecimal()</i> method.....	122
Programs	122
Conclusion	127
Questions	127
9. Lists	129
Introduction.....	129
Structure	129
Objectives.....	130
Lists in Python.....	130
Accessing values in a list	130
Updating lists	130
Deleting list elements	131
Built-in list functions and methods.....	131
<i>List len()</i> method.....	131
<i>List max()</i> method.....	132
<i>List min()</i> method	132
<i>List list()</i> method.....	133
<i>List append()</i> method.....	134

<i>List count() method</i>	134
<i>List extend() method</i>	135
<i>List index() method</i>	136
<i>List insert() method</i>	136
<i>List pop() method</i>	137
<i>List remove() method</i>	137
<i>List reverse() method</i>	138
<i>List sort() method</i>	138
Programs	139
Conclusion	145
Questions	145
10. Tuples	147
Introduction	147
Structure	147
Objective.....	148
Creating a tuple.....	148
Accessing values in tuples.....	148
Updating tuples	148
Deleting tuple elements	149
Basic tuple operations	150
<i>Len</i>	150
<i>Concatenation</i>	150
<i>Repetition</i>	150
<i>Membership</i>	150
<i>Iteration</i>	151
Indexing, slicing, and matrices	151
Built-in tuple functions	152
<i>Tuple len() method</i>	152
<i>Tuple max() method</i>	152
<i>Tuple min() method</i>	153
<i>Tuple tuple() method</i>	153
Programs	154
Conclusion	155
Questions	156

11. Dictionaries	157
Introduction.....	157
Structure.....	157
Objectives.....	158
Creating a dictionary.....	158
Accessing the elements in a dictionary	158
Updating a dictionary	159
Deleting dictionary elements	159
<i>Properties of dictionary keys.....</i>	<i>160</i>
Built-in dictionary functions	160
<i>Dictionary len() method</i>	<i>160</i>
<i>Dictionary str() method</i>	<i>161</i>
<i>Dictionary type() method</i>	<i>161</i>
Methods on dictionary.....	162
<i>Dictionary clear() method</i>	<i>162</i>
<i>Dictionary copy() method.....</i>	<i>163</i>
<i>Dictionary fromkeys() method.....</i>	<i>163</i>
<i>Dictionary get() method</i>	<i>164</i>
<i>Dictionary items() method</i>	<i>165</i>
<i>Dictionary keys() method</i>	<i>165</i>
<i>Dictionary setdefault() method</i>	<i>166</i>
<i>Dictionary update() method</i>	<i>166</i>
<i>Dictionary values() method.....</i>	<i>167</i>
Programs	167
Conclusion	171
Questions	171
12. Functions.....	173
Structure.....	173
Objectives.....	174
Built-in functions	174
User-defined functions.....	174
Defining a function.....	175
Function calling.....	175
Docstring.....	176

Return statement.....	176
Pass by reference vs. value	177
Function arguments	178
<i>Required arguments</i>	178
<i>Default arguments</i>	179
<i>Keyword arguments</i>	180
<i>Arbitrary arguments</i>	180
Anonymous functions.....	181
Use of lambda function.....	182
Scope of variable.....	183
Recursion	184
<i>Advantages of recursion</i>	185
<i>Disadvantages</i>	185
<i>Comparisons between iteration and recursion</i>	185
Programs	185
Conclusion	191
Key terms	191
13. Modules.....	193
Introduction.....	193
Structure.....	193
Objective.....	194
The import statement.....	194
The from...import statement	195
The from...import * statement.....	195
Executing modules as scripts.....	196
The module search path.....	196
The dir() function.....	197
The globals() and locals() functions	197
The reload() function.....	198
Packages in Python.....	198
Programs	199
Conclusion	200
Questions	200

14. Files I/O	201
Introduction.....	201
Structure.....	202
Objectives.....	202
Open and close files.....	202
The open function.....	202
Modes of opening a file.....	203
The file object attributes.....	204
<i>The close() method</i>	<i>204</i>
Reading and writing files	205
<i>The write() method.....</i>	<i>205</i>
<i>The read() method</i>	<i>205</i>
Renaming and deleting files.....	206
<i>The rename() method</i>	<i>206</i>
<i>The remove() method.....</i>	<i>206</i>
<i>os.remove(file_name).....</i>	<i>206</i>
File methods	206
<i>File flush() method</i>	<i>207</i>
<i>File fileno() method</i>	<i>207</i>
<i>fileObject.fileno().....</i>	<i>207</i>
<i>File isatty() method.....</i>	<i>208</i>
<i>File next() method.....</i>	<i>208</i>
<i>File readline() method</i>	<i>209</i>
<i>File readlines() method.....</i>	<i>210</i>
<i>File seek() method.....</i>	<i>211</i>
<i>File tell() method.....</i>	<i>212</i>
Programs	213
Conclusion	217
Key terms	218
15. Exception Handling	219
Structure.....	219
Objectives.....	220
Standard exceptions	221

Python assertions.....	222
Handling an exception.....	223
The except clause with else clause	225
With no exceptions except close	226
The except clause with multiple exceptions	226
The Try-finally clause	229
Argument of an exception.....	231
Raising an exception.....	231
User-defined exceptions	232
Conclusion	234
Key terms	234
16. Object-Oriented Programming.....	235
Structure.....	235
Objectives.....	236
An overview of OOP terminology	236
Creating classes.....	237
<i>The self</i>	237
<i>The __init__ method</i>	238
Creating instance objects	238
Accessing attributes.....	238
Built-in class attributes	239
Destroying objects (garbage collection).....	241
Class inheritance.....	241
Types of inheritance	242
<i>Multilevel inheritance</i>	243
<i>Multiple inheritance</i>	243
The super () keyword.....	244
Functions to check the relationship between two classes and instances...	245
Overriding methods.....	245
<i>Base overloading methods</i>	245
Programs.....	246
Conclusion	251
Key terms	251
Index	253-0000000

CHAPTER 1

Introduction

Introduction

Python is a strong and straightforward programming language that delivers the power and complexity of traditional compiled languages and the ease-of-use (and then some) of simpler scripting and interpreted languages.

Structure

We will cover the following topics in this chapter:

- History of Python
- Features of Python
- Benefits of Python
- Applications of Python

Objectives

The main objectives of this chapter are to provide you with an overview of Python, and discuss its benefits, its future scope and importance, and its application in the real world.

History of Python

Python came into the picture in the late 1980s, and its implementation was started in December 1989 by Guido van Rossum at CWI in Netherlands.

At the time, Guido van Rossum was a researcher with considerable language design experience with the interpreted language ABC, also developed at CWI. Still, he was unhappy with its ability, so he came up with something more. Some of the tools he visualized performed general system administration tasks, so he also wanted access to the power of system calls, which is available with distributed operating systems such as Amoeba. Even though Amoeba-specific language gave some new thoughts, a generalized language gave more sense, and the seeds of Python were sown late in 1989.

Python was extracted from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, Unix shell, and other scripting languages.

Features of Python

The following are the main features of Python:

- **Easy-to-learn:** Python hardly has any keywords or reserved words, understandable organization, and unambiguous syntax, which makes the language easy to understand for a student.
- **Readable:** Python code is distinctly set and noticeable.
- **Simple-to-maintain:** Python's source code is relatively simple to manage.
- **An extensive, conventional libraries:** Python's conventional library is portable and cross-platform friendly with Windows, UNIX, and Macintosh operating systems.
- **Hands-on approach:** Python provides a Do-It-Yourself REPL interpreter to learn the language and library features before moving them into the application. The REPL console can also write small test scripts.
- **Compactable:** Python can work on various platforms and needs an indistinguishable interface for every platform.
- **Extendable:** Python interpreter supports low-level modules and helps programmers add or customize their tools for efficient programming.
- **Databases:** Python provides libraries for data-driven applications to interact with all major commercial databases.
- **GUI programming:** Python supports GUI applications. It can be produced and ported to different system calls, libraries, and windows systems, for

example, Windows MFC, Macintosh, and the X Window system of Unix operating systems.

- **Scalable:** Python caters to a better organization and upholds wide programs than shell scripting.

Python has a list of other good features as well:

- It supports functional and structured programming methods and also OOPs concepts.
- It provides high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can also be used as a scripting language or compiled to bytecode for developing a wide range of applications that perform enormous operations.
- Even Python scripts that are executed are compiled to Python bytecode before execution.
- It provides high-level dynamic data types and supports dynamic type checking. It can be easily unified with C, C++, COM, ActiveX, CORBA, and Java using Python APIs.

Benefits of Python

The divergent application of the Python language is a result of its combination of features. It makes Python more adaptable for a wide range of applications. Some of the perks of Python programming are:

- **The presence of third-party modules:** The Python Package Index (PyPI) comprises numerous third-party modules, enabling Python to interact with distinct languages and platforms available.
- **Broad support libraries:** Python provides a large standard library, including internet protocols, string operations, web services tools, and operating system interfaces. Many high-level programming tasks have already been scripted into the standard library, leading to the reduction of the length of code to be written.
- **Open source and community development:** Python language is constructed beneath an OSI-approved open-source license, making it free to use and distribute, even for commercial purposes.
- **Learning ease and support available:** Python offers excellent readability and accessible simple-to-learn syntax, which helps beginners adopt the language

effectively. The code style guidelines, such as PEP 8, provide rules to aid code formatting. Also, the distinct users and active developers made the internet resource bank rich, leading to the development and continued usage of this programming language. The Python Package Index (PyPI) comprises numerous third-party modules and enables Python to interact with distinct languages.

- **Handy data structures:** Python holds a built-in list and dictionary data structures and is used to construct data structures with fast runtime. Moreover, it affords dynamic high-level data types, leading to reduction in code length.
- **Productivity and speed:** Python holds an object-oriented design structure, giving a redesigned procedure control capacities, solid consolidation, text preparing capacities, and its unit testing framework. Each one of these capacities increase profitability. Python is seen as an appropriate choice for building complex multi-protocol network applications.

Applications of Python

Here are some of the application areas of Python:

- **GUI-based desktop applications:** Python has a basic structure, modular design, rich tools for text processing, and the capacity to work on different operating systems, making it an acceptable choice for developing desktop-based applications. Several GUI toolkits like wxPython, PyQt, and PyGtk accessible assist designers in making profoundly useful Graphical User Interface (GUI). The different applications created utilizing Python incorporates are:
 - **Image processing and graphic design applications:** Python is used to develop 2D imaging software, for example, Inkscape, GIMP, Paint Shop Pro, and Scribus. It is also used in variable proportions like in 3D animation packages such as Blender, 3ds Max, Cinema 4D, Houdini, Lightwave, and Maya.
 - **Scientific and Computational Applications:** The high speed, productivity, and scope of various tools, like Scientific Python and Numeric Python, make Python an integral part of various applications involving computations and processing of scientific data. Various software are coded in Python, such as 3D modeling software like FreeCAD and finite element method software like Abaqus.
 - **Games:** Python supports the development of games with diverse modules, libraries, and platforms. There are many examples available, and one of those is PySoy. It is a 3D game engine that supports Python 3, and PyGame contains a set of modules providing

functionality and a related library for the game's development. Various games are built using Python, such as Battlefield 2, Bridge Commander, and Eve Online.

- **Web frameworks and web applications:** Python has been used to build various web frameworks, including CherryPy, Django, Pyramid, TurboGears, Bottle, and Flask. These frameworks come up with standard libraries and modules that simplify tasks such as managing content, database interactions, and interfacing with different flavors of internet protocols like HTTP, SMTP, XML-RPC, FTP, and POP. Here are some of the popular web applications based on Python:
 - Plone is a content management system
 - ERP5 is an open-source ERP used mainly in aerospace, apparel, and banking
 - Odoo is a blended suite of different business applications and Google App Engine
- **Enterprise and business applications:** Python is the most suitable language for coding and has special libraries, extensibility, scalability, and easily readable syntax. Reddit, which was initially developed in Common Lisp, was rescripted in Python in the year 2005. Python also came up with different functionalities for the YouTube online video sharing platform.
- **Operating systems:** Python is often a basic part of Linux distributions. For example, Ubuntu's Ubiquity Installer and Fedora and Red Hat Enterprise Linux's Anaconda Installer are developed in Python. Gentoo Linux used Python for its package management system known as Portage.
- **Language development:** Python has affected the development of many languages with its design and module architecture. For example, Boo language uses an object model, syntax, and indentation equivalent to Python. Additionally, the syntax of many languages like Apple's Swift, CoffeeScript, Cobra, and OCaml share the features of the Python programming language.
- **Prototyping:** Other than being brisk and simple to learn, Python has the open-source bit of leeway of being free with the help of a huge network. This helps it to be a favored decision for the development of prototypes. Python allows faster development of its prototype with features like agility, extensibility and scalability, and ease of refactoring code. Python is suited for prototyping primarily due to its scripting nature, ease of writing code, and library support. The availability of a developer network might not be a right pointer for creating prototypes as even JAVA/C# have solid developer networks.

Conclusion

In this chapter, we focused on basic things about Python. Python is a robust and straightforward programming language that delivers the power and complexity of traditional compiled languages and the ease of use of simpler scripting and interpreted languages. Guido van Rossum developed Python in the late 1980s, and its implementation was started in December 1989. This chapter discussed the basic features of Python. Currently, it is the most popular programming language, which is why it is used in every sector to develop web and desktop applications, AI, machine learning, and blockchain. So, we can develop user-friendly applications that are more useful to people.

In the upcoming chapter, you will learn the basic syntax of the Python programming language.

Key terms

- Robust
- Interpreted languages
- Web frameworks
- Django, CherryPy, Pyramid, TurboGears, Bottle, Flask
- Python Package Index
- Python libraries
- Anaconda installer
- GUI toolkits

Questions

1. Explain the features of Python.
2. Explain the real-world applications of Python.
3. What is Python, and who developed it?
4. Compare Python with C.
5. Python is written in which language?

Name the extension of Python files.. Is Python a purely object-oriented programming language?

CHAPTER 2

Basic Syntax

Introduction

To become an expert-level programmer, a learner must have a strong understanding of programming concepts. The knowledge a learner gets at the initial stages makes them an expert-level programmer later. This chapter focuses on the basic syntax of the Python programming language. Python is an interpreted language that works on popular OS platforms like Windows, Linux, and Mac, and it also works on container-based hostings like Docker and managed Kubernetes. Python has simple syntax like our English language. When compared to other programming languages, Python has a syntax that allows developers to code their programs with fewer lines. The programming language was actually designed for readability. Let's look at the basic syntax of the Python programming language.

Structure

We will cover the following topics in this chapter:

- Interactive mode programming
- Python program output and print statement
- Script mode programming

- Python identifiers
- Keywords
- Lines and indentation
- Multi-line statements
- Quotations used in Python
- Comments in Python
- Multiple statements on a single line

Objectives

The chapter focuses on helping you understand the basic syntax of the Python programming language. So, it helps you learn about the syntax used for specifying an identifier or a keyword and discusses how to handle multi-line statements or place comments in Python, and so on.

First Python program

Let's run the given programs in different methodologies of programming:

Interactive mode programming

The interpreter brings up the following prompt without passing a script file while invoking:

On Linux

1. `$ python`
2. Python **3.3.2** (default, Dec **10 2013**, **11:35:01**)
3. [GCC **4.6.3**] on Linux
4. Type `"help"`, `"copyright"`, `"credits"`,
or `"license"` for more information.

On Windows

1. Python **3.4.3** (v**3.4.3:9b73f1c3e601**,
Feb **24 2015**, **22:43:06**) [MSC v.**1600 32** bit (Intel)] on win32
2. Type `"copyright"`, `"credits"` or `"license()"` for more information.

Program output and print statement

Python uses the print statement to display output on the screen. Python contains the **printf()** function to display output to the console. Most shell script languages use the echo command for program output.

```
1. print ("Hello, Python!")
```

Output:

Hello, Python!

Note: The print statement, when paired with the string format operator (%), behaves even more like the **printf()** function in C:

```
1. print("%s is number %d" %("python",1))
```

Output:

python is number 1

Script mode programming

The execution of the script begins after the invocation of the interpreter with a script parameter and continues till the end of the script, leading to an inactive interpreter.

Let's write a simple Python program in the script. Python files support the **.py extension**. Type the following source code in a **test.py** file:

```
>>> print ("Hello, Python!")
```

We believe that you have the Python interpreter placed in the PATH variable. Now, try to run this program on different platforms, as follows:

On Linux

```
$ python test.py
```

Output

Hello, Python!

On Windows

```
C:\Python34>Python test.py
```

This produces the following result:

Hello, Python!

Let's try another approach to execute a Python script in Linux. Here's the modified **test.py** file:

1. `#!/usr/bin/python3`
2. `print ("Hello, Python!")`

We presume that you have a Python interpreter available in the `/usr/bin` directory. Now, try to run this program as follows:

```
$ chmod +x test.py      # This is to make file executable
$ ./test.py
```

This produces the following result:

Hello, Python!

Python identifiers

Identifier is the user-defined name given to entities like class, functions, variables, and so on in Python. It helps differentiate one entity from the others.

Rules for naming identifiers

The following list mentions the rules for naming identifiers in Python:

- An identifier begins with a letter A to Z or a to z or an underscore (`_`) suffixed by zero or more letters, underscores, and digits (0 to 9).
- An identifier should not begin with a digit.
- Keywords must not be used as identifiers.
- An identifier must not contain special characters, that is, `'.'`, `'$'`, `'!'`, `'@'`, `'#'`, `'%'`, etc.
- Uppercase letters are used to start class names, and all other identifiers start with a lowercase letter.
- Python is a case sensitive programming language i.e., it treats uppercase and lowercase differently.
- The identifier becomes private when an identifier is prefixed with a single leading underscore.
- The identifier is said to be a strong private identifier when it is prefixed with two leading underscores.
- When the identifier is suffixed with two trailing underscores, it becomes a language-defined special name.
- An identifier length must not exceed 79 characters.

For example: **Hello**, **_shape**, **d3_3d**, **Learn** are valid identifiers.

12house, **\$dollar**, **fa#n**, *while* are invalid identifiers.

Keywords

These are the unique words reserved for a specific purpose and must not be used as class names, variable names, function names, or any other identifiers. All the Python keywords comprise of lowercase letters only. *Table 2.1* lists some examples of keywords in Python:

and	exec	not
as	finally	or
assert	for	pass
break	from	print
class	global	raise
continue	if	return
def	import	try

Table 2.1: Examples of keywords in Python

Lines and indentation

Python constitutes block structure and nested block structure with indentation, not with beginning and end brackets.

The benefits of using indentation to specify structure are as follows:

- It reduces the requirement for a coding standard. The crucial point is that indentation is four spaces, and no hard tabs are required.
- It decreases irregularity. Code from various other sources follows the same approach of indentation.
- It diminishes work. The utmost need is to get the indentation corrected, not both the indentation and brackets.
- It reduces clutter as it eliminates all the curly brackets.
- If it looks correct, it is correct. Indentation cannot fool the reader.

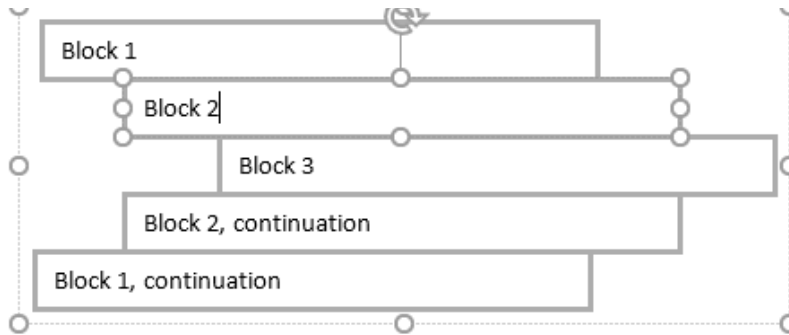


Figure 2.1: Flow of control in an indented statement

Multi-line statements

In Python, a new line character is used to indicate the end of the statement. However, it supports the use of the line continuation character (`\`) to denote that the line should continue. Consider this example:

```
1. total = item_one + \
2. item_two + \
3. item_three
```

The statements contained within the `[]`, `{}`, or `()` brackets need not use the line continuation character. Consider this example:

```
months = ['Jan', 'Feb', 'Mar',
          'Apr', 'May']
```

Quotations used in Python

Python supports single (`'`), double (`"`) and triple (`'''` or `"""`) quotes, indicates string literals, when the same type of quote is used to begin and terminate the given string.

The triple quotes are used to stretch the string across multiple lines. Look at this example:

```
1. Name = 'XYZ'
2. Fathurname = "ABC DEF"
3. About = """I love Python.
4. I can study it the whole day."""
5. print(Name)
```

```
6. print(Fathername)
print(About)
```

Output:

```
XYZ
ABC DEF
I love Python.
I can study it the whole day.
```

Note: The preceding program is in the editor, which is opened by clicking on 'File' and then 'New'. After writing it, click on 'Run' and 'run module' to see the output.

Comments in Python

As with most scripting and Unix-shell languages, the hash/pound (#) sign indicates that a comment begins right from the # and continues until the end of the line.

Here's an example:

```
1. #a is initialized to hello
2. a="hello"
3. #we will print hello now
4. print(a)
```

Output:

```
hello
```

Multiple statements on a single line

The semicolon (;) in Python allows multiple statements on a single line with a constraint that no statement starts a new code block. Here's an example that uses the semicolon:

```
a=5;    b = 'hello';    c = "This is how we write multi statements using
semicolon(;)"
```

Conclusion

Python can be treated as functional, procedural, or object-oriented. In this chapter, we learned about the basic syntax needed for writing code using Python. We also learned how to include multiple lines on a single line using a semicolon (;). We explored the rules used for naming an identifier in Python and understood the importance of including lines and indentation in Python.

In the next chapter, we will learn more about the types of variables used in Python. This will help decide which type of variables to use in various situations.

Questions

1. What is the importance of the Print function in Python?
2. What do you mean by interactive mode programming?
3. List some of the rules for naming identifiers in Python.
4. Explain with an example how to handle multiple line statements on a single line in Python.
5. Discuss the importance of lines and indentation in Python.

CHAPTER 3

Variable Types

Introduction

A variable is defined as a reserved memory location that holds a value that may change. It can be seen as a container to store specific values. Technically, it is a way of referring to a memory location used by a computer program and is a symbolic name for this physical location.

Based on the variable's data type, the interpreter usually allocates memory and decides what to store in the reserved memory.

Structure

We will cover the following topics in this chapter:

- Variables and assignment
- Multiple assignment
 - Built-in data types
 - Numbers
 - Strings

- Lists
- Tuples
- Dictionary
- Data type conversion

Objectives

This chapter focuses on how data is stored in a reserved memory location, assigning values to a variable and completing multiple assignments in Python. It also covers lists, tuples, and the dictionary used in the Python programming language. Additionally, it covers data type conversion in Python programming.

This chapter will help you understand the basics of variable types with the help of examples.

Variables and assignment

Python is dynamically typed, so pre-defining a variable or its type is not mandatory. The type (and value) is initialized on the assignment, and assignments are performed using the equals sign.

Here's an example:

```
1. rollno=123
2. name="XYZ"
3. marks=93.67
4. print("NAME is %s\nROLLNO is %d\nMARKS are %f" %(name,rollno,marks))
5. NAME is XYZ
6. ROLLNO is 123
7. MARKS are 93.670000
```

The string format operator (%) is displayed again at the end of the print statement.

Each %x given in the code matches the type of the argument to be printed. We have seen %s (for strings), %d (for integers), and %f (for floating-point values) as it is in C programming language.

Multiple assignments

Python supports the assignment of a single value to several variables simultaneously.

Consider this example:

```
a=b=c=1
```

Here, an integer object is created and assigned with the value one, and the same memory location is assigned to the three variables. Also, multiple objects can be assigned to multiple variables.

Here's an example:

```
a, b, c = 1, 2, "John"
```

Here, values 1 and 2 are assigned to two variables, a and b, whereas variable c is assigned with one string object with the value "John".

Built-in data types

These are the standard data types that are built into the interpreter:

- Numbers
- Strings
- List
- Tuple
- Dictionary

Numbers

The **number** data type stores numeric values. Number objects are created when we assign a numeric value.

```
a=5
```

```
b=27
```

The three different numerical types are as follows:

- int (signed integers)
- float (decimal point real values)
- complex (imaginary numbers)

Note: Here, in complex numbers, j is used with an imaginary part of complex numbers. It is expressed in the form of $a+bj$ where a and b are any real numbers.

All integers are represented as long integers, so there is no need for any separate long data type.

```
a=5
```

```
b=3.145
```

```
c=3+6j
```

Strings

Strings are identified as a continuous sequence of characters represented in single ('), double (" or ""), or triple (''' or """) quotes. There is no character data type in Python, and it is a string with a length equal to 1. Triple quotes are used when we write a string in multiple lines. Subsets of strings can be seized using the slice operator ([] or [:]). The index starts at 0 from the beginning and -1 from the end. The + sign is used for concatenation, and the * asterisk is used for repetition.

Example:

```
1. str = 'Let us study Python'
2. print (str)
3. print (str[0])
4. print (str[2:5])
5. print (str[2:])
6. print (str * 2)
7. print (str + "TEST")
```

Output:

```
Let us study Python
L
t u
t us study Python
Let us study PythonLet us study Python
Let us study PythonTEST
```

Lists

Like string, **list** is a collection. A string is a sequence of characters, not a collection by default, whereas a list can have values of any type. The values of a list are known as elements and are written in a list separated by commas in square brackets ([]).

```
List=[1,2.3,"list"]
```

The list is mutable, i.e., the value of the elements in it can be changed.

The values in the list are accessed using the slice operator ([] or [:]), just like we did in strings.

These are some ways of accessing the list:

```
1. list = [ 'abcd', 786 , 2.23, 'xyz', 70.2 ]
2. tinylist = [123, 'xyz']
```

```

3. print (list)
4. print (list[0])
5. print (list[1:3])
6. print (list[2:])
7. print (tinylis * 2)
8. print (list + tinylis)

```

Output:

```

['abcd', 786, 2.23, 'xyz', 70.2]
abcd
[786, 2.23]
[2.23, 'xyz', 70.2]
[123, 'xyz', 123, 'xyz']
['abcd', 786, 2.23, 'xyz', 70.2, 123, 'xyz']

```

Note: Here, the value 2.3 is given as 2.2999999999998 because the values in the interpreter are actually stored as double, which contain more number of bits as compared to float. So, the approximate double value is 2.3, and there is no difference.

Tuples

The **tuple** is another sequence similar to the list. The single difference between a list and a tuple is that a list is mutable and tuple is immutable, which indicates that the elements in the tuple cannot be manipulated. Tuples can be treated as read-only lists. One more difference is that tuples are written between parentheses (), while lists are written in brackets ([]). Tuples are accessed just like lists but cannot be updated.

These are the ways of accessing tuples:

```

1. tuple = ( 'abcd', 786 , 2.23, 'xyz', 70.2 )
2. tinytuple = (123, 'xyz')
3. print (tuple)
4. print (tuple[0])
5. print (tuple[1:3])
6. print (tuple[2:])
7. print (tinytuple * 2)
8. print (tuple + tinytuple)

```

Output:

```
('abcd', 786, 2.23, 'xyz', 70.2)
abcd
(786, 2.23)
(2.23, 'xyz', 70.2)
(123, 'xyz', 123, 'xyz')
('abcd', 786, 2.23, 'xyz', 70.2, 123, 'xyz')
```

A list is mutable and tuple is immutable; the following program illustrates this:

```
1. tuple = ( 'abcd', 786 , 2.23, 'xyz', 70.2 )
2. list = [ 'abcd', 786 , 2.23, 'xyz', 70.2 ]
3. list[2] = 1000
4. print(list)
5. tuple[2] = 1000
6. print(tuple)
```

Output:

```
['abcd', 786, 1000, 'xyz', 70.2]
```

Traceback (most recent call last):

```
File "C:/Python33/LISTxx.py", line 5, in <module>
    tuple[2] = 1000
```

TypeError: 'tuple' object does not support item assignment

Dictionary

A **dictionary** is said to be a set of key-value pairs whose values are accessed by keys. We can think of it as a list, but we have integral indices in a list, while a dictionary can have any type of indices, usually numbers or strings.

Dictionaries are enclosed in curly braces (`{}`) and accessed through square brackets (`[]`).

These are the ways of performing operations on the dictionary:

```
1. dict = {}
2. dict['three'] = "This is three"
3. dict[3]      = "This is three"
4. tinydict = {'name': 'ymbc', 'code': 3010, 'dept': 'education'}
5. print (dict['one'])
```

```

6. print (dict[2])
7. print (tinydict)
8. print (tinydict.keys())
9. print (tinydict.values())

```

The output is as follows:

```

This is threeThis is three
{'code': 3010, 'dept': 'education', 'name': 'ymbc'}
dict_keys(['code', 'dept', 'name'])
dict_values([3010, 'education', 'ymbc'])

```

Dictionaries have no order, and they are accessed by the keys provided. The keys inside the dictionaries are given values with colon (:) in between them.

Data type conversion

Sometimes, we need to convert the built-in types. For this, we use some built-in functions that convert one type to another, as shown in the following table:

Data Type	Explanation
<code>int(x [,base])</code>	Converts x to an integer and specifies the base if x is a string.
<code>float(x)</code>	Transforms x to a floating-point number.
<code>complex(real [,imag])</code>	Creates a complex number.
<code>str(x)</code>	Changes object x to a string representation.
<code>repr(x)</code>	Changes object x to an expression string.
<code>eval(str)</code>	Evaluates a string and returns an object.
<code>tuple(s)</code>	Transforms s to a tuple.
<code>list(s)</code>	Transforms s to a list.
<code>set(s)</code>	Changes s to a set.
<code>dict(d)</code>	Builds a dictionary, and the d must be a sequence of (key,value) tuples.
<code>frozenset(s)</code>	Changes s to a frozen set.
<code>chr(x)</code>	Changes an integer to a character.

Data Type	Explanation
<code>unichr(x)</code>	Transforms an integer to a Unicode character.
<code>ord(x)</code>	Transforms a single character to its integer value.
<code>hex(x)</code>	Transforms an integer to a hexadecimal string.
<code>oct(x)</code>	Transforms an integer to an octal string.

Table 3.1: List of data types

Conclusion

In simple terms, we can say that variables are just like containers with a name or label that is used to hold the value. A variable can take data in various formats, such as a string, an integer, a number with fractional parts (float), and so on. These formats, as mentioned earlier, are called data types. Here, we learned about variables and how to assign values to variables of different types. We also learned the different ways of converting a variable of a particular data type to another data type under data type conversion.

In the next chapter, we will learn about the basic operators used in the Python programming language. The use of basic operators is explained with the help of examples for easier understanding.

Questions

1. Define a variable.
2. Explain with an example how a value is allocated to a memory location when a variable is assigned with a value.
3. Explain the four basic data types used in Python programming language.
4. Explain data type conversion in Python.
5. What do you mean by an implicit conversion in Python?

CHAPTER 4

Basic Operators

Introduction

Python has special symbols for operators that perform arithmetic or logical operations. The value on which the operator operates is called an operand; for example, $A + B$, where 'A' and 'B' are operands and '+' is the operator that operates on the given operands. In this example, it is an addition operation. Several operators are used for performing different operations in Python.

Structure

We will focus on the following topics in this chapter:

- Arithmetic operators
- Relational operators
- Python assignment operators
- Bitwise operators
- Logical operators
- Membership operators

- Identity operators
- Operators precedence

Objectives

This chapter helps readers learn more about the different types of operators used in the Python programming language. A number of operators are available in Python, and each one is explained with an example. Additionally, the chapter focuses on making the concept easy to understand for learners.

Take a look at this example:

1. `2+3`

Here, `+` is the operator that performs the addition operation on the given operands `2` and `3`. The output of the operation is `5`.

Arithmetic operators

Operators are defined as constructs that can manipulate the value of operands. Arithmetic operators perform **addition**, **subtraction**, **multiplication**, **division**, **exponentiation**, and **modulus operations**.

Assume that `a = 25` and `b = 4`.

Operator	Description	Example
+Addition	Adds the values on both sides of the operator	<code>a + b = 29</code>
-Subtraction	Subtracts the operand on the right from the operand on the left	<code>a - b = 21</code>
* Multiplication	Multiplies the values on both sides of the operator	<code>a * b = 100</code>
/ Division	Divides the operand on the left by the operand on the right	<code>a / b = 6.25</code>
%Modulus	Divides the operand on the left by the operand on the right and returns the remainder	<code>a % b = 1</code>
**Exponent	Performs exponential(power) calculation on the operands	<code>a ** b = 25 to the power of 20</code>

//	Floor division: The division of operands, where the result is the quotient in which the digits after the decimal point are removed	$a // b = 6$
----	--	--------------

Table 4.1: Different arithmetic operators with their explanations

Consider this example:

```

1. a=5
2. b=3
3. c=a+b
4. print("addition of %d and %d is %d" %(a,b,c))
5. c=a-b
6. print("subtraction of %d and %d is %d" %(a,b,c))
7. c=a*b
8. print("multiplication of %d and %d is %d" %(a,b,c))
9. c=a/b
10. print("division of %d and %d is %f" %(a,b,c))
11. c=a%b
12. print("modulus of %d and %d is %d" %(a,b,c))
13. c=a**b
14. print("exponent of %d and %d is %d" %(a,b,c))
15. c=a//b
16. print("floor division of %d and %d is %d" %(a,b,c))

```

This is the output:

```

addition of 5 and 3 is 8
subtraction of 5 and 3 is 2
multiplication of 5 and 3 is 15
division of 5 and 3 is 1.666667
modulus of 5 and 3 is 2
exponent of 5 and 3 is 125
floor of 5 and 3 is 1

```

Relational operators

These operators compare the values on each of their two sides and decide the relation between them. They are also called comparison operators.

Assume that variable **a** holds the value five and variable **b** holds the value ten, then:

Operator	Explanation	Example
<code>==</code>	The condition becomes true when the values of two operands are equal	<code>(a == b)</code> is not true
<code>!=</code>	The condition becomes true when the values of the two operands are not equal	<code>(a != b)</code> is true
<code>></code>	The condition becomes true when the value of the operand on the left becomes greater than the value of the operand on the right	<code>(a > b)</code> is not true
<code><</code>	The condition becomes true when the value of the operand on the left becomes less than the value of the operand on the right	<code>(a < b)</code> is true
<code>>=</code>	The condition becomes true when the value of the operand on the left becomes greater than or equal to the value of the operand on the right	<code>(a >= b)</code> is not true
<code><=</code>	The condition becomes true when the value of the operand on the left becomes less than or equivalent to the value of the operand on the right	<code>(a <= b)</code> is true

Table: 4.1: Different relational operators with their explanations

Here's an example:

```

1.  a=5
2.  b=10
3.  if(a==b):
4.      print("a is equal to b")
5.  else:
6.      print("a is not equal to b")
7.  if(a!=b):

```

```
8.     print("a is not equal to b")
9.     else:
10.    print("a is equal to b")
11.    if(a<b):
12.        print("a is less to b")
13.    else:
14.        print("a is not less to b")
15.    if(a>b):
16.        print("a is greater to b")
17.    else:
18.        print("a is not greater to b")
19.    if(a<=b):
20.        print("a is either less than nor equal to b")
21.    else:
22.        print("a is neither less than nor equal to b")
23.    if(a>=b):
24.        print("a is either greater than nor equal to b")
25.    else:
26.        print("a is neither greater than nor equal to b")
```

This is the output:

a is not equal to b

a is not equal to b

a is less to b

a is not greater to b

a is either less than nor equal to b

a is neither greater than nor equal to b

Python assignment operators

Assignment operators are used in **Python to assign values to variables**. For example, `a = 5` is a simple assignment operator that assigns the value 5 on the right to the variable `a` on the left. There are various compound operators in Python, like `a += 5` that adds the value 5 to the variable `a` and later assigns the same to the variable `a`.

The following table lists the different assignment operators used in Python:

Operator	Explanation	Example
=	Assigns values from operands on the right side to those on the left side	<code>c = a + b</code> sets the value of <code>a + b</code> into <code>c</code>
<code>+=</code> Add AND	Adds the operand on the right-hand side to the operand on the left-hand side and assigns the result to the operand on the left-hand side	<code>c += a</code> is similar to <code>c = c + a</code>
<code>-=</code> Subtract AND	Subtracts the operand on the right-hand side from the operand on the left-hand side and stores the result in the operand on the left-hand side	<code>c -= a</code> is similar to <code>c = c - a</code>
<code>*=</code> Multiply AND	Multiplies the operand on the right-hand side with the operand on the left-hand side and stores the result in the operand on the left-hand side	<code>c *= a</code> is similar to <code>c = c * a</code>
<code>/=</code> Divide AND	Divides the operand on the left-hand side by the operand on the right-hand side and stores the result in the operand on the left-hand hand	<code>c /= a</code> is equivalent to <code>c = c / a</code> <code>c /= a</code> is equivalent to <code>c = c / a</code>
<code>%=</code> Modulus AND	Takes modulus using operands on both sides and assigns the remainder to the operand on the left-hand side	<code>c %= a</code> is similar to <code>c = c % a</code>
<code>**=</code> Exponent AND	Performs exponential (power) calculation using the operands on both sides and assigns the value to the operand on the left-hand side	<code>c **= a</code> is similar to <code>c = c ** a</code>
<code>//=</code> Floor Division	Performs floor division on the given operators and assigns the value to the operand on the left-hand side	<code>c //= a</code> is similar to <code>c = c // a</code>

Table: 4.2: Different assignment operators with their explanations

Consider an example where variables `a` and `b` hold the values 10 and 20, respectively; then:

1. `a=5`
2. `c=3`

```

3. c+=a
4. print("addition is %d" %(c))
5. c-=a
6. print("subtraction is %d" %(c))
7. c*=a
8. print("multiplication is %d" %(c))
9. c/=a
10. print("division is %f" %(c))
11. c%=a
12. print("modulus is %d" %(c))
13. c**=a
14. print("exponent is %d" %(c))
15. c//=a
16. print("floor division is %d" %(c))

```

Here's the output:

addition is 8

subtraction is 3

multiplication is 15

division is 3.000000

modulus is 3

exponent is 243

floor division is 48

Bitwise operators

The bitwise operator functions on bits and performs bit-by-bit operation. Let's take an example where a = 60 and b = 13. In binary format, they will be as follows:

```

1. a = 0011 1100
2. b = 0000 1101
3. -----
4. a&b = 0000 1100
5. a|b = 0011 1101

```

6. `a^b = 0011 0001`
7. `~a = 1100 0011`

Python's built-in **bin()** function is used to get the binary representation of an integer.

Operator	Explanation	Example
& Binary AND	If bit exists in both operands, then the operator copies a bit to the result as well	(a & b) (equals 0000 1100)
Binary OR	When the operator exists in each of two operands, it copies a bit	(a b) = 61 (equals 0011 1101)
^ Binary XOR	This copies the bit if it sets to one of the two operands only	(a ^ b) = 49 (means 0011 0001)
~ Binary One's Complement	It has the 'flipping' effect on bits, and it is unary in behavior	(~a) = -61 (equals 1100 0011 in 2's complement form due to a signed binary number)
<< Binary Left Shift	This moves the value of the left operand to the left by the number of bits specified by the right operand	a << = 240 (means 1111 0000)
>> Binary Right Shift	This moves the value of the left operand to the right by the number of bits specified by the right operand	a >> = 15 (means 0000 1111)

Table 4.3: Different bitwise operators with their explanations

Here's an example:

1. `a = 60` `# 60 = 0011 1100`
2. `b = 13` `# 13 = 0000 1101`
3. `print ('a=',a,': ',bin(a), 'b=',b,': ',bin(b))`
4. `c = 0`
5. `c = a & b;` `# 12 = 0000 1100`
6. `print ("result of AND is ", c,': ',bin(c))`
7. `c = a | b;` `# 61 = 0011 1101`
8. `print ("result of OR is ", c,': ',bin(c))`

```

9.  c = a ^ b;           # 49 = 0011 0001
10. print ("result of XOR is ", c,':',bin(c))

```

The output is as follows:

```

a= 60 : 0b111100 b= 13 : 0b1101
result of AND is  12 : 0b1100
result of OR is   61 : 0b111101
result of XOR is  49 : 0b110001
result of COMPLEMENT is -61 : -0b111101

```

Logical operators

The following are the logical operators supported by Python language. Assume that variable **a** holds the value True and variable **b** holds the value False.

Operator	Explanation	Example
and Logical AND	The condition becomes true when both the operands are true	(a and b) is False
or Logical OR	The condition becomes true when any of the two operands are non-zero	(a or b) equals True
not Logical NOT	It is used to alter the logical state of its operand	Not(a and b) equals True

Table: 4.4: Different logical operators with their explanations

Consider this example:

```

1. a=input("enter a( False or True ): ")
2. b=input("enter b( False or True ): ")
3. print("a AND b is ",a and b)
4. print("a OR b is ",a or b)
5. print("NOT a is ",not a)

```

This is the output:

```

enter a( False or True ): True
enter b( False or True ): True
a AND b is false
a OR b is true
NOT a is False

```

Membership operators

Python's membership operators are used to test the membership in a sequence, such as in strings, lists, or tuples. There are two membership operators:

Operator	Explanation	Example
In	Results to true if the specified sequence contains a variable, else results to false.	<code>x in y</code> If <code>x</code> is a member of sequence <code>y</code> , then 'in' results in a 1.
not in	Results to true if the specified sequence does not contain a variable, else results to false.	<code>x not in y</code> If <code>x</code> is not a member of sequence <code>y</code> , then 'not in' results in a 1.

Table: 4.5: Different membership operators with their explanations

Here's an example:

```
1. a=int(input("enter a number: "))
2. b=input("enter a character: ")
3. s1="hello python"
4. l1= [1,12.4,22,73,4.4,90]
5. if ( a in l1 ):
6.     print (a," is available in the given list")
7. else:
8.     print (a," is not available in the given list")
9. if ( b not in s1 ):
10.    print (b," is not available in the given string")
11. else:
12.    print (b," is available in the given string")
```

The output is as follows:

enter a number: 34

enter a character: e

34 is not available in the given list

e is available in the given string

Identity operators

These operators differentiate between the memory locations of two objects. There are two identity operators:

Operator	Explanation	Example
Is	Returns true if the variables on each of the two sides of the operator indicate the same object, else returns false	x is y If id(x) equals id(y), then here 'is' results in 1
is not	Returns false when the variables on each of the two sides of the operator indicate the same object, else returns true	x is not y When id(x) 'is not' equal to id(y), then is not results in 1

Table 4.6: Different identity operators with their explanations

Consider this example:

```

1. a = int(input("enter a: "))
2. b = int(input("enter b: "))
3. c = int(input("enter c: "))
4. if ( a is b ):
5.     print ("a and b have same identity")
6. else:
7.     print ("a and b do not have same identity")
8. if ( id(a) == id(b) ):
9.     print ("a and b have same identity")
10. else:
11.     print ("a and b do not have same identity")
12. if ( a is not c ):
13.     print ("a and c do not have same identity")
14. else:
15.     print ("a and c have same identity")

```

The output is as follows:

enter a: 2

enter b: 2

```
enter c: 3
a and b have same identity
a and b have same identity
a and c do not have same identity
```

Operators precedence

The following table describes all the operators from the highest precedence to the lowest:

Operators	Description
**	Exponentiation (raise to the power)
~ + -	Denotes complement and unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiplication, division, modulo, and floor division
+ -	Addition and subtraction
>><<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= <>=>	Comparison operators
== !=	Equality operators
= %= /= //=- += *= **=	Assignment operators
is not	Identity operators
in not in	Membership operators
not or and	Logical operators

Table 4.7: Operators precedence

Operator precedence affects the process of an expression's evaluation.

For example, $x = 5 + 3 * 2$; here, x is assigned to 11, not 16. This is because the operator multiplication (*) has higher precedence than addition (+), so it first multiplies $3 * 2$ and then adds 5.

Here, the operators at the top of the table have the highest precedence, and those at the bottom have the lowest precedence.

Here's an example:

```

1. a = 10
2. b = 15
3. c = 20
4. d = 25
5. ans = (a + b) * c / d
6. print ("Value of (a + b) * c / d is ", ans)
7. ans = ((a + b) * c) / d
8. print ("Value of ((a + b) * c) / d is ", ans)
9. ans = (a + b) * (c / d)
10. print ("Value of (a + b) * (c / d) is ", ans)
11. ans = a + (b * c) / d
12. print ("Value of a + (b * c) / d is ", ans)

```

The output is as follows:

```

Value of (a + b) * c / d is  20.0
Value of ((a + b) * c) / d is  20.0
Value of (a + b) * (c / d) is  20.0
Value of a + (b * c) / d is  22.0

```

Programs

1. Write a program to calculate the area and perimeter of a square:

```

1. a=int(input("enter side "))
2. area=a*a
3. peri=4*a
4. print("area of square is",area)
5. print("perimeter of square is",peri)

```

The output is as follows:

```
enter side 5  
area of square is 25  
perimeter of square is 20
```

2. Write a program to calculate the average of three numbers:

```
1. a=int(input("enter 1st number"))  
2. b=int(input("enter 2nd number"))  
3. c=int(input("enter 3rd number"))  
4. avg=(a+b+c)/3  
5. print("average is: ",avg)
```

This is the output:

```
enter 1st number34  
enter 2nd number12  
enter 3rd number22  
average is: 22.666666666666668
```

3. Write the code to calculate the area of a circle:

```
4. a=int(input("enter 1st number"))  
5. b=int(input("enter 2nd number"))  
6. c=int(input("enter 3rd number"))  
7. avg=(a+b+c)/3  
8. print("average is: ",avg)
```

Here's the output:

```
enter radius4  
area of circle is 50.24x
```

4. Write a program to calculate the square of a number:

```
5. a=int(input("enter a number"))  
6. sqr=a*a  
7. print("square of %d is %d"%(a,sqr))  
8.
```

The output is as follows:

```
enter a number4
square of 4 is 16
```

5. Write a program code to convert temperature from Fahrenheit to Celsius:

```
1. temp=input("enter temperature in Fahrenheit: ")
2. temp=float(temp)
3. cel=(temp-32)/1.8
4. print("Temperature in Celsius is :",cel)
```

This is the output:

```
enter temperature in Fahrenheit: 98
Temperature in Celsius is : 36.666666666666664
```

6. Write a program to swap two numbers using a third variable:

```
1. # to swap 2 numbers
2. a=int(input("Enter first number: "))
3. b=int(input("Enter second number: "))
4. print("Before swapping")
5. print("a=",a)
6. print("b=",b)
7. c=a
8. a=b
9. b=c
10.print("After swapping")
11.print("a=",a)
12.print("b=",b)
```

Here's the output:

```
Enter first number: 12
Enter second number: 60
Before swapping
a= 12
b= 60
After swapping
```

```
a= 60
```

```
b= 12
```

7. Write a program to swap two numbers without using a third variable:

```
1. a=int(input("Enter first number: "))
2. b=int(input("Enter second number: "))
3. print("Before swapping")
4. print("a=",a)
5. print("b=",b)
6. a=a+b
7. b=a-b
8. a=a-b
9. print("After swapping")
10. print("a=",a)
11. print("b=",b)
```

This gives the following output:

```
Enter first number: 23
```

```
Enter second number: 56
```

```
Before swapping
```

```
a= 23
```

```
b= 56
```

```
After swapping
```

```
a= 56
```

```
b= 23
```

8. Write a program to convert kilometers into miles:

```
1. # Program to convert kilometers into miles
2. km=float(input('Enter kilometers: '))
3. mul_value = 0.621371
4. miles = km * mul_value
5. print('%0.3f kilometers is equal to %0.3f miles' %(km,miles))
```

The output is as follows:

Enter kilometers: 34

34.000 kilometers is equal to 21.127 miles

Conclusion

In this chapter, we learned about the different types of operators, like arithmetic operators, relational operators, and bitwise operators. We also looked at examples of these operators for better understanding.

In the next chapter, we will cover decision-making statements.

Questions

1. Explain arithmetic operators in Python with examples.
2. What does operators precedence mean?
3. What is the importance of operators precedence in arithmetic operations?
4. Explain bitwise operators with examples.
5. Explain Python assignment operators with examples.
6. Compare identity operators and membership operators.

CHAPTER 5

Decision Making

Introduction

In a programming language, the decision-making statements determine the direction of the program execution flow.

The decision-making concept comes in a program when the program has conditional choices to execute code blocks. Let's take an example of traffic lights lit up at different conditions based on the road rules or any particular rule. The outcome that occur while executing gets evaluated with the result of either TRUE or FALSE, which is assumed as a non-zero or non-null value if it is TRUE and zero or null value if it is FALSE.

These are logical decisions, and Python provides decision-making statements to help make decisions within an application program based on the user requirements.

Statement	Description
if statement	An if statement contains a Boolean expression followed by one or more statements
if...else statements	An if statement can be suffixed by an optional else statement, which executes when the Boolean expression is FALSE
nested if statements	Python supports the use of one if or else if statement inside another if or else if statement(s)

Table 5.1: This table is all about condition statements in Python

Structure

We will focus on the following topics in this chapter:

- if statement
- if..else statement
- elif statements
- Nested if statements

Objectives

There are situations in our lives when we need to make a decision and take the next steps accordingly. Similar situations arise in a programming language, and we need to make decisions here as well. Based on the decision, we execute the next block of code.

if statement

In this, the statement starts with an **if** reserved word. The condition in the statement is a Boolean expression that determines whether or not the body will be executed. A colon (:) must follow the condition. The block is defined as one or more statements that are executed when the condition is true. The statement in the **if** block is a Boolean expression, which determines whether or not the block of statements will be executed.

Syntax:

If expression:

statement(s)

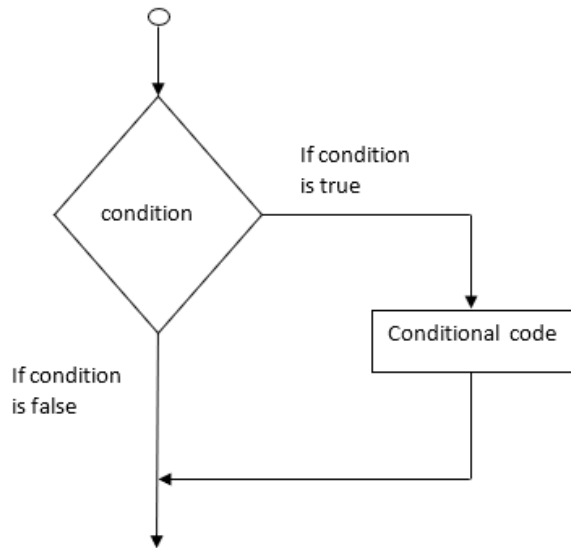


Figure 5.1: Flow diagram of the working flow of an if-statement

Here's an example:

```

1.  a=15
2.  b=5
3.  if (b!=0):
4.      c=a//b
5.      print ("The value of divisor is: ", c)
  
```

This is the output:

The value of divisor is: 3

if..else statement

With **if else** statements, **if** is the reserved word, and the code block under it is executed if its boolean expression evaluates to **TRUE**; otherwise, the code block under the **else** statement is executed.

Syntax:

If expression:

statement(s)

else:

statement(s)

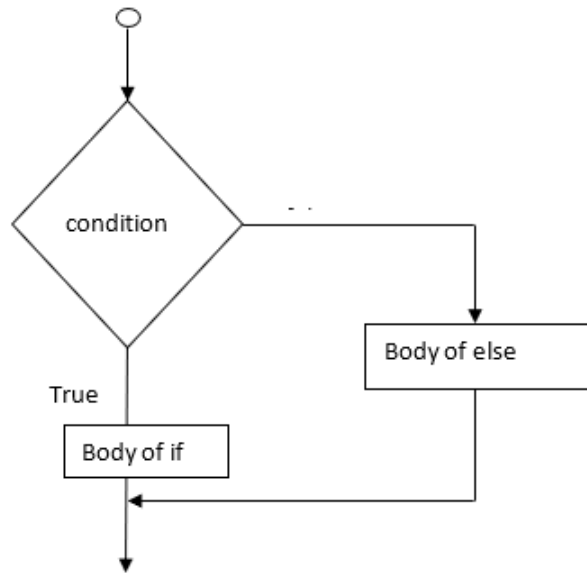


Figure 5.2: Flow diagram of the working flow of an if-else statement

Here's an example:

```
1. a=int(input("enter a number: "))
2. if(a%2==0):
3.     print(a,"is an even number")
4. else:
5.     print(a,"is an odd number")
```

The output is as follows:

```
enter a number: 34
34 is an even number
```

elif statements

The **elif** statement helps you evaluate multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

Syntax:

```
if expression1:
    statement(s)
elif expression2:
```

```
statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```

The following figure shows the working flow of an elif statement:

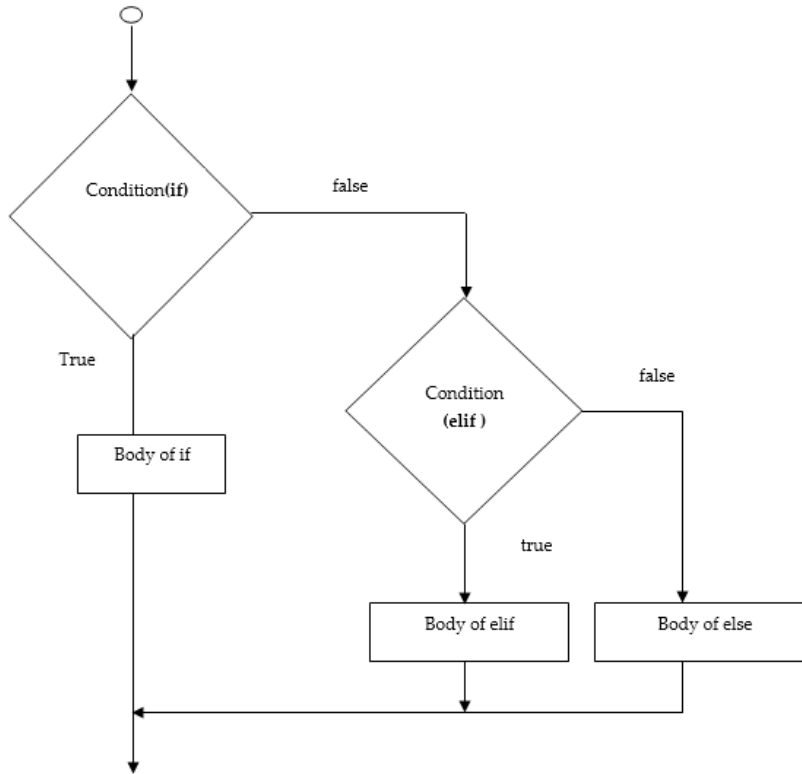


Figure 5.3: Flow diagram of the working flow of an elif-statement

Consider this example:

1. `a = int(input("enter a :"))`
2. `b = int(input("enter b: "))`
3. `if a > b:`
4. `print("a is greater")`
5. `elif a == b:`

```
6.     print("both are equal")
7.     else:
8.     print("b is greater")
```

It will give the following output:

```
enter a :22
enter b: 12
a is greater
```

Nested if statements

We require conditions inside conditions sometimes, and we use **if** under **if** or **if...elif...else** under **if...elif...else** or other nested conditions at such times.

Syntax:

```
if expression1:
    statement(s)
    if expression2:
        statement(s)
elif expression3:
    statement(s)
else
    statement(s)
elif expression4:
    statement(s)
else:
    statement(s)
```

Take a look at this example:

```
1.  if num%2==0:
2.      if num%3==0:
3.          print ("Divisible by 3 and 2")
4.      else:
5.          print ("divisible by 2 not divisible by 3")
6.  else:
7.      if num%3==0:
```

```

8.         print ("divisible by 3 not divisible by 2")
9.     else:
        print ("not Divisible by 2 not divisible by 3")

```

The output is as follows:

enter number 8

divisible by 2 not divisible by 3

Single statement suites

When the suite of an if clause contains a single line, it may go on to the same line as the header statement.

Here's an example:

```

1.  var = 100
2.  if ( var == 100 ) : print ("Value of expression is 100")
3.  print ("Good bye!")

```

This is the output:

Value of expression is 100

Good bye!

Programs

1. Program to check whether a number is positive or negative:

```

1.  n=int(input("enter a number: "))
2.  if n>0:
3.      print("number is positive")
4.  elif(n==0):
5.      print("number is zero")
6.  else:
7.      print("number is negative")

```

The output is as follows:

enter a number: 23

positive

2. Program to check whether a number is odd or even:

```
1. a=int(input("enter a number: "))
2. if(a%2==0):
3.     print(a,"is an even number")
4. else:
5.     print(a,"is an odd number")
```

Here's the output:

```
enter a number: 23
23 is an odd number
```

3. Program to check whether or not the year is a leap year:

```
1. year=int(input("Enter year to be checked: "))
2. if(year%4==0 and year%100!=0 or year%400==0):
3.     print("The year is a leap year.")
4. else:
5.     print("The year is not a leap year.")
```

It gives the following output:

```
Enter year to be checked: 2018
The year is not a leap year.
```

4. Program to find the greatest among two given numbers:

```
1. a=int(input("enter the first number: "))
2. b=int(input("enter the second number: "))
3. if a>b:
4.     print(a,"is greatest")
5. elif(a==b):
6.     print(b,"is greatest")
7. else:
8.     print(a,"and",b,"are equal")
```

The output is as follows:

```
enter the first number: 23
```

```
enter the second number: 12
23 is greatest
```

5. Program to find the greatest among three given numbers:

```
1. a=int(input("enter the first number: "))
2. b=int(input("enter the second number: "))
3. c=int(input("enter the third number: "))
4. if a==b==c:
5.     print(a,b,c ,"are equal")
6. else:
7.     if a>b and a>c:
8.         print(a,"is greatest")
9.     elif b>a and b>c:
10.        print(b,"is greatest")
11.    elif c>a and c>b:
12.        print(c,"is greatest")
```

Here's the output:

```
enter the first number: 23
enter the second number: 56
enter the third number: 44
56 is greatest
```

6. Program to input marks in 5 subjects and display the grade:

```
1. m1=int(input("enter marks in 1st subject: "))
2. m2=int(input("enter marks in 2nd subject: "))
3. m3=int(input("enter marks in 3rd subject: "))
4. m4=int(input("enter marks in 4th subject: "))
5. m5=int(input("enter marks in 5th subject: "))
6. avg=(m1+m2+m3+m4+m5)/5
7. print(avg)
8. if avg>=90:
9.     print("Grade A")
10. elif avg<90 and avg>=75:
```

```
11.     print("Grade B")
12. elif avg < 75 and avg >= 60:
13.     print("Grade C")
14. elif avg < 60 and avg >= 45:
15.     print("Grade D")
16. else:
17.     print("Grade fail")
```

The output is as follows:

```
enter marks in 1st subject: 89
enter marks in 2nd subject: 78
enter marks in 3rd subject: 60
enter marks in 4th subject: 90
enter marks in 5th subject: 87
80.8
```

Conclusion

The *if* statement executes a block of statements if a condition is true and does not execute them if the condition becomes false. But what if we want to perform some other actions if the condition is false. This is where the *else* statement comes in. The *else* statement with the *if* statement is mainly used to run a block of code whenever the condition becomes false. A user can decide multiple options, but the *if* statement is executed from top to bottom. If one of the conditions controlling the *if* condition is true, the statement associated with it is executed, and the *else* statements are bypassed. If none of the conditions is true, the final nested statement is executed.

In the next chapter, we will learn about repeating code using loops in Python.

Key terms

- Condition statement
- nested statement
- block of code
- elif
- decision making
- Syntax

CHAPTER 6

Repeating Code Using Loops

Introduction

Generally, statements execute in sequence. However, you may sometimes need to run a block of code several times, and you can use loops at such times.

Loops are statements that repeat an action over and over.

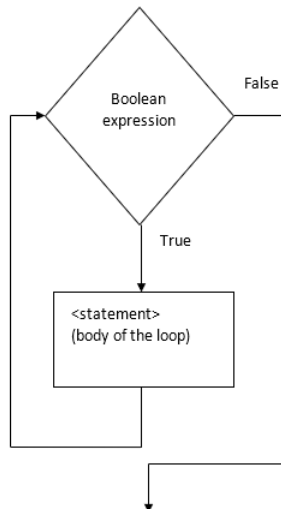


Figure 6.1: Flow of the primary loop

The control flow goes to the loop's body and executes the statement if the Boolean expression is True in the preceding figure; otherwise, it exits from the loop.

Structure

We will cover the following topics in this chapter:

- While loop
- For loop
- Nested loop
- Infinite loop

Objective

Python for loops iterate over an object until it is complete. For example, you can iterate over the contents of a list or a string. For loops use the syntax for an item in an object, where the object indicates the iterable over which you wish to iterate. You can repeat similar operations in your code by using loops. In this chapter, we will discuss looping concepts and working flow.

While loop

Python supports the **while** loop, which is used to iterate over a block of code as long as the test expression (condition) is true. We generally use this loop when we don't know the number of times to iterate beforehand.

Syntax:

```
while <Boolean expression>:    # as long as the expression evaluates to
    True
    <indented block of code>
```

Note that this syntax is parallel to an **if** statement. The **while** statement contains a Boolean expression followed by a colon, and there is an indented block of code after the **while** statement. The block of code is referred to as the body of the loop, and the **while** statement and the indented block together are called a while loop. The statements in the loop's body are repeated until the Boolean expression in the while statement evaluates to True. If the Boolean expression evaluates to False when executing the while statement, the execution continues with the next immediate statement after the body of the loop.

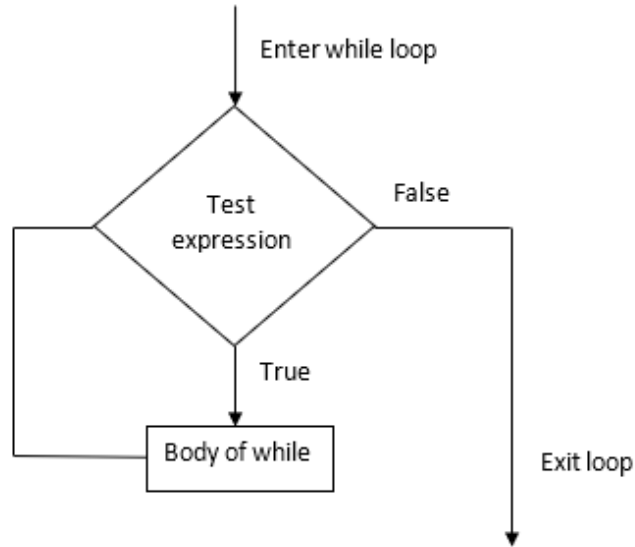


Figure 6.2: Block diagram of while loop

Example:

```
1. >>> n=1
2. >>> while(n<=5):
3.     print("Hello")
4.     n+=1
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

For loop

The **for** loop is a common iterator in Python. It can step through the items in an ordered sequence or other iterable objects. The **for** loop statement supports strings, lists, tuples, and other built-in iterables as well as new user-defined objects.

Syntax:

```
For iterating_var in sequence:
    Statements(s)
```

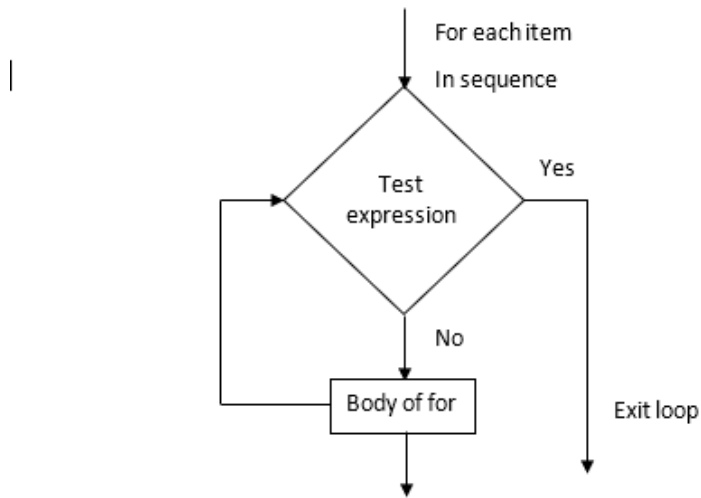


Figure 6.3: Block diagram of for loop

Example:

```

1. # List of numbers
2. numbers = [6,5,3,8,4,2,5,4,11]
3. # variable to store the sum
4. sum total = 0
5. # iterate over the list
6. for val in numbers:
7.     sum total = sumtotal+totalval
8. # print the sum print("The sum is",sumtotal)
9. print("sum is ",sumtotal)
  
```

Output:

sum is 48

Range() function

We can generate a sequence using the **range()** function. The range (10) will generate numbers from 0 to 9 (10 numbers). The common format of the range function call is as follows:

```
range( begin,end,step )
```

Here,

- **begin** is the initial value given in the range; the default value becomes zero if this is not included.
- **end** is the value that comes after the last value in the range; the end value is not deleted.
- **step** indicates the amount to increment or decrement; it defaults to 1 (increments by one) if the change parameter is omitted.

The values in **begin**, **end**, and **step** must be integer values; floating-point values and other types are not allowed.

Example:

```
1.  n=5
2.  for i in range(0,n+1):
3.      print("Hello")
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

In the preceding example, the loop will begin from 0 and go up till **n**. The **range()** function excludes the upper range, so we need to write **(n+1)** instead of **n**.

Break, continue, pass, and the loop else

In Python:

- **Break:** Allows you to jump out of the closest enclosing loop (past the entire loop statement)
- **Continue:** Jumps to the top of the closest enclosing loop (to the 'loop's header line)
- **Pass:** Does nothing at all; 'it's an empty statement placeholder
- **Loop else block:** Runs if the loop is exited normally (i.e., without hitting a **break**)

Break

Python supports the **break** statement to implement middle-exiting control logic. The **break** statement leads to an immediate exit from a loop.

Example:

```
1. n=1
2. while(n<=10):
3.     if(n==5):
4.         break
5.     print(n)
6.     n+=1
```

Output:

```
1
2
3
4
```

The **break** statement is used sparingly because it introduces an exception to the normal control logic of the loop. Preferably, every loop should have a single entry point and a single exit point. Programmers generally use **break** statements within **while** statements with conditions that are not always true. In such a **while** loop, including a **break** statement provides an additional exit point (the one point is the top of the loop where the condition is checked, and the **break** statement is another). Using multiple **break** statements within a single loop is particularly dubious and should be avoided.

Continue

Python supports the **continue** statement, which returns the control to the beginning of the current loop. When the *continue* is found, the loop starts the next iteration without executing the remaining statements in the current iteration. The **continue** statement is used in both **while** and **for** loops.

Example:

```
1. n=0
2. while(n<=10):
3.     n+=1
4.     if(n==5):
```

```
5.         continue
6.     print(n)
```

Output:

```
1
2
3
4
6
7
8
9
10
11
```

Pass

Python supports **pass**, which is a null statement. The interpreter differentiates a comment and passes by completely ignoring a comment and supporting *pass*. However, nothing happens when it is executed, and it results in no operation (NOP).

Pass statements are used when your code will eventually execute but has not been drafted yet, that is, in stubs. The usage of pass statements for stubs is an excellent example within Python.

Syntax:

```
pass
```

Example:

```
1. a="hello"
2. for i in range(len(a)):
3.     if a[i]=='e':
4.         print("this is a pass statement")
5.     print(a[i])
```

Output:

```
h
this is a pass statement
e
```

```
1
1
0
```

Loop else

Python supports the inclusion of an **else** statement in a loop statement.

If the **else** statement is included in a **for** loop in Python when the loop has exhausted iterating the list, the else statement is executed then.

If the **else** statement is included in a **while** loop, it is executed when the condition becomes false.

Example:

```
1.  n=1
2.  while(n<=5):
3.      print(n)
4.      n+=1
5.  else:
6.      print("hello")
```

Output:

```
1
2
3
4
5
hello
```

Nested loops

Python programming language supports the use of one loop inside another.

Syntax:

```
For iterating_var in sequence:
```

```
    For iterating_var in sequence:
```

```
        Statements(s)
```

```
    Statements(s)
```

Example:

```

1. i=1
2. for i in range(0,5):
3.     for j in range(0,5):
4.         print("*",end=" ")
5.         print()

```

Output:

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

```

The **print()** function's inner loop includes `end = ' '`, which adds a space rather than a default new line. Hence, the numbers will pop up in one row.

The last **print()** executes at the end of the inner for loop.

Infinite loop

An infinite loop exists when the condition in the loop never becomes FALSE. You must be careful when using while loops as there is a risk that this condition never resolves to a FALSE value, which leads to a situation where the loop never ends. Such a loop is an infinite loop.

Example:

```

1. n=1
2. while(n<=5):    #this constructs an infinite loop
3.     a=int(input("Enter a number"))
4.     print("you entered ",a)

```

Output:

```

Enter a number12
you entered  12
Enter a number34
you entered  34
Enter a number12
you entered  12

```

```
Enter a number34
you entered 34
Enter a number55
you entered 55
Enter a number
Traceback (most recent call last):
  File "infinite_loop.py", line 3, in <module>
    a=int(input("Enter a number"))
KeyboardInterrupt
```

The preceding example leads to an infinite loop, and you need to manually exit the program using `CTRL + C` or operating system exit controls.

Iterator and generator

An **iterator** is an object that permits any programmer to visit all the elements of a collection, regardless of its specific implementation. Python allows an iterator object to implement two methods: **iter()** and **next()**.

String, list, or tuple objects are used to create an iterator.

A generator is a function that uses the *yield* method to produce or yield a series of values.

The **generator** function returns a generator object without even starting execution. When the **next()** method is called for the first time, the function begins executing until the yield statement is reached, which returns the yielded value. The **next()** method remembers the last execution due to this, so the second execution proceeds from the last execution.

Iterating examples

1. Program to find factorial of a number:

```
1. n=int(input("Enter a number: "))
2. fact=1
3. for i in range(1,n+1):
4.     fact=fact*i
5. print("Factorial of %d is %d"%(n,fact))
```

Output:

```
Enter a number: 5
Factorial of 5 is 120
```

2. Program to input a number and print its table:

```
1. num=int(input("Enter a number: "))
2. l=int(input("Enter limit: "))
3. print("Table of ",num)
4. for i in range(1,l+1):
5.     print("%d X %d = %d"%(num,i,num*i))
```

Output:

Enter a number: 4

Enter limit: 10

Table of 4

```
4 X 1 = 4
4 X 2 = 8
4 X 3 = 12
4 X 4 = 16
4 X 5 = 20
4 X 6 = 24
4 X 7 = 28
4 X 8 = 32
4 X 9 = 36
4 X 10 = 40
```

3. Program to determine whether a number is prime:

```
1. n=int(input("Enter a number: "))
2. flag=0
3. for i in range(2,(n//2)+1):
4.     if(n%i==0):
5.         flag=1
6.         break
7. if(flag==0):
8.     print("%d is a Prime number"%(n))
9. else:
10.    print("%d is not a Prime number"%(n))
```

Output:

```
Enter a number: 23
23 is a Prime number
```

4. Program to print all prime numbers between two numbers m and n:

```
1. m=int(input("Enter the lower limit: "))
2. n=int(input("Enter the upper limit: "))
3. print("Prime numbers between %d and %d are:"%(m,n))
4. for i in range(m,n+1):
5.     flag=0
6.     for j in range(2,(i//2)+1):
7.         if(i%j==0):
8.             flag=1
9.             break
10.    if(flag==0):
11.        print(i)
```

Output:

```
Enter the lower limit: 2
Enter the upper limit: 23
Prime numbers between 2 and 23 are:
2
3
5
7
11
13
17
19
23
```

5. Program to determine whether the number is Armstrong:

```
1. a=input("Enter a number: ")
```

```
2. l=len(a)
3. a=int(a)
4. orig=a
5. sum=0
6. while(a>0):
7.     temp=a%10
8.     sum=sum+(temp**l)
9.     a=a//10
10. if(sum==orig):
11.     print("Armstrong number")
12. else:
13.     print("Not an Armstrong number")
```

Output:

Enter a number: 153

Armstrong number

6. Program to print the Fibonacci series:

```
1. a=int(input("Enter a number"))
2. f=0
3. s=1
4. i=1
5. print("The Fibonacci series is")
6. print(f,end=" ")
7. print(s,end=" ")
8. while(i<=(a-2)):
9.     t=f+s
10.    print(t,end=" ")
11.    f=s
12.    s=t
13.    i+=1
```

Output:

```
Enter a number 5
The Fibonacci series is
0 1 1 2 3
```

7. Code to print the sum of the digits of a number:

```
1. a=int(input("Enter a number :"))
2. sum=0
3. while(a>0):
4.     temp=a%10
5.     sum=sum+temp
6.     a=a//10
7. print("Sum of the digits is ",sum)
```

Output:

```
Enter a number :345
Sum of the digits is 12
```

8. Program to determine whether a number is a palindrome:

```
1. a=int(input("Enter a number :"))
2. orig=a
3. sum=0
4. while(a>0):
5.     temp=a%10
6.     sum=sum*10+temp
7.     a=a//10
8. if(sum==orig):
9.     print("Palindrome")
10. else:
11.     print("Not Palindrome")
```

Output:

```
Enter a number :343
Palindrome
```

9. Program to print the following pattern:

```
*  
* *  
* * *  
* * * *
```

```
1. for i in range(1,5):  
2.     for j in range(1,i+1):  
3.         print("*",end=" ")  
4.     print()
```

10. Program to print this pattern:

```
1  
2 2  
3 3 3  
4 4 4 4  
5 5 5 5 5
```

```
1. i=1  
2. for i in range(0,5+1):  
3.     for j in range(0,i):  
4.         print(i,end=" ")  
5.     print()
```

11. Program to print the pattern as follows:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

```
1. i=1  
2. for i in range(1,5+1):  
3.     for j in range(1,i+1):  
4.         print(j,end=" ")  
5.     print()
```

12. Program to print the following pattern:

```
*
**
***
****
*****
****
***
**
          *
```

```
1. n=5
2. for i in range(n):
3.     for j in range(i):
4.         print('*',end=" ")
5.     print()
6. for i in range(n,0,-1):
7.     for j in range(i):
8.         print('*',end=" ")
9.     print()
```

13. Program to print the pattern shown here:

```
*
* * *
* * * * *
* * * * * * *
```

```
1. for i in range(1,5+1):
2.     for k in range(1,5-i+1):
3.         print(end=" ")
4.         for j in range(1,2*i):
5.             print("*",end=" ")
6.         print()
```

14. Program to print this pattern:

```
1
23
456
78910
```

```
1. k=1
2. for i in range(1,5):
3.     for j in range(1,i+1):
4.         print(k,end="")
5.         k+=1
6.     print()
```

15. Program to print the Fibonacci series using iterator and generator:

```
1. import sys
2. def Fibonacci(n): #generator function
3.     a, b, counter = 0, 1, 0
4.     while True:
5.         if (counter > n):
6.             return
7.         yield a
8.         a, b = b, a + b
9.         counter += 1
10. f = Fibonacci(5) #f is iterator object
11. while True:
12.     try:
13.         print (next(f), end=" ")
14.     except Stop Iteration:
15.         sys.exit()
```

Output:

```
0 1 1 2 3 5
```

Conclusion

The manual executes instructions until it gets the accurate outcome, using loops to provide a shortcut to minimize the code, increase its readability, and reduce complexity. Loops and statements hold various components in any programming language. Loops help alter the conventional flow of the program, execute specific sets of instructions with developer-specified conditions, and produce optimal results. The working of a loop depends upon the condition specified. In loops, a group of instructions tends to repeat itself until the termination condition occurs. After that, the program revises the already devised relevant action. So, loops provide an opportunity to bring similar executable instructions together. This technique makes the code reusable for the same tasks.

In the next chapter, we will look at numbers in Python.

Multiple Choice Questions

1. What will be the output of the following Python program?

```
For(I in range(0,2,-1))
    Print("Hello")
```

- A. No output
- B. Hello
- C. Hello Hello
- D. Error

2. Which type of iteration is carried out in Python with a *while* loop?

- A. Definite
- B. Indefinite
- C. Discriminant
- D. Indeterminate

3. Predict the output of the following code:

```
value = "abcdef"
i = "i"
while i in value:
    print(i, end=" ")
```

- A. iiii
- B. a b c d e f

- C. abcdef
- D. No output

4. Python for loop is:

- A. Entry Control Loop
- B. Exit Control loop
- C. Simple Loop
- D. None of the above

5. Which of the following is an Infinite loop

- A. While(1)
- B. While(infinite)
- C. For(1)
- D. None of the above

CHAPTER 7

Numbers

Introduction

Number data types store numeric values and are used in almost every program. Favorably, Python has several different types of numbers to fit in all your game or application programming needs.

Structure

We will cover the following topics in this chapter:

- Types of numbers
- Type conversions
- Mathematical functions

Objectives

We will discuss the different types of numbers used in Python, learn how to convert one type into another, and understand the mathematical operations supported by Python.

Types of numbers

The three types of numbers are:

- Integers
- Floating-point numbers (or floats)
- Complex numbers

Integers (signed integers): These are whole numbers or numbers with no fractional part. Another way to think about them is that they can be written without a decimal point. Python 3 supports integers of unconditional sizes but does not support 'long integers'. Python 2 has the *int* and *long* integer types. The numbers 1, 27, -100, and 0 are all examples of integers.

Float (floating point real values): They are also known as floats and mean real numbers. They are written with a decimal point that divides the integer and fractional parts. Floats may also be represented in scientific notation, with E or e that indicates the power of 10 ($3.5e2 = 3.5 \times 10^2 = 350$). Numbers like 2.376, -99.1, 1.0, and -32.54e100 are examples of floats.

Complex (complex numbers): Complex numbers are of the form $x + yZ$, where x and y are floats, and Z (or z) represents the square root of -1 (which is an imaginary number). The real part of the number is x , and the imaginary part is y . Python rarely uses complex numbers. Some examples are 3.14z, -.6545+0z, and $3e+26Z$.

Numbers are immutable (unchangeable) values that support numeric operations. This implies that changing the value of a number of particular data type results to a new number. Number type conversion

Python internally converts the numbers in expressions from mixed types to a common type for evaluation. In order to satisfy the requirements of an operator or function parameter, you sometimes need to explicitly convert a number from one type to another.

- Type **int(m)** to convert m to a plain integer.
- Type **long(m)** to convert m to a long integer.
- Type **float(m)** to convert m to a floating-point number.
- Type **complex(m)** to convert m to a complex number with real part m and imaginary part zero.
- Type **complex(m, n)** to convert m and n to a complex number with real part m and imaginary part n , where m and n are numeric expressions.

Mathematical functions

Python programming contains a `math` module that provides most of the familiar mathematical functions to implement. We have to import the module to use it in programming:

```
import math
```

To access any of the available functions, you must provide the module name and the function name separated by a dot (also known as a period). This format is called dot notation.

```
Math.function_name
```

Number `abs()` method

The **`abs()`** method gives the absolute value of **`x`**, that is, the positive distance from **`x`** to zero, as output.

Syntax

The **`abs()`** method has the following syntax:

```
abs(x)
```

Parameters

`x` - Indicates a numeric expression.

Return value

This method provides output as the absolute value of **`x`**.

Example

```
1. >>> print(abs(34.55))
2. 34.55
3. >>> print(abs(45))
4. 45
5. >>> print(abs(-467))
6. 467
7. >>> print(abs(-33.55))
8. 33.55
```

Number `ceil()` method

The **`ceil()`** method provides output as the ceiling value of **`x`**, that is, the smallest integer, which should not be less than **`x`**.

Syntax

The syntax for the **`ceil()`** method is given as follows:

```
import math.ceil(m)
math.ceil
```

Note: The `ceil()` function is not directly accessible. In order to access the `ceil()`, we must import the `math` module and then use the `math` static object to call this function.

Parameters

`m` – This refers to a numeric expression.

Return value

This method gives an output of the smallest integer not less than **`m`**.

Example

The following example explains the usage of the **`ceil()`** method:

```
1. >>> import math
2. >>> print(math.ceil(12.23))
3. 13
4. >>> print(math.ceil(-34.33))
5. -34
6. >>> print(math.ceil(278.555))
7. 279
```

Number `exp()` method

The **`exp()`** method gives the output as **`e`** raised to the power of **`m`**.

Syntax

The syntax for the **`exp()`** method is as follows:

```
import math
math.exp( m )
```

Note: The `exp()` function is only accessible using the `exp` static module object.

Parameters

m - This is a numeric expression.

Example

The use of **`exp()`** is shown as follows:

```
1. >>> print(math.exp(45))
2. 3.4934271057485095e+19
3. >>> print(math.exp(-45.17))
4. 2.4150062132629406e-20
5. >>> print(math.exp(math.pi))
6. 23.140692632779267
```

Number **`fabs()`** method

The **`fabs()`** method provides the output of the absolute value of float **m**. It seems similar to the **`abs()`** function, but there are some differences between the two:

- The **`abs()`** method is a built-in function, whereas **`fabs()`** is defined in the `math` module.
- The **`fabs()`** function only takes on float and integer values, whereas **`abs()`** works with complex numbers as well.

Syntax

The following is the syntax for the **`fabs()`** method:

```
import math
math.fabs( m )
```

Note: The **`fabs()`** function is only accessible using the **`fabs`** static module object.

Parameters

m - This is a numeric value.

Return value

This method returns the absolute value of **m**.

Example

The use of the **`fabs()`** function is shown here:

```
1. >>> import math
2. >>> print( math.fabs(34.67))
3. 34.67
4. >>> print(math.fabs(-34.4))
5. 34.4
6. >>> print(math.fabs(-34))
7. 34.0
8. >>> math.fabs(math.pi)
9. 3.141592653589793
```

Number floor() method

The floor() method returns the floor of m as an integer, that is, the largest integer not greater than m.

Syntax

The following is the syntax for the **floor()** method:

```
import math
math.floor( m )
```

Note: The floor() function is only accessible using the floor static module object.

Parameters

m - This is a numeric expression.

Example

```
1. >>> print(math.floor(22.67))
2. 22
3. >>> print(math.floor(39.9))
4. 39
5. >>> print(math.floor(-36.9))
6. -37
7. >>> print(math.floor(-56.2))
8. -57
```

Number log() method

The **log()** function returns the logarithm of **x** to the given base. If the base is not specified, it returns the natural logarithm (base e) of m.

Syntax

The following is the syntax for the **log()** method:

```
import math
math.log( m )
```

Note: The log() function is only accessible using the log static module object.

Parameters

m - This is a numeric expression.

Return value

This method returns the natural logarithm of m, for $m > 0$.

Example

The **log()** method is used as follows:

```
1. >>> print(math.log(22.67))
2. 3.1210424645194377
3. >>> print(math.log(100.45))
4. 4.609660091260944
5. >>> print(math.log(math.pi))
6. 1.1447298858494002
```

Number log10() method

The **log10()** method returns base-10 logarithm of m for $m > 0$.

Syntax

Here's the syntax for the log10() method:

```
import math
math.log10( m )
```

Note: The log10() function is only accessible using the log10 static module object.

Parameters

m - This is a numeric expression.

Return value

This method returns the base-10 logarithm of m for $m > 0$.

Example

The use of the `log10()` method is shown here:

```
1. >>> print(math.log10(22.67))
2. 1.3554515201265174
3. >>> print(math.log10(100.45))
4. 2.001949941084268
5. >>> print(math.log10(math.pi))
6. 0.49714987269413385
```

Number `modf()` method

The `modf()` method outputs the fractional and integer parts of m in a two-item tuple. Both parts have the same sign—m. The output of the integer part is a float.

Syntax

The following is the syntax for the `modf()` method:

```
import math
math.modf( m)
```

Note: The `modf()` function is only accessible using the `modf` static module object.

Parameters

m - This is a numeric expression.

Example

The following example explains the use of the `modf()` method:

```
1. >>> print(math.modf(34.56))
2. (0.5600000000000023, 34.0)
3. >>> print(math.modf(100))
4. (0.0, 100.0)
5. >>> print(math.modf(math.pi))
```

```

6. (0.14159265358979312, 3.0)
7. >>> print(math.modf(-67.54))
8. (-0.5400000000000063, -67.0)

```

Number pow() method

The **pow()** function in Python returns the power of a given number. It computes **u**v**. In this function, the arguments are converted to floats before the power is calculated.

Syntax

```
pow(u, v, w)
```

Return value

This method provides the value of **u**v** (with two arguments) or **u**v % w** (with three arguments).

Example

The use of the **pow()** method is as follows:

```

1. >>> print(pow(4,3))
2. 64
3. >>> print(pow(2,7))
4. 128
5. >>> print(pow(5,5,3))
6. 2
7. >>> print(pow(6,7,4))
8. 0

```

Number sqrt() method

The **sqrt()** method gives the square root of **m** for $m > 0$.

Syntax

Here's the syntax for the **sqrt()** method:

```
import math
math.sqrt(m)
```

Note: The **sqrt()** function is only accessible using the **sqrt** static module object.

Parameters

m - This is a numeric expression.

Example

The use of the **sqrt()** method is shown here:

```
1. >>> print(math.sqrt(16))
2. 4.0
3. >>> print(math.sqrt(23))
4. 4.795831523312719
5. >>> print(math.sqrt(math.pi))
6. 1.7724538509055159
```

Number round() method

The **round()** method is a built-in function in Python. It rounds a number to a given precision in decimal digits (default 0 digits). This returns an int when called with one argument; otherwise, it returns the same type as the number. Additionally, n-digits may be negative.

Syntax

The following is the syntax for the **round()** method:

```
round( m, n )
```

Parameters

m - It indicates an expression that must be numeric.

n – It indicates the number of digits from the decimal point up to which m is to be rounded. The default is 0.

Return value

This method provides m rounded to n digits from the decimal point.

Example

The use of the **round()** method is shown as follows:

```
1. >>> round(55.7)
2. 56
3. >>> round(23.345,2)
4. 23.34
```

```
5. >>> round(78.23, -2)
6. 100.0
7. >>> round(18.23, -2)
8. 0.0
9. >>> round(25.234, -1)
10. 30.0
11. >>> round(23.345, 3)
12. 23.345
13. >>> round(23.345, 1)
14. 23.3
15. >>> round(23.345, 0)
16. 23.0
```

Random number functions

Python supports random numbers. Many applications, such as games, simulations, testing, security, and privacy applications, use random numbers.

Number choice() method

The **choice()** method outputs a random item from a list, tuple, or string.

Syntax

The following is the syntax for the **choice()** method:

```
import random
random.choice(sq)
```

Note: The **choice()** function is only accessible using the **choice** static module object.

Parameters

sq – It could be a list, tuple, or string.

Return value

This method provides a random item as an output.

Example

The following example shows the usage of the **choice()** method:

```
1. >>> a="hello world"
2. >>>random.choice(a)
3. 'l'
4. >>>random.choice((2,4,5,6,7,1))
5. 2
6. >>>random.choice(["ram","rom","prom",45,22])
7. 'prom'
8. >>>random.choice("hello")
9. 'e'
```

This function doesn't work for sets because sets do not have indices.

Number randrange() method

The **randrange()** method gives a randomly selected element from range (**start, stop, step**).

Syntax

```
randrange ([start,] stop [,step])
```

Note: The randrange() method is not directly accessible. In order to access the randrange(), we must import the random module and use the random module object to call this function.

Parameters

start -Start point of the range. This would be included in the range; the default is 0.

stop -Stop point of the range. This would be excluded from the range.

step -Value with which the number is incremented. The default is 1.

Example

```
1. >>> import random
2. >>> print ("randrange(1,100, 2) : ", random.randrange(1, 100, 2))
3. randrange(1,100, 2) :  11
4. >>> print ("randrange(100) : ", random.randrange(100))
5. randrange(100) :  86
```

Number random() method

The **random()** method provides a random floating point number in the range **[0.0, 1.0]** as output.

Syntax

```
random ( )
```

Note: The **random()** method is not accessible directly. In order to access it, we must import the random module and then use the random static object to call this function.

Parameters

start – indicates the starting position of the range. The range includes the start; the default value is zero.

stop – indicates the stop position of the range. The given range doesn't include a stop.

step – indicates the value with which the number is incremented. The default value is one.

Result

This method provides a random item from the given range as the output.

Example

```
1. >>> import random
2. >>> print ("random() : ", random.random())
3. random() :  0.9279919237961513
4. >>> print ("random() : ", random.random())
5. random() :  0.9800448891204537
```

Number seed() method

The **seed()** method loads the basic random number generator. You must call this function before calling any other random module function.

Syntax

```
seed ([x], [y])
```

Note: The **seed()** function initializes the basic random number generator.

Parameters

The next random number will be generated by this seed. When omitted, the next random number has to be generated by the system; X is used directly if it is an integer.

The version number defaults to 2. A string, byte, or byte array object will be converted to an integer if this value is set. Version 1 uses **hash()** of x.

Result

This method does not return any value.

Example

```
1. import random iop
2. random.seed()
3. print ("random number with default seed 11", random.random())
4. random.seed(110)
5. print ("random number with int seed", random.random())
6. random.seed("hello",2) {when seeded with a string "hello", the
    hash() of the string is used as the seed value during randomization.}
7. print ("random numbers with string seed", random.random())
```

Output

```
random number with default seed 0.2524977842762465
random number with int seed 0.5714025946899135
random number with string seed 0.3537754404730722
```

Number shuffle() method

The **shuffle()** function randomizes the items of a list in place.

Syntax

```
shuffle (lst,[random])
```

Note: The **shuffle()** method is only accessible using the **shuffle** static module object.

Parameters

lst - indicates a list or a tuple.

random - indicates an optional 0 argument function returning float between 0.0 - 1.0; the default value is none.

Result

This method returns the reshuffled list.

Example

```

1. import random
2. number_list = [7,14,21, 28, 35, 42, 49, 56, 63, 70]
3. random.shuffle(number_list)
4.

```

Output

14, 21, 70, 28, 49, 35, 56, 7, 42, 63

Number uniform() method

The **uniform()** method provides as output a random float r , such that $m \leq r$ and $r < n$.

Syntax

`uniform(m, n)`

Note: The **uniform()** method is only accessible using the **uniform** static module object.

Parameters

x - fixes the lower limit of the random float

y - fixes the upper limit of the random float

Result

This method provides as output a floating-point number r , such that $x \leq r < y$.

Example

```

1. import random
2. >>> print ("Random Float uniform(5, 10) :", random.uniform(5, 10))
3. Random Float uniform(5, 10) : 8.315992905134308

```

Trigonometric functions

Python includes the following functions that perform trigonometric calculations:

Function	Description
acos(x)	Provides the arc cosine of x in radians
asin(x)	Provides the arc sine of x in radians
atan(x)	Provides the arc tangent of x in radians
atan2(y, x)	Returns atan(y / x) in radians
cos(x)	Returns the cosine of x radians
hypot(x, y)	Provides the Euclidean norm, $\sqrt{x^2 + y^2}$
sin(x)	Returns the sine of x radians
tan(x)	Returns the tangent of x radians
degrees(x)	Converts angle x from radians to degrees
radians(x)	Converts angle x from degrees to radians

Table 7.1: Trigonometric functions

Number acos() method

The **acos()** method provides the arc cosine of x (in radians) as output.

Syntax

`acos(x)`

Note: The `acos()` method is only accessible using the `acos` static module method.

Parameters

x - Indicates a numeric value that comes in the range -1 to 1. If $x > 1$, it will develop 'math domain error'.

Example

```

1. import math
2. print ("acos(0.46) : ", math.acos(0.64))
3. print ("acos(0) : ", math.acos(0))

```

Output

```
acos(0.46) : 0.8762980611683406
```

```
acos(0) : 1.5707963267948966
```

Number asin() method

The **asin()** method provides the arc sine of **x** (in radians) as output.

Syntax

```
asin(x)
```

Note: The **asin()** method is only accessible using the **asin** static module object.
Parameters:

x – indicates a numeric value ranges from -1 to 1. If $x > 1$, it will produce ‘math domain error’.

Example

```
1. import math
2. print ("asin(0.46) : ", math.asin(0.64))
3. print ("asin(-2) : ", math.asin(-1))
4. Output
5. asin(0.46) : 0.694498265626556
6. asin(-2) : -1.5707963267948966
```

Number atan() method

The **atan()** method provides the arc tangent of **x** (in radians) as output.

Syntax

```
atan(x)
```

Note: The **atan()** method is only accessible using the **atan** static module object.

Parameters

x – indicates a numeric value that ranges from -1 to 1. If $x > 1$, it will produce ‘math domain error’.

Example

```
1. import math
2. >>> print ("atan(0) : ", math.atan(0))
3. atan(0) : 0.0
```

Number atan2() method

The **atan2()** method provides **atan(y / x)** (in radians) as output.

Syntax

`atan2(y, x)`

Note: The `atan2()` method is only accessible using the `atan2` static module object.

Parameters

y – indicates a value that must be a numeric value.

x – indicates a value that must be a numeric value.

Example

```
1. import math
2. print ("atan2(-1.50,-1.50) : ", math.atan2(-0.50,-0.50))
3.
4. Output
5. atan2(-1.50,-1.50) : -2.356194490192345
```

Number cos() method

The **cos()** method provides the cosine of x (in radians) as output.

Syntax

`cos(m)`

Note: The `cos()` method is only accessible using the `cos` static module object.

Parameters

m - This indicates a value that must be numeric.

Return value

This method provides a numeric value that ranges from -1 and 1, and it represents the cosine of the angle.

Example

```
1. >>> import math
2. >>> print ("cos(3) : ", math.cos(3))
3. cos(3) : -0.9899924966004454
```

Number hypot() method

The **hypot()** method provides the Euclidean norm, **sqrt(x*x + y*y)**, as output. This indicates the length of the vector from the origin to point (x,y).

Syntax

hypot(x, y)

Note: The hypot() method is only accessible using the static hypot module object.

Parameters

x – This indicates a value that must be numeric.

y – This indicates a value that must be numeric.

Example

```
1. >>> import math
2. >>> print ("hypot(3, 2) : ", math.hypot(3, 2))
3. hypot(3, 2) : 3.605551275463989
```

Number degrees() method

The **degrees()** method transforms angle x from radians to degrees.

Syntax

degrees(x)

Note: The degrees() method is only accessible using the degrees static module object.

Parameters:

x – This indicates a value that must be numeric.

Return value

This method provides an output in the degree value of an angle.

Example:

```
1. >>> import math
2. >>> print ("degrees(3) : ", math.degrees(3))
3. degrees(3) : 171.88733853924697
4. >>> print ("degrees(-3) : ", math.degrees(-3))
5. degrees(-3) : -171.88733853924697
```

Number radians() method

The **radians()** method transforms angle x from degrees to radians.

Syntax

`radians(x)`

Note: The `radians()` method is only accessible using the static `radians` module object.

Parameters

x - This indicates a value that must be numeric.

Return value

This method returns the radian value of an angle.

Example

```
1. >>> import math
2. >>> print ("radians(3) : ", math.radians(3))
3. radians(3) :  0.05235987755982989
4. >>> print ("radians(-3) : ", math.radians(-3))
5. radians(-3) :  -0.05235987755982989
```

Mathematical constants

Constants, which are constant numbers whose value is unambiguously defined, appear in many areas of mathematics, with constants like e and π appearing in such diverse contexts as geometry, number theory, and calculus.

The module also defines two mathematical constants, as follows:

Constants	Description
Pi	The mathematical constant pi
e	The mathematical constant e

Table 7.2: Mathematical constants

Conclusion

A number in Python refers to a numeric data type. Numeric types in Python are `int`, `float`, and `complex`. In this chapter, we learned how to use numbers and understood their significance in Python programming.

In the next chapter, we will discuss strings.

Key terms

- Floating points
- Complex numbers
- `Random()`, `hypot()`, `degrees()`, `radians()`
- `seed()`, `acos()`, `asin()`, `atan()`
- Constant

CHAPTER 8

Strings

Introduction

When characters like letters, digits, and symbols are arranged in a sequence, it is known as a string. The usual string object is an immutable (unchangeable) sequence of characters accessed by offset (position).

Strings are immutable, but you can update the value of a string by assigning a new string to the same string variable.

Structure

We will cover the following topics in this chapter:

- Creating a string
- Accessing values in string
- Changing or deleting a string
- String operations
- Build-in string methods
- Programs

Objectives

Strings can be worked on within Python using a variety of operators, functions, and methods. After going through the chapter, you will learn how to access and extract strings, and you will also be able to manipulate and modify string data using the methods available. Additionally, you will learn about the built-in string functions in Python.

Creating a string

We can create strings by enclosing characters within a single quote or double quotes. Python supports triple quotes, but these are generally used to represent multiline strings and docstrings.

Single quoted and double-quoted strings are identical.

In Python strings, single- and double-quote characters are interchangeable, i.e., string literals can be expressed in two single or two double quotes. Here, the two representations work in the same way and return the same type of object.

The reason for supporting the two representations is that it allows programmers to enclose a quote character of the other type within a string without a backslash. It also helps programmers to enclose a single-quote character in a string enclosed in double-quote characters, and vice-versa.

Example

```
1. >>> print("hello")
2. >>> print('hello')
3. >>> print("doesn't")
4. >>> print('He said "Hello"')
```

Output

```
hello
hello
doesn't
He said "Hello"
```

Accessing values in a string

Python does not allow a character type; these are considered as strings of length one, so they are also considered substrings.

We can access individual characters using indexing and with different characters using the process of slicing. The index always starts from zero, but an `IndexError` may arise while trying to access a character out of the index range. The index is always an integer; we cannot use float or other data types; this will lead to `TypeError`.

Python allows negative indexing for its sequences. For example, the index of `-1` indicates the last item, `-2` the second last item, and so on. Python also allows you to access a range of items in a string using the slicing operator (colon).

Slice expressions support the optional third index, which is used as a step (sometimes called stride). Steps are added to the index of each item extracted. Slices are now `X[I:J:K]`, which means that all items in `X` are removed from offsets `I` through `J*1`, by `K`. `K` is generally set to `+1`, which is why all items in a slice are generally extracted from left to right. However, the third limit can be used to skip items or reverse their order if you specify an explicit value. `X[1:10:2]` will fetch every other item in `X` from offsets `1–9`. It will collect items at offsets `1, 3, 5, 7, and 9`. Since the first and second limits are set to `0` and the length of the sequence, respectively, `X[:2]` receives all items from the beginning to the end of the sequence.

Example

```

1. >>> a="hello world"
2. >>>a[0]
3. >>>a[2: ]
4. >>>a[ : ]
5. >>>a[ :6]
6. >>>a[3:8]
7. >>>a[6:-3]
8. >>> a[::-1]                                # Reversing items
9. >>>a[0:10:2]                                # Skipping items
```

Output

```

'h'
'llo world'
'hello world'
'hello '
'lo wo'
'wo'
'olleh'
'dlrowolleh'
'hlowr'
```

Changing or deleting a string

Strings are immutable, which implies that elements of a string once assigned cannot be changed. Python allows you to set the same name to different strings.

```
1. >>> a="hello"
2. >>>a[4]='a'
3. Traceback (most recent call last):
4.   File "<pyshell#17>", line 1, in <module>
5.     a[4]='a'
6.   TypeError: 'str' object does not support item assignment
7. >>> a="world"
8. >>> a
9.     'world'
```

We cannot delete or remove characters from a string, but we can delete the string entirely using the **del** keyword.

```
1. >>> del a[3]
2. Traceback (most recent call last):
3.   File "<pyshell#20>", line 1, in <module>
4.     del a[3]
5.   TypeError: 'str' object doesn't support item deletion
6. >>> del a
7. >>> a
8. Traceback (most recent call last):
9.   File "<pyshell#22>", line 1, in <module>
10.    a
11.   NameError: name 'a' is not defined
```

Escape characters

Escape sequences or non-printable characters are characters that are not printed on the screen as is. They convey a different meaning, and backslash notation is used to represent them.

An escape character is interpreted with single quoted as well as double quoted strings.

Backslash notation	Description
a	Bell or alert
B	Backspace
\cx or \Cx	Control-x
\e	Escape
\f	Formfeed
\n	Newline
\nnn	Octal notation, where n is in range 0 to 7
\r	Represents a carriage return
\s	Represents a space
\t	Indicates a tab
\v	Indicates a vertical tab
\x	Represents the character x
\xnn	Represents a hexadecimal notation, where n is in the range 0 to 9, a to f, or A to F

Table 8.1: Backslash notation

String operations

Concatenation

You can concatenate strings using the **+** operator. Formally, adding two string objects with the **+** symbol creates a new string object with the contents of its operands joined.

1. `>>> a="hello"`
2. `>>> b="world"`

```
3. >>> c=a+b
4. >>> c
5. 'helloworld'
```

Repetition

You can repeat a string using the `*` operator. Repetition with `*` is related to the addition of a string to itself a number of times.

```
1. a="hello "
2. b=a*4
3. b
4. 'hello hellohellohello'
```

Membership

- **in** - Returns true if a character exists in the given string.
- **not in** - Returns true if a character does not exist in the given string.

```
1. a="hello world"
2. b="world"
3. c="word"
4. b in a
5. True
6. c in a
7. False
8. b not in a
9. False
```

Raw string

Raw strings consider escape characters to be part of the complete string. The syntax for raw strings and normal strings are the same, except that the letter "r" precedes the quotation marks in the raw string operator. The raw string operator is case insensitive, which means, "r" can either be lowercase (r) or uppercase (R) and should be included immediately preceding the first quote mark.

Example

```
1. >>> print("hello\nworld")
```

```

2. hello
3. world
4. >>>print("hello" r "\nworld")
5. hello\nworld

```

String formatting

Python also supports a leading method to blend various string processing tasks; string formatting permits you to do multiple type-specific substitutions on a string in one step. It's not strictly needed, but it can be convenient, especially when formatting text to be displayed to a program's users.

- The initial expression (all Python versions), coded with the `%` operator: `fmt % (values)`
- The latest method (3.0, 2.6, and later), coded with **call syntax**: `fmt.format(values)`

Example

```

1. >>> '%s, %s, %f' %(23,"hello",2.13)
2. '23, hello, 2.130000'
3. >>> '{0},{1},{2:f}'.format(23,"hello",2.13)
4. '23,hello,2.130000'

```

Triple quote code

Python also supports triple-quoted string literal format, also known as a block string. It is a syntactic accessibility for performing various multiline coding with different text data. This format begins with three quotes (of either the single or double variety), is followed by one or more lines of text, and ends with the same type of three quotes that opened it. It's not necessary to embed single and double quotes in the string's text, escaped—the Python does not end the string until it sees three unescaped quotes of the same type as the ones used to start it.

Example

```

1. >>>print("""hello
2. how're you!!!
3. I am good""")
4.
5. hello

```

6. how're you!!!
7. I am good

Unicode string

All text is unicode text, including the text encoded with one character per byte (8 bits) in the ASCII scheme. Python supports richer character sets and encoding schemes with unicode, that is, strings that may use multiple bytes to represent characters in memory, and which translate text to and from various encodings on files.

Built-in string methods

String capitalize() method

This method provides us a copy of the string with the first letter capitalized and the rest lowercase as the output.

Syntax

```
str.capitalize()
```

Parameters

NA

Return value

string

Example

1. `>>> a="hello world"`
2. `>>>a.capitalize()`
3. `'Hello world'`

String center() method

The **center()** method returns the string at the center. Padding is done using the specified fillchar, and the default filler is a space.

Syntax

str.center(width[, fillchar])

Parameters

- width - This returns the total width of the given string

- **fillchar** - This acts as the filler character to fill the gap

Example

```
1. >>> a="hello world"
2. >>>a.center(30,'*')
3. '*****hello world*****'
```

String count() method

The **count()** method gives the number of occurrences of the substring **sub** within the range [**start**, **end**] as the output. Arguments **start** and **end** are optional, and they are interpreted the same as in slice notation.

Syntax

```
str.count(sub, start= 0,end=len(string))
```

Parameters

- **sub** - This is the substring needed for searching
- **start** - Search always starts from this index, and the first character usually starts from the 0th index, so search starts from the 0th index by default.
- **end** - Search ends at this index; the first character always starts from 0th index, and search ends at the last index in the default case

Return value

Centered in a string with length width

Example

```
1. >>> a="hello world hello"
2. >>> sub='lo'
3. >>>a.count(sub,3)
4. 2
```

String decode() method

The **decode()** method is used to decode the string using the codec registered for encoding. It defaults to the default string encoding.

Syntax

```
str.decode(encoding='UTF-8',errors='strict')
```

Parameters

- **encoding** - This is used for encoding.
- **errors** – It provides different error handling schemes. 'Strict' is the default value for errors, which indicates a **UnicodeError**. Other available values for errors are **ignore**, **replace**, **xmlcharrefreplace**, **backslashreplace** and any other name registered via **codecs.register_error()**.

Return value

It returns a decoded string.

Example

1. String = 'Python'
2. Print('The string is:', String)
3. String_utf=String.encode()
4. Print('The encoded version is:',String_utf)
5. Print('The decode version is:',String_utf.decode())

Output

The String is: Python

The encoded version is b'pyth\xc3\xb6n'

The decoded version is Python

String encode() method

The purpose of the **encode()** method is to provide the encoded version of the string. The default encoding is the current default string encoding. The errors produced are given to set a different error handling scheme.

Syntax:

```
str.encode(encoding='UTF-8',errors='strict')
```

Parameters:

- **encoding** - This is used for encodings.
- **errors** - It provides different error handling scheme. **Strict** is the default value for errors, which indicates a **UnicodeError**, and other available values for errors are **ignore**, **replace**, **xmlcharrefreplace**, **backslashreplace** and any other name registered via **codecs.register_error()**.

Return value

It returns an encoded string

Example

1. `String = 'Python'`
2. `Print('The string is:', String)`
3. `String_utf=String.encode()`
4. `Print('The encoded version is:',String_utf)`

Output

The String is: Python

The encoded version is b'pyth\xc3\xb6n!

String endswith() method

The boolean value true is returned when the string ends with the given suffix; otherwise, it returns false. Also, it restricts the matching with the given indices start and end, which are optional.

Syntax

```
str.endswith(suffix[, start[, end]])
```

Parameters

- **suffix** – It can be a string or a tuple of suffixes to look for
- **start** – It represents the start of the slice
- **end** – It indicates the end of the slice

Example

1. `Str='this is a test of endswithfunction....!!'`
2. `suffix='!!'`
3. `print (Str.endswith(suffix))`
4. `print (Str.endswith(suffix,25))`
5. `suffix='exam'`
6. `print (Str.endswith(suffix))`
7. `print (Str.endswith(suffix, 0, 24))`

Output

True

True

False

False

String `expandtabs()` method

This method provides an exact copy of the string in which the tab character, that is, `'\t'`, uses spaces to expand, using the given `tabsize` (default `8`).

Syntax

```
str.expandtabs(tabsize=8)
```

Parameters

- **tabsize**- This indicates the number of characters needed to replace for the given tab character `\t`

Return value

An expanded string is returned in which tab character expansions have been performed using spaces, for example, `/t`

Example

```
1. str = "this is expandtabsfunction....!!"
2. print ("Original string: " + str)
3. print ("Defaultexpanded tab: " + str.expandtabs())
4. print ("Double expanded tab: " + str.expandtabs(16))
```

Output

```
Original String: this is expandtabsfunction....!!
Defaultexpanded tab: this is expandtabsfunction....!!
Double expanded tab: this is expandtabsfunction....!!
```

String `find()` method

The `find()` method determines whether the given string `str` occurs in the string or in a substring of the string if the starting index `beg` and ending index `end` are given.

Syntax

```
str.find(str, beg=0 end=len(string))
```

Parameters

- **str** - It is the string needed to be found
- **beg** - It is the starting index of the string; by default, it is zero
- **end** - It indicates the ending index; by default, it is equal to the length of the string.

Return type

Index if found and -1 otherwise

Example

```
1. str1 = "this is string find function...!!"
2. str2 = "find";
3. print (str1.find(str2))
4. print (str1.find(str2, 10))
5. print (str1.find(str2, 40))
```

Output

```
15
15
-1
```

String index() method

The **index()** method determines whether the string `str` occurs in string or in a substring of the string, given the starting index `beg` and ending index `end`. This method is similar to **find()**, but it raises an exception if `sub` cannot be found.

Syntax

```
str.index(str, beg=0 end=len(string))
```

Parameters

- **Str**: It specifies the string to be searched
- **beg**: It is the starting index; by default, it is 0
- **end**: The ending index, by default, is equal to the string length

Return value

Index if found, and raises an exception if `str` is not found

Example

```
1. str1 = "this is string index function...!!"
2. str2 = "index";
3. print (str1.index(str2))
4. print (str1.index(str2, 10))
```

Output

15

15

String `isalnum()` method

The `isalnum()` method checks whether the string contains alphanumeric characters.

Syntax

```
str.isalnum()
```

Parameters

NA

Return type

This method returns true if all characters in the string are alphanumeric and there is at least one character; otherwise, it returns false.

Example

```
1. str = "thisisnot1234"
2. print (str.isalnum())
3. str = "this is string alphanumfunction..!!"
4. print (str.isalnum())
```

Output

True

False

String `isalpha()` method

The `isalpha()` method checks whether the string consists of alphabetic characters only.

Syntax

```
str.isalpha()
```

Parameters

NA

Return type

The method returns true if there is at least one character in the string and all characters are alphabetic; otherwise, it returns false.

Example

```
1. str = "thisisastring";
2. print (str.isalpha())
3. str = "this is string isalphafunction..!!"
4. print (str.isalpha())
```

Output

True
False

String islower() method

You can check whether all case-based characters (letters) in a string are lowercase by calling the **islower()** method.

Syntax

```
str.islower()
```

Parameters

NA

Return type

The method returns true if all cased characters in the string are lowercase

Example

```
1. str = "THIS is string islowerfunction..!!"
2. print (str.islower())
3. str = "this is string islowerfunction..!!"
4. print (str.islower())
```

Output

False

String isnumeric() method

The **isnumeric()** method checks whether a string contains only numeric characters. It is only available on unicode objects.

Syntax

```
str.isnumeric()
```

Parameters

NA

Return type

This method returns true if all characters are numeric; otherwise, it returns false.

Example

```
1. str = "this2016"
2. print (str.isnumeric())
3. str = "23443434"
4. print (str.isnumeric())
```

Output

False

True

String isspace() method

The **isspace()** method checks whether the string consists of whitespace.

Syntax

```
str.str.isspace()
```

Parameters

NA

Return type

This method returns true if there is at least one character in the string and there are only whitespace characters; otherwise, it returns false.

Example

```
1. str = "      "
2. print (str.isspace())
3. str = "This is string isspace function!!"
4. print (str.isspace())
```

Output

True

String istitle() method

The **istitle()** method checks whether all case-based characters in a string following non-case-based letters are uppercase and whether all other case-based characters are lowercase.

Syntax

```
str.istitle()
```

Parameters

NA

Return type

As an example, uppercase characters can only follow uncased characters, and lowercase characters can only follow cased characters. So, this method returns true if the string is a title cased string and there is at least one character; otherwise, it returns false.

Example

```
str = "This Is String Function..!!"
print (str.istitle())
str = "This is string function..!!"
print (str.istitle())
```

Output

True

String isupper() method

The **isupper()** method determines whether all the case-based characters (letters) in the string are uppercase.

Syntax

```
str.isupper()
```

Parameters

NA

Return type

The method returns true only if all characters in the string are uppercase

Example

1. `str = "THIS IS STRING ISUPPER FUNCTION..!!"`
2. `print (str.isupper())`

```
3. str = "THIS is string upper function..!!"  
4. print (str.isupper())
```

Output

True

String join() method

The **join()** method returns a string containing all the string elements of a sequence separated by an str separator.

Syntax

```
str.join(sequence)
```

Parameters

- **sequence** - In this case, the elements will be joined in a sequence

Return type

The method returns a string that is the concatenation of the strings in the sequence seq. The string providing this method acts as a separator between elements.

Example

```
1. s = ">"  
2. seq = ("3", "2", "1") # This is sequence of strings.  
3. print (s.join( seq ))
```

Output

3>2>1

String len() method

This **len()** method returns the length of the string.

Syntax

```
len( str )
```

Parameters

NA

Example

```
1. str = "This Is String lenFunction..!!"  
2. print ("Length of the string: ", len(str))
```

Output

Length of the String: 30

String ljust() method

The **ljust()** method returns the left-justified string in a string of length width. The padding is calculated using the specified fillchar (by default, a space is used). The original string is returned if width is smaller than len (s).

Syntax

```
str.ljust(width[, fillchar])
```

Parameters

- **width** - After padding, this is the total length of the string
- **fillchar** - This is a filler character; the default is a space

Example

1. `str = "This Is String ljustFunction..!!"`
2. `print (str.ljust(50, '*'))`

Output

```
This Is String ljustFunction..!!*****
```

String lower() method

The **lower()** method returns a copy of the string if all case-based characters have been lowercased.

Syntax

```
str.lower()
```

Parameters

NA

Example

1. `str = "This Is String lower Function..!!"`
2. `print (str.lower())`

Output

```
this is string lowerfunction..!!
```

String lstrip() method

The **lstrip()** method returns a copy of the string that has been stripped of all characters at the beginning (default whitespace characters).

Syntax

```
str.lstrip([chars])
```

Parameters

- **chars** - You can specify which characters need to be trimmed.s

Example

```
1. str = "This Is String lstripFunction..!!"
2. print (str.lstrip())
3. str = "*****this is string lstripfunction..!!*****"
4. print (str.lstrip('*'))
```

Output

```
This Is String lstripFunction..!!
this is string lstripfunction..!!*****
```

String maketrans() method

Using **maketrans()**, each character in the intabstring is mapped into the character in the outtabstring at the same position. **Translate()** then takes this table as input.

Syntax

```
str.maketrans(intab, outtab]);
```

Parameters

- **intab** - The actual characters are in this string
- **outtab** - There are corresponding mapping characters in this string

Return type

The method returns a translate table to be used with the **translate()** function

Example

```
1. Intab="aeiou"
2. outtab = "12345"
3. trantab = str.maketrans(intab, outtab)
4. str = "this is string translate function...!!"
```

```
5. print (str.translate(trantab))
```

Output

```
Th3s 3s str3ng tr1nsl2t3 f5ct34n...!!
```

String max() method

The **max()** method returns the maximized alphabetical character from the string **str**.

Syntax

```
max(str)
```

Parameters

- **str** - From this string, the maximum alphabetical character should be returned

Example

```
1. str = "this is a string example..!!"
2. print ("Max character: " + max(str))
3. str = "this is a string max function..!!"
4. print ("Max character: " + max(str))
```

Output

```
Max character: x
```

String min() method

The **min()** method returns the minimum alphabetical character in the string **str**.

Syntax

```
min(str)
```

Parameters

str - The string from which the minimum alphabetical character needs to be returned.

Return type

This method returns the minimum alphabetical character from the **str** string.

Example

```
1. str = "thisisastringexample..!!"
2. print ("Min character: " + min(str))
3. str = "thisisastringmaxfunction..!!"
4. print ("Min character: " + min(str))
```

Output

Min character: !

String replace() method

A string returned by **replace()** has all occurrences of the old replaced with the new, optionally, with a maximum number of replacements.

Syntax

```
str.replace(old, new[, max])
```

Parameters

- **old** – This is the old substring to be replaced
- **new** - This is a new substring that replaces the old substrings
- **max** - When the optional argument **max** is given, only the first count occurrences are replaced

Return type

This method returns a copy of the string with all instances of substring **old** replaced by the new one. By specifying the optional argument **max**, only the first count occurrences are replaced.

Example

```
1. str = "this is a replace function"
2. print (str.replace("is", "was"))
```

Output

this was a replace function

String rfind() method

The **rfind()** method returns the last index in which the **str** substring is found or **-1** if no such index exists; optionally, the search is restricted to **String [beg:end]**.

Syntax

```
str.rfind(str, beg=0 end=len(string))
```

Parameters

- **str** – It specifies the string to be searched
- **beg** - It is the starting index; by default, it is 0
- **end** - By default, the ending index is equal to the length of the string

Example

```

1. str1 = "this is a string rfindfunction...!!"
2. str2 = "is"
3. print (str1.rfind(str2))
4. print (str1.rfind(str2, 0, 10))
5. print (str1.rfind(str2, 10, 0))
6. print (str1.find(str2))
7. print (str1.find(str2, 0, 10))
8. print (str1.find(str2, 10, 0))

```

Output

```

5
5
-1
2
2
-1

```

String rindex() method

When **rindex()** is called, it returns the last index where the str substring has been found or raises an exception if no index exists; optionally the search is restricted to **string[beg:end]**.

Syntax

```
str.rindex(str, beg=0 end=len(string))
```

Parameters

- **str** - It specifies the string to be searched
- **beg** – The starting index; by default, it is 0
- **len** – The ending index; by default, it is equal to the length of the string

Example

```

1. str1 = "this is a string rindex function...!!"
2. str2 = "is"
3. print (str1.rindex(str2))
4. print (str1.rindex(str2,10))

```

Output

5

Traceback (most recent call last):

File "C:/Users/Admin/Desktop/python files/srindex.py", line 4, in
<module>

```
print (str1.rindex(str2,10))
```

ValueError: substring not found

String rjust() method

A string of length width is returned by the **rjust()** method with the string right justified. The padding is calculated using the specified fillchar (by default, a space is used). The actual string is returned if the width is less than len (s).

Syntax

```
str.rjust(width[, fillchar])
```

Parameters

- **width** - After padding, this is the length of the string in total
- **fillchar** - It is the filler character; by default, it is a space

Example

1. `str = "this is string rjustfunction..!!"`
2. `print (str.rjust(50, '*'))`

Output

```
*****this is string rjustfunction..!!
```

String.rstrip() method

Whitespace characters are stripped from the string using **rstrip()** (default).

Syntax

```
str.rstrip([chars])
```

Parameters

chars - You can give what chars have to be trimmed

Example

1. `str = " this is a string rstripfunction....!! "`
2. `print (str.rstrip())`

Output

this is a string rstripfunction....!!

String split() method

The **split()** function splits a string into words using str as a separator (breaks on whitespace if not specified), there is an option limiting the number of splits to num.

Syntax

```
str.split(str="", num=string.count(str))
```

Parameters

- **str** - It is any delimiter; by default, this is space
- **num** - This is the number of lines to be made

Return type

This method returns a list of lines

Example

```
1. str = "this is a string split function..!!"
2. print (str.split( ))
3. print (str.split('i',1))
4. print (str.split('l'))
```

Output

```
['this', 'is', 'a', 'string', 'split', 'function..!!']
['th', 's is a string split function..!!']
['this is a string sp', 'it function..!!']
```

String splitlines() method

The **splitlines()** method returns the lines of string, optionally including the line breaks (if the num parameter is true).

Syntax

```
str.splitlines( num=string.count('\n'))
```

Parameters

- **num** - If present, it would be assumed that the line breaks need to be included in the text

Return type

This method returns true if found matching with the string; otherwise, it returns false

Example

```
1. str = "this is \nstringsplitlines \nfunction..!!"
```

Output

```
['this is ', 'string splitlines ', 'function..!!']
```

String startswith() method

The **startswith()** method checks whether the string starts with str, optionally restricting the matching with the given indices start and end.

Syntax

```
str.startswith(str, beg=0,end=len(string))
```

Parameters

- str - String to be checked
- beg - This is the optional parameter to set start index to matching boundaries
- end - This is the optional parameter to set end index of the matching boundary

Return type

This method returns true if found matching with the string; otherwise, it returns false.

Example

```
1. str = "this is string startswithfunction....!!"
2. print (str.startswith( 'this' ))
3. print (str.startswith( 'string', 8 ))
4. print (str.startswith( 'this', 2, 4 ))
```

Output

```
True
```

```
True
```

```
False
```

String strip() method

The **Strip()** method returns a copy of a string in which all whitespace characters have been stripped (default).

Syntax

```
str.strip([chars])
```

Parameters

- **chars** - Characters will be trimmed at the start or the end with the selected characters.

Example

```
1. str = "*****this is a string strip function....!!*****"
2. print (str.strip( '*' ))
```

Output

```
this is a string strip function....!!
```

String swapcase() method

Swapping the case of the characters in a string is accomplished by the **swapcase()** method.

Syntax

```
str.swapcase()
```

Parameters

NA

Return type

If all the case-based characters are swapped, then this method returns a copy of the string

Example

```
1. str = "this is string swapcasefunction....!!"
2. print (str.swapcase())
3. str = "This Is String Swapcase Function....!!"
4. print (str.swapcase())
```

Output

```
THIS IS STRING SWAPCASE FUNCTION....!!
```

String title() method

The **title()** method returns a copy of the string in which the starting characters of all the words are capitalized.

Syntax

```
str.title()
```

Parameters

NA

Example

```
1. str = "this is string title function....!!"
2. print (str.title())
```

Output

This Is String Title Function....!!

String translate() method

A copy of the string is returned by the **translate()** method, in which the characters have been translated using table (created by **maketrans()** in the string module), optionally removing any characters found in the string **deletechars**.

Syntax

```
str.translate(table[, deletechars]);
```

Parameters

- **table** –You can use the **maketrans()** function to create translate table
- **deletechars** - Several characters in the source string need to be removed

Return type

This method returns a copy of the translated string

Example

```
1. from string import maketrans    #
   Required to call maketrans function.
2. input = "aeiou"
3. output = "12345"
4. traslt = maketrans(input, output)
5. str = "this is string example....wow!!!";
6. print (str.translate(traslt))
```

Output

th3s 3s str3ng 2x1mp12....w4w!!!

String upper() method

The **upper()** method returns a copy of the string in which all case-based characters have been uppercased.

Syntax

```
str.upper()
```

Parameters

NA

Return type

Returns the string with all characters converted to uppercase

Example

```
1. str = "this is string upper function....!!"
2. print ("str.upper : ",str.upper())
```

Output

```
str.upper :  THIS IS STRING UPPER FUNCTION....!!
```

String zfill() method

The **zfill()** method pads string on the left, with zeros to fill the width.

Syntax

```
str.zfill(width)
```

Parameters

width - This is the final width of the string; it is the width we would get after filling in zeros.

Return type

This method returns padded string

Example

```
1. str = "this is string zfill function...!!"
2. print ("str.zfill : ",str.zfill(40))
```

Output

```
str.zfill :  000000this is string zfill function...!!
```

String `isdecimal()` method

You can use the `isdecimal()` method to determine whether a string contains only decimal characters; only unicode objects have this method.

Note: Unlike in Python 2, all strings are represented as Unicode in Python 3. The following example illustrates it.

Syntax

```
str.isdecimal()
```

Parameters

NA

Return type

This method returns true if all the characters in the strings are decimal; otherwise, it returns false.

Example

```
1. str = "thisisnot01234"
2. print (str.isdecimal())
3. str = "2334567434"
4. print (str.isdecimal())
```

Output

False

True

Programs

1. Program to find the index of a character in a string:

```
1. a=input("Enter a string")
2. b=input("Enter a character")
3. pos=-1
4. for i in range(len(a)):
5.     if(a[i]==b):
6.         pos=i
7. if(pos!=-1):
8.     print("character found at index",pos)
```

```
9.  else:
10. print("character not found")
```

Output

```
Enter a string -hello
Enter a character- e
character found at index 1
```

2. Program that counts the number of characters in a string:

```
1. str=input('Enter a string: ')s
2. count=0
3. for i in str:
4.     for j in str:
5.         if j is i:
6.             count=count+1
7.     print(i,count)
8.     count=0
```

Output

```
Enter a string: python
p 1
y 1
t 1
h 1
o 1
n 1
```

3. Program to reverse a string:

```
1. a=input("Enter a string: ")
2. l=len(a)
3. rev=""
4. for i in a:
5.     rev=rev+a[l-1]
6.     l=l-1
7. print(rev)
```

Output

```
Enter a string: hello world
dlrowolleh
```

4. Program to find a substring in the string:

```
1. a=input("enter a string: ")
2. s=input("enter substring: ")
3. ans=a.find(s)
4. p=ans+len(s)
5. if(ans!=-1) and a[ans-1]==" " and a[p]==" ":
6.     print("found")
7. else:
8.     print("not found")
```

Output

```
enter a string: hello how are you
enter substring: are
found
```

5. Program to count vowels in a string:

```
1. str=input('Enter a string: ')
2. count=0
3. for i in str:
4.     if( (i is 'a') or (i is 'e') or
        (i is 'i') or (i is 'o') or (i is 'u')):
5.         count=count+1
6.     print(count)
```

Output

```
Enter a string: hello world
3
```

6. Program to check whether a string is palindrome:

```
1. a=input("enter a string: ")
2. a=str.upper(a)
```

```
3. b=""
4. for i in a:
5.     b=i+b
6. if(b==a):
7.     print("palindrome")
8. else:
9.     print("not palindrome")
```

Output

enter a string: Malayalam

palindrome

7. Program to find the occurrence of a word in a string:

```
1. s1=input("enter a string: ")
2. word=input("enter word")
3. s2=[]
4. count=0
5. s2=s1.split(" ")
6. for i in range(0,len(s2)):
7.     if(word==s2[i]):
8.         count+=1
9. print("count of word is ",count)
```

Output

enter a string: abcfghhjabcabcfwabcrabchhabc

enter word: abc

count of word is 3

8. Program to sort a string:

```
1. a=input("Enter a string")
2. b=a.split( )
3. b.sort()
4. print(b)
```

Output

```
Enter a string- hello how are you
['are', 'hello', 'how', 'you']
```

9. Program to check whether the two strings are anagram:

```
1.  # check whether strings are anagram or not
2.  s1=input("enter first string: ")
3.  s2=input("enter second string: ")
4.  if(sorted(s1)==sorted(s2)):
5.      print("both strings are anagram")
6.  else:
7.      print("strings are not anagram")
```

Output

```
enter first String: hello
enter second String: eholl
both strings are anagram
```

10. Program to calculate the number of digits and characters in a string:

```
1.  s1=input("enter a string: ")
2.  countN=0
3.  countC=0
4.  for i in s1:
5.      if(i.isdigit()):
6.          countN+=1
7.          countC+=1
8.  print("digits = ",countN)
9.  print("characters = ",countC)
```

Output

```
enter a string: hello 123 hows you 678
digits = 6
characters = 22
```

Conclusion

String manipulation is easy with Python's built-in functions. Strings in Python are immutable, so these functions return a new string and leave the original string unchanged. You can do a lot with strings in Python, but it's not possible to remember them all. In this chapter, we discussed the different operations of string and the built-in functions of string.

In the next chapter, we will take a look at Python **List**.

Questions

1. If a string contains words, how can you check if they begin with a capital letter?
2. Which method can be used to verify whether two strings are the same?
3. Check whether a string contains any substring.
4. Count the characters in a string.
5. Find a string that contains numbers and any uppercases.

CHAPTER 9

Lists

Introduction

The primary data structure of Python is the sequence. Every element of a sequence is assigned a number—its position or index. The first index will be zero, the second index will be one, and so on.

There are six types of sequences in Python, but lists and tuples are the most common among these.

Structure

We will cover the following topics in this chapter:

- Lists in Python
- Accessing values in the list
- Updating list
- Deleting list elements
- Built-in list functions and methods

Objectives

Python lists have a bunch of methods to perform different tasks. For example, we can sort records with *sort*, eliminate the last element from a list with *pop*, count the quantity of things we're searching for with *count*, and remove elements with *clear*. After going through the chapter, readers will learn about the key operations on a Python list.

Lists in Python

Lists are heterogenous data structures created by elements separated with commas and enclosed within square brackets.

Example

```
1. List1 = ['physics', 'biology', 1994, 1997];
2. List2 = [1, 3, 6, 4, 5];
3. List3 = ["a", "b", "d", "c"];
```

Accessing values in a list

The `[]` index operator can be used on a list to access elements by the index.

Example

```
1. list1 = ['physics', 'chemistry', 1994, 1997]
2. list2 = [1, 2, 3, 4, 5, 6, 7]
3. print ("list1[0]:", list1[0])
4. print ("list2[1:5]:", list2[1:5])
```

Output

```
list1[0]: physics
list2[1:5]: [2, 3, 4, 5]
```

Updating lists

The variable on the left-hand side of the assignment operator can be used to update or add to elements in lists. The **`append()`** method can be used to add to the elements in a list.

Example

```
1. list1 = ['physics', 'maths', 1994, 1997]
2. print ("Value available at index 2: ", list1[2])
```

```
3. list1[2] = 2000
4. print ("New value available at index 2: ", list2[0])
```

Output

Value available at index 2: 1994

New value available at index 2: 2000

Deleting list elements

When deleting a list element, you can use either of the `del` statements to know exactly which element(s) you are deleting. You can use the `remove()` method if you don't know exactly which items to delete. `Rundown` can be replaced with the collection as the development community is familiar with this term rather than the name `rundown`.

Example

```
1. list = ['maths', 'chemistry', 1994, 1997]
2. print (list)
3. del list [2]
4. print ("After deleting value at index 2: ", list)
5.
```

Output

['maths', 'chemistry', 1994, 1997]

After deleting value at index 2: ['maths', 'chemistry', 2000]

Built-in list functions and methods

List `len()` method

The `len()` method returns the total number of components in the `rundown`.

Syntax

```
len(list)
```

Parameters

`list` - This is the list in which the elements are to be counted.

Return value

This returns the number of elements in the list.

Example

```
1. list1 = ['physics', 'chemistry', 'maths']
2. print (len(list1))
3. list2=list(range(5)) #creates list of
   numbers between 0-4
4. print (len(list2))
```

Output

```
3
5
```

List max() method

This **max()** method returns the components from the rundown with the most noteworthy worth.

Syntax

```
max(list)
```

Parameters

list - The number of elements in this list is to be counted.

Return value

The **max()** method returns the components in the rundown with the most elevated worth.

Example

```
1. list1, list2 = ['C', 'Java', 'Python'], [654, 600, 200]
2. print ("Maximum value element: ", max(list1))
3. print ("Maximum value element: ", max(list2))
```

Output

```
Maximum value element : Python
```

List min() method

The **min()** method returns the components from the rundown with the least worth.

Syntax

```
min(list)
```

Parameters

list - This is the list in which the number of elements is to be counted.

Return value

This **min()** method returns the elements from the list with minimum value.

Example

```
1. list1, list2 = ['C++', 'Java', 'Python'],  
   [654, 700, 100]  
  
2. print ("min value element : ", min(list1))  
  
3. print ("min value element : ", min(list2))
```

Output

```
min value element : C++  
min value element : 100
```

List list() method

The **list()** method converts sequence types into lists. For example, it is used to convert a tuple into a list.

Note: Tuples are similar to records, the difference being that component upsides of a tuple can't be changed, and tuple components are put between enclosures rather than square sections. Similarly, this method changes characters in a string into a rundown.

Syntax

```
list( seq )
```

Parameters

seq- This is a tuple or string to be converted into a List.

Return value

This method returns the list.

Example

```
1. Tuple1 = (987, 'C', 'Java', 'Python')
2. list1 = list(Tuple1)
3. print ("List elements : ", list1)
4. str="Welcome"
5. list2=list(str)
6. print ("List elements : ", list2)
```

Output

```
List elements : [897, 'C', 'Java', 'Python']
List elements : ['W', 'e', 'l', 'c', 'o', 'm', 'e']
```

Python includes the list methods discussed here.

List append() method

The append() method appends a passed obj to the existing list.

Syntax

```
list.append(obj)
```

Parameters

obj- The object to be appended to the list.

Return value

There is no return value, but the updated list is printed.

Example

```
1. list1 = ['C++', 'Java', 'Python']
2. list1.append('C#')
3. print ("updated list : ", list1)
```

Output

```
updated list : ['C++', 'Java', 'Python', 'C#']
```

List count() method

The count() method returns the count of how frequently **obj** occurs in the list.

Syntax

```
list.count(obj)
```

Parameters

obj - Object to be counted in the list.

Return value

The **count()** method returns the count of how many times **obj** occurs in the list.

Example

```
1. alist = [123, 'xyz', 'zara', 'abc', 123];
2. print ("Count for 123 : ", alist.count(123))
3. print ("Count for zara : ", alist.count('zara'))
```

Output

```
Count for 123 :  2
```

```
Count for zara :  1
```

List extend() method

The **extend()** method appends the contents of seq to list:

Syntax

```
list.extend(sequence)
```

sequence is the list's element.

Return value

The **extend()** method does not return any value but adds the content to an existing list.

Example

```
1. list1 = ['physics', 'chemistry', 'maths']
2. list2=list(range(5)) #creates List of numbers between 0-4
3. list1.extend('Extended List :', list2)
4. print (list1)
```

Output

```
Extended List :  ['physics', 'chemistry', 'maths', 0, 1, 2, 3, 4]
```

List index() method

The `index()` method returns the lowest index in list that `obj` appears.

Syntax

```
list.index(obj)
```

Parameters

`obj` – `obj` is the object to be determined.

Return value

This method returns the index of the found object or raises an exception indicating that the value was not found.

Example

```
1. list1 = ['physics', 'chemistry', 'maths']
2. print ('Index of chemistry', list1.index('chemistry'))
3. print ('Index of C#', list1.index('C#'))
```

Output

```
Index of chemistry 1
```

```
Traceback (most recent call last):
```

```
File "test.py", line 3, in
```

```
print ('Index of C#', list1.index('C#')) ValueError: 'C#' is not in list
```

List insert() method

The `insert()` method inserts object into list at offset index.

Syntax

```
list.insert(index, obj)
```

Parameters

`index` - This is the index; the object to be inserted

`obj` - This is the object to be inserted into the list

Return value

It does not return a value; rather, it inserts the given element at the given index.

Example

```
1. list1 = ['physics', 'chemistry', 'maths']
```

```
2. list1.insert(1, 'Biology')
3. print ('Final list : ', list1)
```

Output

```
Final list :  ['physics', 'Biology', 'chemistry', 'maths']
```

List pop() method

The **pop()** method eliminates the last item and returns the last item from the list.

Syntax

```
list.pop(obj=list[-1])
```

Parameters

obj – This is the index of the object to be removed from the list, and it is an optional parameter.

Return value

This method returns the removed object from the list.

Example

```
1. list1 = ['physics', 'Biology', 'chemistry', 'maths']
2. list1.pop()
3. print ("list now : ", list1)
4. list1.pop(1)
5. print ("list now : ", list1)
```

Output

```
list now :  ['physics', 'Biology', 'chemistry']
list now :  ['physics', 'chemistry']
```

List remove() method

The **remove()** function is an in-built function in the list collection that removes a given object from the list.

Parameters

obj - This is the object to be removed from the list.

Return value

The **remove()** method does not return any value but removes the given object from the list.

Example

```
1. list1 = ['physics', 'Biology', 'chemistry', 'maths']
2. list1.remove('Biology')
3. print ("list now : ", list1)
4. list1.remove('maths')
5. print ("list now : ", list1)
```

Output

```
list now : ['physics', 'chemistry', 'maths']
list now : ['physics', 'chemistry']
```

List reverse() method

The **reverse()** method reverses the objects of the list in place.

Syntax

```
list.reverse()
```

Parameters

NA

Return value

The **reverse()** method does not return any value; rather, it reverses the given object from the list.

Example

```
1. list1 = ['physics', 'Biology', 'chemistry', 'maths']
2. list1.reverse()
3. print ("list now : ", list1)
```

Output

```
list now : ['maths', 'chemistry', 'Biology', 'physics']
```

List sort() method

The Python list **sort()** capacity can be utilized to sort a list in climbing, plummeting, or client characterized request.

Syntax

```
list.sort([func])
```

Parameters

NA

Example

```
1. list1 = ['physics', 'Biology', 'chemistry', 'maths']
2. list1.sort()
3. print ("list now : ", list1)
```

Output

```
list now : ['Biology', 'chemistry', 'maths', 'physics']
```

Programs

1. Program to find the largest number in a list:

```
1. a=[]
2. n=int(input("Enter number of elements: "))
3. for i in range(1,n+1):
4.     b=int(input("Enter element: "))
5.     a.append(b)
6.     a.sort()
7.     print("Largest element is: " ,a[n-1])
```

This is the output:

```
Enter number of elements: 4
Enter element: 12
Enter element: 78
Enter element: 23
Enter element: 45
Largest element is: 78
```

2. Program to categorize odd and even elements in a list into two different lists:

```
1. a=[]
2. n=int(input("Enter number of elements:"))
3. for i in range(1,n+1):
```

```
4.         b=int(input("Enter elements: "))
5.     a.append(b)
6.     even=[]
7.     odd=[]
8.     for j in a:
9.         if(j%2==0):
10.            even.append(j)
11.        else:
12.            odd.append(j)
13.    print("The even list",even)
14.    print("The odd list",odd)
```

The output is as follows:

Enter number of elements:5

Enter elements: 34

Enter elements: 67

Enter elements: 12

Enter elements: 98

Enter elements: 55

The even list [34, 12, 98]

The odd list [67, 55]

3. Program to merge two lists and sort it:

```
1.  a=[]
2.  c=[]
3.  n1=int(input("Enter number of elements:"))
4.  for i in range(1,n1+1):
5.      b=int(input("Enter element"))
6.      a.append(b)
7.  n2=int(input("Enter number of elements: "))
8.  for i in range(1,n2+1):
9.      d=int(input("Enter element: "))
10.  c.append(d)
11.  new=a+c
```

```
12. new.sort()
13. print("Sorted list is: ",new)
```

Here's the output:

```
Enter number of elements:5
Enter element34
Enter element22
Enter element67
Enter element45
Enter element11
Enter number of elements: 4
Enter element: 23
Enter element: 66
Enter element: 77
Enter element: 99
Sorted list is: [11, 22, 23, 34, 45, 66, 67, 77, 99]
```

4. Program to sort a list according to the length of the elements:

```
1. a=[]
2. n=int(input("Enter number of elements: "))
3. for i in range(1,n+1):
4.     b=input("Enter element: ")
5.     a.append(b)
6.     a.sort(key=len)
7.     print(a)
```

The output is as follows:

```
Enter element: 1
Enter element: 234
Enter element: 65
Enter element: 8834
['1', '65', '234', '8834']
```

5. Program to swap the first and last values in a list:

```
1. a=[]
2. n=int(input("Enter the number of elements in list: "))
3. for x in range(0,n):
```

```
4.     element=int(input("Enter element"+str(x+1)+":"))
5.     a.append(element)
6.     temp=a[0]
7.     a[0]=a[n-1]
8.     a[n-1]=temp
9.     print("New list is: ")
10.    print(a)
```

Here's the output:

```
Enter the number of elements in list: 4
Enter element1:12
Enter element2:67
Enter element3:33
Enter element4:45
New list is:
[45, 67, 33, 12]
```

6. Program to remove duplicate items from a list:

```
1.  a=[]
2.  n=int(input("Enter the number of elements in list: "))
3.  for x in range(0,n):
4.      element=int(input("Enter element"+str(x+1)+":"))
5.      a.append(element)
6.      b=set()
7.      unique=[]
8.      for x in a:
9.          if x not in b :
10.             unique.append(x)
11.             b.add(x)
12.     print("Non-duplicate items: ")
13.     print(unique)
```

It gives the following output:

```
Enter the number of elements in list: 7
Enter element1:23
```

```
Enter element2:56
Enter element3:43
Enter element4:23
Enter element5:67
Enter element6:98
Enter element7:67
Non-duplicate items:
[23, 56, 43, 67, 98]
```

7. Program to remove the *i*th occurrence of the given word in a list .:

```
1. a=[]
2. n=int(input("Enter the number of elements in list: "))
3. for x in range(0,n):
4.     element=input("Enter the elements : ")
5.     a.append(element)
6.     print(a)
7. c=[]
8. count=0
9. b=input("Enter word to remove: ")
10. n=int(input("Enter the occurrence to remove: "))
11. for i in a:
12.     if(i==b):
13.         count=count+1
14.         if(count!=n):
15.             c.append(i)
16. if(count==0):
17.     print("Item not found")
18. else:
19.     new=[]
20.     for i in a:
21.         if(i!=b):
22.             new.append(i)
23.
```

```
24.         print("The number of repetitions is: ",count)
25.         print("The distinct elements are: ",new)
```

Here's the output:

Enter the number of elements in List: 5

Enter the elements : hello

Enter the elements : bye

Enter the elements : world

Enter the elements : bye

Enter the elements : Python

['hello', 'bye', 'world', 'bye', 'python']

Enter word to remove: bye

Enter the occurrence to remove: 2

The number of repetitions is:

The distinct elements are: ['hello', 'world', 'python']

8. Program to search the number of times a particular number occurs in a list:

```
1.  a=[]
2.  n=int(input("Enter number of elements: "))
3.  for i in range(1,n+1):
4.      b=int(input("Enter element: "))
5.      a.append(b)
6.      k=0
7.  num=int(input("Enter the number to be counted: "))
8.  for j in a:
9.      if(j==num):
10.         k=k+1
11. print("Number of times",num,"appears is",k)
```

This is the output:

Enter number of elements: 5

Enter element: 23

Enter element: 34

```

Enter element: 23
Enter element: 78
Enter element: 56
Enter the number to be counted: 23
Number of times 23 appears is 2

```

9. Program to generate random numbers from 1 to 20 and append them to the list:

```

1. import random
2. a=[]
3. n=int(input("Enter number of elements: "))
4. for j in range(n):
5.     a.append(random.randint(1,20))
6. print("Randomised list is: ",a)

```

The output is as follows:

```

Enter number of elements: 5
Randomized list is: [2, 15, 20, 1, 7]

```

Conclusion

This chapter covered the basic list operations with examples. The list is quite possibly the most widely used data type in Python. A Python List can be effectively recognized by square sections [] and stores objects of all types. The next chapter covers tuples in detail.

Questions

1. Write a Python function to interchange the first and last elements in a list.
Input: [12, 35, 9, 56, 24]
Output: [24, 35, 9, 56, 12]
2. What is the difference between append and extend?
3. What is the difference between *pop()* and *remove()*?
4. Write a program to concatenate two lists.
5. How can you remove duplicate elements in a list?

CHAPTER 10

Tuples

Introduction

A **tuple** is a sequence of immutable or unchangeable Python objects, like lists. The difference between tuples and lists is that tuples cannot be changed. Additionally, lists use square brackets, whereas tuples use parentheses.

Structure

We will cover the following topics in this chapter:

- Creating a tuple
- Accessing values in tuples
- Updating tuples
- Deleting tuple elements
- Basic tuple operations
- Indexing, slicing, and matrices
- Built-in tuple functions

Objective

This chapter aims to make the learner understand the concept of tuples and the various operations performed upon tuples. Several examples are also included in the chapter to help the learner understand the concept clearly and easily.

Creating a tuple

Creating a tuple is as simple as separating values using commas. You may also put these comma-separated values between parentheses.

Consider this example:

```
1. t1=(1,3,'hello',98.4,76)
2. t4='a','f',43
3. t=tuple('hello')
4. t
5. ('h', 'e', 'l', 'l', 'o')
6. t2=()                                #empty tuple
7. t3=tuple()                            #empty tuple
```

Comma is not mandated to create a single element tuple:

```
tup1 = (50,)
```

Accessing values in tuples

Use the square brackets to access values in a tuple and the index or indices to obtain a value of that index. Take a look at this example:

```
1. t=(23,'hello',56.8,90,'python')
2. print(t[2])
3. 56.8
4. print(t[1:3])
5. ('hello', 56.8)
```

Updating tuples

A tuple cannot be changed once it is created, so tuples are immutable.

However, there is a workaround. We can convert a tuple into a list, change it, and then convert it back into a tuple.

```

1. t1=(65,90,89.7,'abc',78)
2. t1[3]=88
3. Traceback (most recent call last):
4.   File "<pyshell#50>", line 1, in <module>
5.     t1[3]=88
6. TypeError: 'tuple' object does not support item assignment
7. t2=(78,98)
8. t1=t1+t2
9. print(t1)
10. (65, 90, 89.7, 'abc', 78, 78, 98)

```

Deleting tuple elements

Individual elements from a tuple cannot be removed, but there is nothing wrong with constructing another tuple without the unwanted elements.

Use the `del` statement to explicitly remove a tuple. Consider this example:

```

1. t=(23,'hello',56.8,90,'python')
2. del t[2]
3. Traceback (most recent call last):
4.   File "<pyshell#55>", line 1, in <module>
5.     del t[2]
6. TypeError: 'tuple' object doesn't support item deletion
7. del(t)
8. t
9. Traceback (most recent call last):
10.  File "<pyshell#57>", line 1, in <module>
11.    t
12. NameError: name 't' is not defined

```

Note: An exception occurs. Tuple does not exist anymore after `del tup`, so we cannot delete a particular element from it.

Basic tuple operations

Len

This function is used to determine the length of the tuple.

Syntax

```
len(tuple)
```

Example:

1. `t=(12,45,'abc',67.5,'def',5)`
2. `len(t)`
3. `6`

Concatenation

This function adds values of tuples on either side of the operator:

1. `a=(1,2,3)`
2. `b=('abc','def','xyz')`
3. `a+b`
4. `(1, 2, 3, 'abc', 'def', 'xyz')`

Repetition

This function concatenates copies of the tuple to create a joined tuple with the duplicate elements.

```
a=(12,'abc',56.6)
```

1. `print(a*3)`

Membership

in - returns true if a character exists in the given string.

String to tuple

not in - returns a true value if a character does not exist in the given string.

Example:

1. `t1=('h','e','l','l','o')`

```
2. t2=('w','o','r','l','d')
3. 'h' in t1
4. True
5. 'h' in t2
6. False
7. 'e' not in t1
8. False
9. 'e' not in t2
10. True
```

Iteration

It indicates the repetition of a statement or group of statements while a given condition is TRUE.

```
1. for x in (1,2,3) :
2.     print (x, end=' ')
3. 1 2 3
```

Indexing, slicing, and matrices

Since tuples are sequences, indexing and slicing work the same way for tuples as they do for strings, assuming the following input:

```
1. t=('abc',45,90,45.6,'xyz',12.2,'mno')
2. t[2]
3. 90
4. t[4]
5. 'xyz'
6. t[2:4]
7. (90, 45.6)
8. t[4:]
9. ('xyz', 12.2, 'mno')
10. [-3]
11. 'xyz'
```

```
12. t[-3:]  
13. ('xyz', 12.2, 'mno')
```

Built-in tuple functions

Python includes the tuple functions listed in this section.

Tuple len() method

This method returns the number of elements in the tuple similar to the list.

Syntax

```
len(tuple)
```

Parameters

tuple - This is a tuple of elements to count.

Return value

This method returns the number of elements in the tuple.

Example:

```
1. tuple1, tuple2 = (123, 'xyz', 'zara'), (456, 'abc')  
2. print ("First tuple length : ", len(tuple1))  
3. print ("Second tuple length : ", len(tuple2))
```

Output:

```
1. First tuple length : 3  
2. Second tuple length : 2
```

Tuple max() method

The **max()** method returns the elements from the tuple with maximum value.

Syntax

```
max(tuple)
```

Parameters

tuple - This is a tuple from which the max valued element is to be returned.

Return value

This method returns the elements from the tuple with maximum value.

Example:

```
1. tuple1, tuple2 = ('maths', 'che', 'phy', 'bio'), (456, 700, 200)
2. print ("Max value element : ", max(tuple1))
3. print ("Max value element : ", max(tuple2))
```

Output:

```
Max value element :  phy
Max value element :  700
```

Tuple min() method

The **min()** method returns the elements from the tuple with minimum value.

Syntax

```
min(tuple)
```

Parameters

tuple - This is a tuple from which min valued element is to be returned.

Return value

This method returns the elements from the tuple with minimum value.

Example:

```
1. tuple1, tuple2 = ('cpp', 'php', 'java', 'python'), (754, 600, 100)
2. print ("min value element : ", min(tuple1))
3. print ("min value element : ", min(tuple2))
```

Output:

```
min value element : Python
min value element :  100
```

Tuple tuple() method

The **tuple()** method converts a list of items into tuples.

Syntax

`tuple( seq )`

Parameters

`seq` - This is a tuple to be converted into tuple.

Return value

This method returns the tuple.

Example:

```
1. list1= ['maths', 'che', 'phy', 'bio']
2. tuple1=tuple(list1)
3. print ("tuple elements : ", tuple1)
```

Output:

tuple elements : ('maths', 'che', 'phy', 'bio')

Programs

1. This is the program to remove all tuples in a list of tuples with the USN outside the given range:

```
2. y=[('a', '12CS117'), ('b', '12CS149'), ('c', '12CS260'), ('d', '12CS295'), ('e', '12CS325')]
3. low=int(input("Enter lower range (starting with 12CS): "))
4. up=int(input("Enter upper range (starting with 12CS): "))
5. l='12CS'+str(low)
6. u='12CS'+str(up)
7. for i in y:
8.     if i[1]>=l and i[1]<=u:
9.         print(i)
```

Output:

```
1. Enter lower range (starting with 12CS): 123
2. Enter upper range (starting with 12CS): 300
3. ('b', '12CS149')
4. ('c', '12CS260')
5. ('d', '12CS295')
```

2. Here's the program to sort a list of tuples in increasing order by the last element in each tuple:

```
1. def last(n):
2.     return n[pos]
3. def sort(tuples):
4.     return sorted(tuples, key=last)
5. a=[(12,34),(22,45),(23,12),(20,10),(45,67)]
6. pos=1
7. print("Sorted: ")
8. print(sort(a))
```

Output

Sorted:

[(20, 10), (23, 12), (12, 34), (22, 45), (45, 67)]

3. This program creates a list of tuples with the first element as the number and the second as the square of the number:

```
1. l_r=int(input("Enter the lower range: "))
2. u_r=int(input("Enter the upper range: "))
3. tuple=[(x,x**2) for x in range(l_r,u_r+1)]
4. print(tuple)
```

Output:

Enter the lower range: 2

Enter the upper range: 8

[(2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49), (8, 64)]

Conclusion

This chapter taught us tuples and the various operations performed upon them, and each operation was explained with the help of an example. You understood the basic concepts related to tuples, from creating a tuple and accessing values in tuples to updating tuples, using built-in functions in tuples, and so on.

In the next chapter, you will understand the dictionaries used in Python.

Questions

1. Create a tuple with a single item 5.
2. What are tuples used for in Python?
3. Are tuples mutable or immutable?
4. What is the importance of the len function in Python?
5. Explain the min() method with an example.
6. Compare the tuple max() and tuple min() methods.
7. How can you access values in tuples?
8. Explain the concatenation function used in tuples.
9. Explain any three built-in functions used in tuples in Python.
10. How is indexing done in tuples?

CHAPTER 11

Dictionaries

Introduction

A **dictionary** is entirely different for Python as it is not a sequence but a mapping. A mapping is the same as an object collection, but it stores objects using keys instead of relative positions. Moreover, mappings don't maintain any reliable left-to-right order; they map keys to the associated values.

Structure

We will cover the following topics in this chapter:

- Creating a dictionary
- Accessing the elements in a dictionary
- Updating a dictionary
- Deleting dictionary elements
- Built-in dictionary functions

Objectives

The main objective of this chapter is to know more about dictionaries in Python. A dictionary is similar to a list in that it allows you to refer to a collection of data by a single variable name. However, it differs from a list in one fundamental way. Order is important in a list, and the elements in a list never change (unless you explicitly make changes to them). Since the order of elements in a list is important, you refer to each element using its index (its position within the list).

Note:

- You can use only immutable objects like strings, numbers, or tuples for the keys of a dictionary. A tuple can contain any mutable or changeable object but cannot be modified post creation.
- The key must be unique.

Syntax

```
{<key>:<value>, <key>:<value>, ..., <key>:<value>}
```

Creating a dictionary

Curly brackets represent dictionary entries. Key-value pairs are separated by a colon in the dictionary.

An empty dictionary without any items is written with just two curly braces, i.e., {}.

```
1. # empty dictionary
2. my_dict = {}
3. # dictionary with integer keys
4. my_dict = {1: 'apple', 2: 'ball'}
5. # dictionary with mixed keys
6. my_dict = {'name': 'divanshi', 1: [2, 4, 3]}
7. # using dict()
8. my_dict = dict({1:'apple', 2:'ball'})
9. # from sequence having each item as a pair
10. my_dict = dict([(1,'apple'), (2,'ball')])
```

Accessing the elements in a dictionary

You use the square brackets along with the key to access the elements of a dictionary.

Example:

```

1. dict={'name':'Girish','age':23}
2. print("name is ",dict[name])
3. print("name is ",dict['name'])
4. name is Girish
5. print("age is ",dict['age'])
6. age is 23
7. print("marks are ",dict["marks"])
8. Traceback (most recent call last):
9.   File "<pyshell#12>", line 1, in <module>
10.    print("marks are ",dict["marks"])
11. KeyError: 'marks'

```

Updating a dictionary

Dictionary entries can be changed. We can add new items or change the values of the existing ones using the assignment operator. The value is updated if the key already exists; otherwise, a new key-value pair is added to the dictionary.

```

1. dict={'name':'Girish','age':23}
2. dict['age']=24
3. dict['marks']=90
4. dict
5. {'name': 'Girish', 'age': 24, 'marks': 90}

```

Deleting dictionary elements

The **pop()** method is used to remove an item from a dictionary. It returns the value of an item removed with the provided key. You can use **popitem()** to remove and return an arbitrary item (key, value) from a dictionary. The **clear()** method is used to remove all items at once. The **del** keyword can also be used to remove individual items or the entire dictionary.

```

1. squares = {1:1, 2:4, 3:9, 4:16, 5:25} # create a dictionary
2. squares.pop(4) # remove a particular item 16
3. 16
4. squares.popitem() # remove an arbitrary item (1, 1)

```

```
5. (5, 25)
6. del squares[2] # delete a particular item
7. squares
8. {1: 1, 3: 9}
9. squares.clear() # remove all items
10. squares
11. {}
12. del squares # delete the dictionary itself
13. squares
14. Traceback (most recent call last):
15. File "<pyshell#28>", line 1, in <module>
16.     squares
17. NameError: name 'squares' is not defined
```

Properties of dictionary keys

There are no restrictions on dictionary values. A Python object can either be a standard object or a user-defined object, but this is not true for dictionary keys.

The two important properties of keys are as follows:

- There will be no more than one entry per key, so duplicate keys are not allowed. If duplicate keys are encountered during assignment, the last assignment **wins**.
- A key must be immutable type, such as a dictionary key. You can use strings, numbers, or tuples, but a list, such as ['key'], is an invalid key.

```
a. dict={"name":"Girish","age":27}
b. Traceback (most recent call last):
c. File "<pyshell#6>", line 1, in <module>
d. dict={"name":"Girish","age":27}
e. TypeError: unhashable type: 'list'
```

Built-in dictionary functions

Dictionary len() method

The **len()** method returns the total length of the dictionary. The number of items in the dictionary would be equal to this.

Syntax

Here's the syntax for the **len()** method:

```
len(dict)
```

Parameter

dict - Calculate the length of the dictionary

Return value

This returns the length of the dictionary.

Example

Take a look at the following example:

```
1. dict={"name":"Girish","age":27}
2. print("length is ",len(dict))
3. length is 2
```

Dictionary str() method

The **str()** method returns a printable string representation of a dictionary.

Syntax

```
str(dict)
```

Parameter

dict - Dictionary

Return value

This method returns the string representation.

Example

This example shows this method's usage:

```
1. dict={"name":"Girish","age":34}
2. str(dict)
3. '{"name': 'Girish', 'age': 34}'"
```

Dictionary type() method

The **type()** function returns the type of the passed variable. It will return a dictionary type if the passed variable is a dictionary.

Syntax`type(dict)`**Parameter**`dict` - This is a dictionary**Return value**

The method will return the passed variable type.

Example

1. Here's an example of the usage of the **type()** method:`lang = 'Python'`
2. `digits = [1,2,3,4]`
3. `digits_dict = {'one':1, 'two':2, 'three':3}`
- 4.
5. `print(type(digits))`
6. `print(type(lang))`
7. `print(type(digits_dict))`

Methods on dictionary

Methods that are available with a dictionary are explained as follows:

1. `d={'name':'Girish', 'age':34}`
2. `type(d)`
3. `<class 'dict'>`

Here, we have considered **d** to be a dictionary.

Dictionary clear() method

The **clear()** method removes all items from the dictionary.

Syntax`d.clear()`**Parameters**

NA

Return value

This method does not return any value.

Example

This example shows the usage of the **clear()** method:

```
1. d={'name':'Girish','age':34}
2. d.clear()
3. d
4. {}
```

Dictionary copy() method

The **copy()** method returns a copy of the dictionary.

Syntax

```
dict.copy()
```

Parameters

NA

Return value

It returns a shallow copy of the dictionary.

Example

This example shows the usage of the **copy()** method:

```
1. x>>> d={'name':'Girish','age':34}
2. d2=d.copy()
3. print(d2)
4. {'name': 'Girish', 'age': 34}
```

Dictionary fromkeys() method

The **fromkeys()** method creates a new dictionary with keys from seq and values set to value.

Syntax

```
dict.fromkeys(seq[, value])
```

Parameters

- **seq** – This parameter is used for preparing dictionary keys
- **value** - If provided, the value will be set to this value

Return value

It returns the list.

Example

This example shows the usage of the **fromkeys()** method.

Dictionary **get()** method

The **get()** method returns a value for the given key; it returns default value None if the key is not available.

Syntax

```
1. >>> d={'name':'Girish','age':34}
2. >>> dict=dict.fromkeys(d)
3. >>> dict
4. {'name': None, 'age': None}
5. >>> l=['name','marks','age']
6. >>> dict=dict.fromkeys(l)
7. >>> dict
8. {'name': None, 'marks': None, 'age': None}
9. >>> dict=dict.fromkeys(l,10)
10. >>> dict
11. {'name': 10, 'marks': 10, 'age': 10}
```

The syntax for the **get()** method is as follows:

```
dict.get(key, default=None)
```

Parameters

- key – The key to be searched in the dictionary
- default – The default value to be returned if the key does not exist

Return value

This method returns a value for the given key; it returns the default value None if the key is not available.

Example

The following example shows the usage of the **get()** method:

```
1. >>> d={'name':'Girish','age':34,'marks':90}
```

```
2. >>> print(d.get('age'))
3. 34
4. >>> print(d.get('gender','male'))
5. male
```

Dictionary items() method

The **items()** method returns a list of dictionary (key, value) tuple pairs.

Syntax

```
dict.items()
```

Parameters

NA

Return value

It returns a list of tuple pairs.

Example

This example shows the usage of the **items()** method:

```
1. >>> d={'name':'Girish','age':34,'marks':90}
2. >>> print(d.items())
```

Dictionary keys() method

The **keys()** method returns a list of all the keys in the dictionary.

Syntax

```
dict.keys()
```

Parameters

NA

Return value

This function will return a list of all the keys in the dictionary.

Example

Here's an example that shows the usage of the **keys()** method:

```
1. >>> d={'name':'Girish','age':34,'marks':90}
2. >>>d.keys()
3. dict_keys(['name', 'age', 'marks'])
```

Dictionary.setdefault() method

The **setdefault()** method is similar to **set()**, but it will set **dict[key]=default** if the key is not already in dict.

Syntax

```
dict.setdefault(key, default=None)
```

Parameters

- key – The key to search
- Default - The value to be returned if the key cannot be found

Return value

The method returns the value of the given key if it is present in the dictionary; it returns the default value if it isn't present.

Example

This example shows the usage of the **setdefault()** method:

```
1. >>> d={'name':'girish','age':34}
2. >>> print(d.setdefault('age',None))
3. 34
4. >>> print(d.setdefault('gender',None))
5. None
6. >>> d
7. {'name': 'girish', 'age': 34, 'gender': None}
```

Dictionary update() method

Dictionary dict2's key-value pairs are added to dict using the **update()** method; there is no return from this function.

Syntax

```
dict.update(dict2)
```

Parameters

dict2 - Dictionary is to be added into the dict.

Return value

It does not return any value.

Example

The following example shows the usage of the **update()** method:

```
1. >>> d={'name':'girish','age':34}
2. >>> d2={'marks':90}
3. >>> d.update(d2)
4. >>> d
5. {'name': 'girish', 'age': 34, 'marks': 90}
```

Dictionary values() method

The **values()** method returns a list of all the values available in a given dictionary.

Syntax

```
dict.values()
```

Parameters

NA

Return value

This method returns an index of the values in the dictionary.

Example

The following example shows the usage of the **values()** method:

```
1. >>> d={'name': 'girish', 'age': 34, 'marks': 90}
2. >>> print ("Values : ", list(d.values()))
3. Values :  ['girish', 34, 90]
```

Programs

1. Program to add a key-value pair to the dictionary:

```
1. d={'fruits':'apple','color':'pink'}
2. k=input("enter a key to be added: ")
3. v=input("enter its value: ")
4. d.update({k:v})
5. print("updated dictionary is :")
6. print(d)
```

Output:

```
enter a key to be added: vegetable
enter its value: tomato
updated dictionary is :
{'fruits': 'apple', 'color': 'pink', 'vegetable': 'tomato'}
```

2. Program to concatenate two dictionaries into one:

```
1. d={'fruits':'apple','color':'pink'}
2. d2={'name':'girish','age':34}
3. d.update(d2)
4. print("concatenated dictionary is :")
5. print(d)
```

Output:

```
concatenated dictionary is :
{'fruits': 'apple', 'color': 'pink', 'name': 'girish', 'age': 34}
```

3. Program to generate a dictionary that contains numbers (between 1 and n) in the (x,x*x) form:

```
4. n=int(input("enter a number: "))
5. d={x:x*x for x in range(1,n+1)}
6. print("the dictionary is ",d)
```

Output:

```
enter a number: 6
the dictionary is  {1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36}
```

4. Program to find the sum of all values of a dictionary:

```
1. d={'a':10,'b':20,'c':30,'d':40}
2. print("the sum of values in the dictionary is ")
3. print(sum(d.values()))
```

Output:

```
the sum of values in the dictionary is
100
```

5. Program to take a string as a parameter and generate a frequency of characters contained in it:

```

1. a=input("enter a string: ")
2. d=dict()
3. for i in a:
4.     if(i not in d):
5.         d[i]=1
6.     else:
7.         d[i]+=1
8. print("frequency of each character is ")
9. print(d)

```

Output:

```

enter a string: dictionary
frequency of each character is
{'d': 1, 'i': 2, 'c': 1, 't': 1, 'o': 1, 'n': 1, 'a': 1, 'r':
1, 'y': 1}

```

6. Program to count the frequency of words in a dictionary:

```

1. string=input("enter a string: ")
2. l=list()
3. d=dict()
4. l=string.split()
5. for i in l:
6.     if(i not in d):
7.         d[i]=1
8.     else:
9.         d[i]+=1
10. print("the frequency of words is ",d)

```

Output:

```

enter a string: abc xyz mno xyz pqrabcabc
the frequency of words is {'abc': 3, 'xyz': 2, 'mno': 1,
'pqr': 1}

```

7. Program to get a minimum and a maximum value in a dictionary:

```
1. d={'a':23, 'b':46, 'c':78, 'd':12}
2. max=max(d)
3. min=min(d)
4. print("maximum value is ",max)
5. print("minimum value is ",min)
```

Output:

```
maximum value is d
minimum value is a
```

8. Program to find the number of positive and negative numbers from a list of numbers:

```
1. l1=[23,-33,90,-3,-12,-90,37,6,-7]
2. d=dict()
3. d['pos']=0
4. d['neg']=0
5. for i in l1:
6.     if(i>0):
7.         d['pos']+=1
8.     else:
9.         d['neg']+=1
10. print(d)
```

Output:

```
{'pos': 4, 'neg': 5}
```

9. Program to convert octal to binary using a dictionary:

```
1. num=input("Enter a number")
2. table={'0':'000', '1':'001', '2':'010', '3':'011', '4':'100',
        '5':'101', '6':'110', '7':'111'}
3. b=""
4. for i in num:
5.     b+=table[i]
6. print("number in binary is ",b)
```

Output:

```
Enter a number32
number in binary is  ob011010
```

11. Program to replace dictionary values with their sum:

```
1.  d={'a':10,'b':20,'c':30,'d':40}
2.  result=1
3.  for key in d:
4.      result=result*d[key]
5.  print("the result is ")
6.  print(result)
```

Output:

```
the result is
240000
```

Conclusion

In this chapter, you learned about dictionaries in Python. You also understood the basic concepts related to creating a dictionary, accessing the values in it, etc. We discussed built-in functions in the dictionary elaborately and looked at examples to help learners understand the concept.

In the next chapter, we will study more about the functions in Python. We will discuss built-in functions and user-defined functions with examples.

Questions

1. How can you create a dictionary in Python?
2. How can you get the number of items in a dictionary?
3. Find the output of the following program:


```
1.  D = dict()
2.  for x in enumerate(range(2)):
3.      D[x[0]] = x[1]
4.      D[x[1]+7] = x[0]
5.  print(D)
```
4. Compare the setdefault() and get() methods.
5. What is the significance of the dictionary items() method?

CHAPTER 12

Functions

You can write more complicated programs in extended, complex modules, but detailed modules are harder to write and difficult to understand. A better approach is to divide a complicated program into smaller, simpler, more focused modules and functions.

A function in Python is a named block of code. It performs a particular task. We can break down our program into smaller, modular chunks using parts. As our program grows, functions make it more manageable and organized. Additionally, they avoid repetition and make code reusable.

Structure

We will cover the following topics in this chapter:

- Built-in functions
- User-defined functions
- Defining a function
- Function calling
- Docstring
- Return statement

- Pass by reference vs value
- Function arguments
- Required arguments
- Default arguments
- Keyword arguments
- Arbitrary arguments
- Anonymous functions
- Use of lambda function
- Scope of variable
- Recursion

Objectives

This chapter explores functions in Python. There are mainly two types of functions: built-in functions and user-defined functions. We will focus on explaining such concepts with examples. The current chapter emphasizes the core concepts related to functions like pass by reference vs value, recursion, and different types of arguments.

Built-in functions

Just like Python has built-in operators (such as plus and minus for addition, asterisk for multiplication, and so on), it has built-in functions. You can use them in your programs as pieces of code. A built-in function can be used by specifying its name and, typically, by providing some information. With that information, the function performs some work and returns the results. For example, **print()**, **len()**, **input()**, **sqrt()**, and so on.

User-defined functions

These are functions that we define ourselves to accomplish specific tasks.

User-defined functions have the following advantages:

- Using user-defined functions, a large program can be broken up into smaller parts that are easier to understand, maintain, and debug.

When repeated code appears in a program, you can include it and execute it when needed by calling a function.

Defining a function

- The `def` keyword marks the beginning of the function header, followed by the function name and parentheses ().
- The function name identifies it. In Python, function names also follow the same rules that are followed by identifiers.
- Parameters and arguments should be enclosed in parentheses; parameters can also be defined inside these parentheses.
- It describes what the function does with an optional documentation string (docstring).
- A colon (:) marks the end of the function header.
- At least one valid Python statement must appear in the body of the function; statements must be indented equally (usually 4 spaces).
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return none`.

Syntax

```
def function_name(parameters):
    """docstring"""
    statement(s)
```

Example

```
1. def first(name):
2.     """This is the first function"""
3.     print(name)
4.     return
```

Function calling

Defining a **function** gives it a name, which specifies the parameter that will be included within the function and the structure of the blocks of code in it.

When we define a function, we can call it from another function, program, or even the Python prompt. The name of the function and its parameters are typed to call it. Here's an example to call the **first()** function:

```
1. first('hello world')
2. hello world
```

Docstring

The **docstring** is the first string after the function header, and it stands for documentation string. It is used for briefly describing a function; documentation is a good programming practice, even if it is optional.

There is a docstring immediately below the function header in the preceding example. Usually, triple quotes are used as docstrings often span multiple lines. The string is available in the `__doc__` attribute of the function.

Example

```
1. def sum(a, b):
2.     ''' Adding two numbers '''
3.     return a + b
```

The docstring of the function can be printed using the following code:

```
print(sum.__doc__)
```

Return statement

The **return** statement is used to exit the function and return to its starting point.

Syntax

```
return [expression_list]
```

The statement can contain expressions that are evaluated, after which the value is returned. The function returns `None` if there is no expression in the statement or if the **return** statement itself is absent inside the function. Here's an example:

```
1. def first(name):
2.     """This is the first function"""
3.     return(name)
4. print(first('hello world'))
5. hello world
6. None
```

Example

```
1. def check(a):
2.     if a%2==0:
3.         ans="even"
4.     else:
```

```

5. ans="odd"
6.     return ans
7. m=int(input("enter a number: "))
8. print(check(m))
9. n=int(input("enter a number: "))
10.print(check(n))

```

Output

```

enter a number: 24
even
enter a number: 65
odd

```

Pass by reference vs. value

Consider this example for better understanding:

```

1. def change(l1):
2.     "changing the original list using function"
3.     print ("Values of list inside function before change: ", l1)
4.     l1[2]='banana'
5.     print ("Values of list inside function after change: ", l1)
6.     return
7.
8. l1 = ['apple','mango','litchi','orange']
9. change(l1)
10.print("\nValues of list are: ", l1) #values outside function

```

Python passes parameters (arguments) by reference, which means when you change what a parameter refers to within a function, the change also reflects in the calling function.

Output

```

Values of list inside function before change: ['apple', 'mango',
'litchi', 'orange']
Values of list inside function after change: ['apple', 'mango',
'banana', 'orange']
Values of list are: ['apple', 'mango', 'banana', 'orange']

```

Function arguments

These types of formal arguments can be used to call a function:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Required arguments

Positional arguments are known as **required arguments** and are passed to a function in the correct order. The arguments in the function call should match the number in the function definition.

Example

```
1. def cal(a,b):
2.     """program to calculate sum of 2 numbers"""
3.     sum=a+b
4.     return sum
5. a=int(input("enter 1st number"))
6. b=int(input("enter 2nd number"))
7. print(cal(a,b))
```

Output

```
enter 1st number23
enter 2nd number22
45
```

Here, **cal()** has two parameters. This function runs smoothly and without any errors since it is called with two arguments. If we call it with different arguments, the interpreter will complain. This function is called with one argument and without one argument, along with the respective error messages.

```
1. print(cal(a))
2. Traceback (most recent call last):
3.   File "<pyshell#1>", line 1, in <module>
4.     print(cal(a))
5. TypeError: cal() missing 1 required positional argument: 'b'
```

Default arguments

A **default argument** is an argument that assumes a default value if a value is not provided in the function call for that argument.

The user may not want to provide values for some parameters for some functions, so you may want to make them optional and use default values. Default argument values are used to accomplish this. Add the assignment operator (=) followed by the default value to the parameter name in the function definition if you want to specify default argument values for parameters.

A constant value should be used for the default argument; it should be an immutable value by default.

Example

```

1. defcal(a,b=20):
2.     """program to calculate sum of 2 numbers"""
3.     sum=a+b
4.     return sum
5. a=10
6. b=12
7. print(cal(a,b))
8. 22
9. print(cal(a))
10. 30

```

Note: We cannot have a parameter with a default argument value preceding a parameter without a default argument value in a function's parameter list.

The parameters are assigned values based on their positions. For example, **defcal(a, b=5)** is valid, but **defcal(a=5, b)** is not valid. Take a look at this:

```

1. a=10
2. b=12
3. print(cal(a,b=5))
4. 15
5. print(cal(a=5,b))
6. SyntaxError: positional argument follows keyword argument

```

Keyword arguments

Arguments related to function calls are known as **keyword arguments**. In function calls, parameter names are used to identify keyword arguments.

If you have some functions with many parameters and only want to specify some of them, you can call them keyword arguments because we now use the name (keyword) instead of the position (which we have been using all along) as the parameter name for the arguments to the function.

You can skip arguments or place them out of order because the Python interpreter can match the values with the parameters using the keywords provided. There are two advantages of Keyword arguments:

- The function is easier to use since we don't have to worry about the order of the arguments.
- Only the parameters we wish to set values for can be given values, provided that the others have default arguments.

Example

```
1. def cal(a,b):
2.     """program to calculate sum of 2 numbers"""
3.     sum=a+b
4.     return sum
5. print(cal(a=12,b=24))           # 2 keyword argument
6. 36
7. print(cal(b=24,a=12))          # 2 keyword argument(out of order)
8. 36
9. print(cal(24,b=4))             # 1 positional, 1 keyword
10. 28
11. print(cal(a=22,12))
12. SyntaxError: positional argument follows keyword argument
```

During a function call, we can mix keyword arguments with positional arguments. However, keyword arguments should follow positional arguments; positional arguments after keyword arguments will result in errors.

Arbitrary arguments

They are also known as variable length arguments.

In some cases, we do not know in advance how many arguments will be passed to a function. This kind of a situation can be handled in Python through function calls with **arbitrary arguments**. For this kind of an argument, we use an asterisk (*) before the parameter name in the function definition.

Example

```
1. def display( arg1, *vartuple ):  
2.     "Arbitrary arguments"  
3.     print ("Output is: ")  
4.     print (arg1)  
5.     for var in vartuple:  
6.         print (var)  
7.     return  
8. display( 34)  
9. display( 10,20,30,40 )
```

Output

Output is:

34

Output is:

10

20

30

40

Anonymous functions

An **anonymous function** in Python is one with no name. The `def` keyword is used to define normal functions, while the `lambda` keyword is used to define anonymous functions. So, anonymous functions are also called `lambda` functions. Here's what you need to know about them:

- `Lambda` forms can take any number of arguments, but they will return only one value in the form of an expression. Neither can they contain commands nor can they contain multiple expressions.
- `Lambda` requires an expression, so an anonymous function cannot be a direct call to `print`.
- In `lambda` functions, variables can only be accessed from the parameter list and from the global namespace, and they have their own local namespace.

- Although lambdas appear to be one-line versions of functions, they are not equivalent to inline statements in C or C++, whose purpose is to stack allocation by passing function during invocation for performance reasons.

Syntax

```
lambda [arg1 [,arg2,.....argn]]:expression
```

Example

```
1. double = lambda x: x * 2           # Lambda function
2. print(double(5))
3. 10
```

Use of lambda function

We use lambda functions when we need a nameless function for a short period of time. They are used as arguments to a higher-order function (a function that takes other functions as arguments). Lambda functions are commonly used as arguments to built-in methods like **filter()**, **map()**, and so on.

Example

A function and a list are both arguments for the **filter()** function in Python. With all the items in the list, the function is called and a new list is returned; it contains the items for which the function evaluates to True.

Here's an example showing the use of the **filter()** function to filter out even numbers from a list:

```
1. list1 = [1, 5, 4, 6, 8, 11, 3, 12]
2. list2 = list(filter(lambda x: (x%2 == 0) , list1))
3. print(list2)
```

Output

```
[4, 6, 8, 12]
```

Python's **map()** function takes a function and list. A list containing the items returned by the function for each list element is returned when the function is called with all items in a list.

For example, the use of the **map()** function to double all the items in a list:

```
1. list1 = [1, 5, 4, 6, 8, 11, 3, 12]
2. list2 = list(map(lambda x: x * 2 , list1))
3. print(list2)
```

Output

```
[2, 10, 8, 12, 16, 22, 6, 24]
```

Scope of variable

- A variable's scope is the area of a program where it is recognized. A function's parameters and variables are not visible from outside, so they have local scope.
- The life time of a variable is the period during which it remains in memory. Variables inside a function exist for the duration of the function; they are destroyed once we return from the function. As a result, a function does not remember the value of a variable from its previous calls.
- Variable scope defines which part of the program can access a particular identifier. In Python, variables fall into two basic scopes:
 - Global variables
 - Local variables

The variables inside a function body have a local scope, while the variables defined outside have a global scope.

Local variables can only be accessed within the function in which they are declared, while global variables can be accessed by all functions throughout the program body. The variables declared inside a function are brought into scope as soon as you call them.

Here's a simple example:

```
1. def func():
2.     str = "hello"
3.     print("Value of str inside function:",str)
4. str = "world"
5. func()
6. print("Value of str outside function:",str)
```

Output

```
Value of str inside function: hello
```

```
Value of str outside function: world
```

Recursion

Recursion is the process of defining something in terms of itself. This refers to the ability to save its state or value in register or stack before calling itself.

A **recursive function** is a function that calls itself till it is terminated by an exit condition. A recursive function usually returns the value of the function call.

Example

```
1. def fact(x):
2.     """This is a recursive function
3.     to find the factorial of an integer"""
4.     if x == 0 or x == 1:
5.         return 1
6.     else:
7.         return (x * fact(x-1))
8. n = int(input("Enter a number: "))
9. print("The factorial of", n, "is", fact(n))
```

Output

Enter a number: 5

The factorial of 5 is 120

While calling

fact(5)

5 * fact(4)

5 * 4 * fact(3)

5 * 4 * 3 * fact(2)

5 * 4 * 3 * 2 * fact(1)

While returning

5 * 4 * 3 * 2 * 1

5 * 4 * 3 * 2

5 * 4 * 6

5 * 24

120

Advantages of recursion

- Recursive functions make the code look clean and elegant
- A complex program can be broken down into simpler sub-problems using recursion
- Sequence generation is easier with recursion as compared to using some nested iteration

Disadvantages

- The logic behind recursion becomes very complex sometimes
- Recursive calls are expensive (inefficient) as they take up a lot of memory and time
- Recursive functions are hard to debug

Comparisons between iteration and recursion

- In recursion, a function calls itself until the base condition is reached. On the other hand, iteration means the repetition of code until a condition fails.
- Iteration involves four steps: initialization, condition, execution, and updation; recursion only has a base condition.
- Recursion is slower than iteration and takes more memory than iteration due to the overhead of maintaining stack.

Programs

1. Program to find the maximum of three numbers:

```

1. defmaxm(a,b,c):
2.     if a>b and a>c:
3.         max=a
4.     elif b>c and b>a:
5.         max=b
6.     elif c>a and c>b:
7.         max=c
8.     return max
9. a=int(input("enter a: "))

```

```
10. b=int(input("enter b: "))
11. c=int(input("enter c: "))
12. print("largest number is ",maxm(a,b,c))
```

Output

```
enter a: 24
enter b: 45
enter c: 33
largest number is  45
```

2. Program to add all the numbers in a list:

```
2. def sum(l1):
3.     sum=0
4.     for i in l1:
5.         sum+=i
6.     return sum
7. l1=[20,15,8,22,10]
8. print("sum of list is",sum(l1))
```

Output

```
sum of list is 75
```

3. Program to reverse a string:

```
3. def reverse(str):
4.     l=len(str)
5.     rev=""
6.     for i in str:
7.         rev=i+rev
8.     return rev
9. str=input("enter a string: ")
10. print("the reverse of string is ",reverse(str))
```

Output

```
enter a string: hello
the reverse of string is  olleh
```

4. Python function to calculate the factorial of a number; the function accepts the number as an argument:

```
1. def facto(n):
2.     fact=1
3.     for i in range(1,n+1):
4.         fact=fact*i
5.     return fact
6. n=int(input("enter a number: "))
7. print("factorial of %d is"%(n),facto(n))
```

Output

```
enter a number: 5
factorial of 5 is 120
```

5. Program that accepts a list and returns a new list with the unique elements of the first list:

```
5. def unique(l):
6.     x=[]
7.     for a in l:
8.         if a not in x:
9.             x.append(a)
10.    return x
11. print(unique([1,2,4,6,7,6,5,3,5,6,6]))
```

Output

```
[1, 2, 4, 6, 7, 5, 3]
```

6. Program to accept a string and calculate the number of uppercase and lowercase characters:

```
6. def count(str):
7.     uc=0
8.     lc=0
9.     for i in str:
10.        if ord(i)>=65 and ord(i)<=90:
11.            uc+=1
```

```
12. elif ord(i)>=97 and ord(i)<=122:
13.     lc+=1
14.     return lc,uc
15. str=input("enter a string")
16. lc,uc=count(str)
17. print("Uppercase characters= ",uc)
18. print("lowercase characters= ",lc)
```

Output

```
enter a string: Hello World
Uppercase characters=  2
lowercase characters=  8
```

7. Program to calculate the sum of a series:

$1/2 - 3/4 + 5/6 \dots\dots\dots n$

```
1. n=int(input("enter n: "))
2. j=1
3. s=0
4. for i in range(1,n+1):
5.     s=s+j/(j+1)*pow(-1,i+1)
6.     j+=2
7. print("sum of series is")
8. print(s)
```

Output

```
enter n: 2
sum of series is
-0.25
```

8. Program to check whether a number is a perfect number:

```
7. #perfect number
8. def perfect(n,sum1):
9.     for i in range(1, n):
10.         if(n % i == 0):
11.             sum1 = sum1 + i
```

```

12.     return sum1
13. n = int(input("Enter any number: "))
14. sum1 = 0
15. ans=perfect(n,sum1)
16. if (ans == n):
17.     print("The number is a Perfect number!")
18. else:
19.     print("The number is not a Perfect number!")

```

Output

```

Enter any number: 6
The number is a Perfect number!

```

9. Python program to evaluate a binomial expression:

```

9. def fact(n):
10.     f=1
11.     for i in range(1,n+1):
12.         f=f*i
13.     return f
14. n=int(input("enter value of n: "))
15. r=int(input("enter value of r: "))
16. ans=fact(n)/fact(r)/fact(n-r)
17. print("binomial coefficient is ",ans)

```

Output

```

enter value of n: 8
enter value of r: 3
binomial coefficient is  56.0

```

10. Program to find the HCF of two numbers:

```

10. defhcf(x, y):
11.     if x > y:
12.         small = y
13.     else:
14.         small = x

```

```
15.     for i in range(1,small + 1):
16.         if((x % i == 0) and (y % i == 0)):
17.             hcf = i
18.     return hcf
19. a = int(input("Enter first number: "))
20. b = int(input("Enter second number: "))
21. print("The H.C.F. of", a,"and", b,"is", hcf(a, b))
```

Output

Enter first number: 128

Enter second number: 36

The H.C.F. of 128 and 36 is 4

11. Program to generate a Fibonacci series using recursion:

```
1.  # fibonacci
2.  def fibo(n):
3.      if n==0:
4.          return 0
5.      if n==1 or n==2:
6.          return 1
7.      else:
8.          return(fibo(n-1)+fibo(n-2))
9.  n=int(input("enter number of terms: "))
10. print("fibonaccies is ")
11. for i in range(0,n):
12.     print(fibo(i),end=" ")
```

Output

enter number of terms: 6

fibonaccies is

0 1 1 2 3 5

12. Program to reverse a string using recursion:

```
12. def reverse(s1):
13.     if(len(s1))==0:
```

```
14.         return s1
15.     else:
16.         return reverse(s1[1:])+s1[0]
17. s1=input("enter a string: ")
18. print("reversed string is ")
19. print(reverse(s1))
```

Output

```
enter a string: hello python
reversed string is
nohtypolleh
```

Conclusion

In this chapter, we learned about functions, their importance in Python, and their types. We also looked at the different types of arguments used in Python, along with their examples. Finally, we compared pass by reference vs value.

In the next chapter, we will focus on the concept of modules in Python. Modules are files containing Python code. They can define functions, classes, and variables.

Key terms

- Built-in function
- User-defined function
- Function calling
- Pass by reference
- Pass by value

CHAPTER 13

Modules

Introduction

A Python module is the highest level of program organization, packing code, and data for reuse and easy interoperability at the functional level and for providing self-contained namespaces to prevent variables from clashing across different programs.

Modules are files containing Python code that define functions, classes, and variables. Runnable code can also be included in a module. These pieces should be shared, so Python allows a module to "bring in" and use attributes from other modules to take advantage of work that has been done, maximizing code reusability. This process of associating the attributes from other modules with your module is called importing.

Structure

We will cover the following topics in this chapter:

- The import statement
- The from..import statement
- The from..import * statement
- Executing modules as scripts

- The module search path
- The `dir()` function
- The `globals()` and `locals()` functions
- The reload function
- Packages in Python

Objective

This chapter aims to help the reader understand the Python module. It also explains the use of import statements in Python and focuses on executing modules as scripts in Python. Finally, the chapter explores the packages in Python.

The import statement

Any Python source file can be used as a module by executing an **import** statement in another Python source file.

Syntax

```
import module1[, module2[,... moduleN]
```

When the interpreter finds an import statement, it imports the module if it is present in the search path.

Search path: The search path refers to a list of directories that the interpreter searches before importing a module.

For example, this is a **test.py** module:

```
1. def print_func( p ):
2.     print "Hello : ", p
3.     return
```

To import the **test.py** module, you need to put the import test at the top using the following command:

```
1. import support
2. support.print_func("Sukhman")
```

If the code is executed, the result will be as follows:

Hello: Sukhman

The from...import statement

You can import specific attributes from a module into the current namespace using Python's **from** statement.

Syntax

```
from modname import name1[, name2[, ... nameN]]
```

For example, import function Fibonacci from the **fib** module:

```
1. def fib(n):
2.     r=[]
3.     a, b = 0, 1
4.     while (b<n):
5.         r.append(b)
6.         a, b = b, a+b
```

Use the following to import the **fib** function from the **Fibonacci.py** module:

```
1. from Fibonacci import fib
2. fib(50)
3. [1, 1, 2, 3, 5, 8, 13, 21, 34]
```

The statement does not import the entire Fibonacci module into the current namespace; it just introduces the **fib** item from the Fibonacci module into the global symbol table of the importing module.

The from...import * statement

All the names in a module can be imported into the current namespace using the following import statement.

Syntax

```
from modname import *
```

A module can be imported into the current namespace using this statement, but the information should be used sparingly.

Executing modules as scripts

Module names (as strings) are available as the value of a global variable `__name__` within a module. The code in the module is executed as you imported it, but the `__name__` will be set to `"__main__"`.

Consider that you run a Python module with:

```
Python fibo.py <arguments>
```

Then, the module code will be executed just as it had been imported, but `__name__` will be set to `"__main__"`. When we add this code at the end of your module, you can execute modules as scripts:

1. If `__name__=="__main__"`:
2. Import `sys`
3. `Fib(int(sys.argv[1]))`

Since the code that parses the command line only runs when the module is executed as the **main** file, it can be used as both a script and an importable module:

1. `$ Python fibo.py 50`
2. `1 1 2 3 5 8 13 21 34`

The code is not run if the module is imported:

1. `import fibo`

The module search path

The interpreter first searches for the built-in module named `spam` when importing a module named `spam`. If **spam.py** is not found, the **sys.path** variable is used to search a list of directories for a file named **spam.py**. These locations are used to initialize **sys.path**:

- The directory that contains the input script (or the current directory)
- This is the `Pythonpath` (a list of directory names with the same syntax as the shell variable `PATH`)
- Installation-dependent defaults

Python programs can modify **sys.path** during initialization. The directory containing the script being run is placed at the start of the search path, before the standard library path. This means when scripts are loaded from that directory, modules from the library directory of the same name are not loaded. Unless the replacement is intended, this is an error.

Note: Here is a typical PYTHONPATH from a Windows system.

```
set PYTHONPATH=c:\python34\lib;
```

The dir() function

Directory() returns a sorted list of strings containing the names defined in a module. The list consists the names of all the variables, and functions defined in a module.

Here's an example:

```
1. import math
2. con = dir(math)
3. print (con)
```

When the preceding code is executed, the result will be as follows:

```
['__doc__', '__loader__', '__name__', '__package__', 'acos', 'acosh',
'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos',
'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs',
'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'hypot',
'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p',
'log2', 'modf', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan',
'tanh', 'trunc']
```

In this example, `__name__` is the module's name, and `__file__` is the filename from which the module was loaded.

The globals() and locals() functions

You can use **globals()** and **locals()** to return names in the global and local namespaces depending on where they are called.

- If the **locals()** method is called from within a function, it returns all the names that can be accessed locally.
- When the **globals()** method is called from within a function, it will return all the names that can be accessed globally.

Both of these functions return dictionaries, so names can be extracted using the **keys()** function.

The reload() function

The code in the top portion of a module is only executed once when imported into a script. So, you can use the **reload()** function if you want to rerun the top-level code in a module; it imports a previously imported module again.

Syntax

```
reload(module_name)
```

In this case, `module_name` is the name of the module you want to reload, not the string containing the module name.

For example, do the following to reload fibonacci module:

```
1. reload(fibonacci)
```

Packages in Python

Modules and sub packages make up a package, which is a hierarchical file directory structure for defining a single Python application environment. Packages were added to Python 1.5 to overcome various problems, including:

- Adding hierarchical organization to flat namespace
- Allowing developers to group related modules
- Allowing distributors to ship directories versus a bunch of files
- Helping resolve conflicting module names

Along with classes and modules, packages use the familiar attribute/dotted attribute notation to access their elements. Importing modules within packages needs the standard `import` and `from-import` statements.

Suppose a **Pots.py** file is available in phone directory. It has the following line of source code:

```
1. #!/user/bin/python3
2. def Pots():
3.     print ("I'm Pots Phone")
```

Similarly, we have two other files with different functions but the same name as above. They are as follows:

- The **Phone/Isdn.py** file with the **Isdn()** function
- The **Phone/G3.py** file with the **G3()** function

Now, create the `__init__.py` file in the **Phone** directory:

- Phone/___init__.py

```
3.  
4.  def cube(x):  
5.                                     return x*x*x
```

Python interpreter

```
1.  import num  
2.  num.square(3)  
3.                                     9  
4.  num.cube(3)  
5.      27
```

3. Program to print the current time:

```
1.  import time  
2.  print(time.asctime())  
3.      'Wed Apr 9 02:54:21 2018'
```

4. Program to print the current working directory:

```
1.  import os  
2.  print(os.getcwd.__doc__)
```

Conclusion

In this chapter, we learned about the concept of Python modules. We also discussed the import statements and their various usages in Python with examples. Additionally, we explored the packages in Python.

In the next chapter, we will learn about files I/O.

Questions

1. What is a module in Python?
2. What is an import statement in Python?
3. Explain the `dir()` function with an example.
4. Compare the `globals()` function and the `locals()` function.
5. What is the `reload()` function?
6. Explain packages in Python.

CHAPTER 14

Files I/O

Introduction

You have been reading and writing to the standard input and output until now. Now, we will see how to use actual data files.

Python provides the basic functions and methods necessary to manipulate files by default. However, you can perform file manipulation using a **file** object.

Python provides numerous built-in functions that are readily available to us at the Python prompt. Some of the functions, like **input()** and **print()**, are widely used for standard input and output operations, respectively. Let's take a look at the output section first.

Python output using the **print()** function: We use the **print()** function to output data to the standard output device (screen). We can also output data to a file, but we will discuss this later.

Here's an example of its use:

```
print('This sentence is output to the screen')
```

Our programs were static until now, and the value of variables was defined or hard coded into the source code.

To allow flexibility, we might want to take the input from the user. In Python, we have the **input()** function to allow this. The syntax for **input()** is as follows:

```
>>> num = input('Enter a number: ')
Enter a number: 10
>>> num
'10'
```

Structure

We will cover the following topics in this chapter:

- Open and close files
- Modes of opening a file
- The file object attributes
- Reading and writing files
- Renaming and deleting files
- File methods

Objectives

The main objective of this chapter is to help the learner understand all I/O functions available in Python. It intends to explain the concept of opening and closing files in Python and reading, writing, renaming, and deleting files. Finally, it explores the file methods available in Python.

Open and close files

So far, your input and output have been through the standard channels. We will now see how to use actual data files.

The open function

Before you can read or write a file, you have to open it using Python's built-in **open()** function. This function creates a file object.

Syntax:

```
file object = open(file_name [, access_mode][, buffering])
```

Parameters:

- **file_name:** The **file_name** argument contains the name of the file you wish to access.
- **access_mode:** Using **access_mode**, you can determine how the file should be opened; for example, reading, writing, or appending. As far as file access modes are concerned, read (r) is the default.
- **buffering:** Buffering is not performed if the buffering value is set to 0. When the buffering value is 1, line buffering is performed when accessing a file. If you specify more than one as the buffering value, the buffer size will be indicated. The default buffer size (default behavior) is negative.

Modes of opening a file

The following table lists all the different modes of file operations:

Modes	Descriptions
R	Allows you to read only a file; at the start of the file is the file pointer. This is the default mode.
Rb	Opens a binary file for reading only; at the start of the file is the file pointer. This is the default mode.
r+	Allows both reading and writing of a file. The starting point of a file.
rb+	Opens a binary file for both reading and writing. The starting point of a file.
W	Allows only writing to a files. If the file is already exists, it will be overwritten; it creates a new file if the file does not exist.
Wb	Opens a binary-only file for writing. If the file already exists, it is overwritten, or it creates a new file if the file does not exist.
w+	Allows both writing and reading of a files. If the file is already exists, it is overwritten. It creates a new file for reading and writing if the file does not exist.
wb+	Opens a binary file for both writing and reading. If the file is already exists, it is overwritten, or it creates a new files for reading and writing if the file does not exist.

A	Appends a file. If the file exists, the file pointer is at the end of the file and is appended to the existing file. The program creates a new file if the file does not exist.
Ab	Appends a binary file to a file. If the file exists, the file pointer is at the end of the file, which is in append mode. It creates a new one if the file does not exist.
a+	Appends and reads a file. If the file is exists, the file pointer is at the end of the file. The file opens in the append mode. It creates a new file for reading and writing if the file does not exist.

Table 14.1: Modes of opening a file

The file object attributes

Once a file is opened and we have one file object, we can use some attributes to get information about it.

The following are the attributes:

- **file.closed**: Returns true if file is closed; returns false otherwise
- **file.mode**: Returns the access mode with which the file was opened
- **file.name**: Returns the name of the file

Example:

```
1. fo = open("foo.txt", "wb")
2. print ("Name of the file: ", fo.name)
3. print ("Closed or not : ", fo.closed)
4. print ("Opening mode : ", fo.mode)
5. fo.close()
```

Output:

```
Name of the file:  foo.txt
Closed or not :  False
Opening mode :  wb
```

The close() method

A file object's **close()** method flushes any unwritten information and closes it.

The reference object of the file is reassigned to another file. Python automatically closes the file, but it is still a good practice to close a file using the **close()** method.

Syntax:

```
fileObject.close();
```

Reading and writing files

We will look at how the **read()** and **write()** methods are used to read and write files.

The write() method

The **write()** method writes any string to an open file. It does not add a newline character ('\n') to the end of the string.

Syntax:

```
fileObject.write(string);
```

Parameter:

String: Content to be written into the opened file.

Example:

1. file = `open("test.txt", "w")`
2. file.`write( "Work hard to achieve success.\nKeep moving!!\n")`
3. file.`close()`

If you open the file named **test**, it will contain the following content:

Work hard to achieve success.

Keep moving!!

The read() method

The **read()** method reads a string from an open file.

Syntax:

```
fileObject.read([count]);
```

Parameter:

- **Count:** This is the number of bytes to read from an opened file. If count is missing while trying to read as much data as possible, and perhaps until the end of the file, then this method starts reading from the beginning of the file.

Example:

1. file = `open("test.txt", "r+")`
2. str = file.`read(10)`

```
3. print ("Read String is : ", str)
4. file.close()
```

Output:

Work hard

Note: Python strings can have binary data and not just text.

Renaming and deleting files

The rename and delete operations are performed by the OS module of Python. To use it, we need to import it and then call related functions.

The rename() method

The **rename()** method changes the current filename to the new filename.

Syntax:

```
os.rename(current_file_name, new_file_name)
```

Example:

```
1. Import os
2. os.rename( "test1.txt", "test2.txt" )
```

The remove() method

The **remove()** method is used to delete files.

Syntax:

os.remove(file_name)

Example:

```
1. import os
2. os.remove("text2.txt")
```

File methods

File objects are created using the open function, and the following functions can be called on this object.

File flush() method

As with `stdio's fflush`, **flush()** flushes the internal buffer; this may be a no-op on some file-like objects.

Note: Python automatically flushes files when they are closed. Before closing a file, flush the data if necessary.

Syntax:

```
fileObject.flush()
```

Parameters:

NA

Return value:

It does not return any value.

Example:

```
1. file = open("test.txt", "wb")
2. file.flush()
3. file.close()
```

File fileno() method

Using **fileno()**, the underlying implementation can request I/O operations from the operating system based on a file descriptor.

Syntax:

fileObject.fileno()

Parameters:

NA

Return value:

Integer file descriptors are returned by the method.

Example:

```
1. file = open("test.txt", "wb")
2. fd = file.fileno()
3. print ("File Descriptor: ", fd)
4. file.close()
```

Output:

File Descriptor: 3

File `isatty()` method

The `isatty()` method returns `true` if the file is connected (to a secondary device) to a `tty`-(like) device, otherwise it returns `false`.

Syntax:

```
fileObject.isatty()
```

Parameters:

NA

Return value:

This method returns `true` if the file is connected (is associated with a terminal device) to a `tty`-(like) device, otherwise it returns `false`.

Example:

```
1. file = open("test.txt", "wb")
2. ret = file.isatty()
3. print ("Return value : ", ret)
4. file.close()
```

Output:

Return value : False

File `next()` method

There is no `next()` method available in the Python 3 file object. By calling its `__next__()` method, Python 3's `next()` function retrieves the next item from an iterator. The default is returned if the iterator is exhausted; otherwise, **StopIteration** is raised. You can use this method to read the next input line from the file object.

Syntax:

```
next(iterator[,default])
```

Parameters:

- **iterator** : The file object from which the lines are to be read
- **default** : This is returned if the iterator is exhausted; **StopIteration** is raised if not given

Return value:

Returns the next input line.

Example:

Let's assume that a test.txt file contains the following lines:

Computer Networks

Discrete Structures

Functional English

Consider that we write the following program:

```
1. file = open("test.txt", "r")
2. for index in range(5):
3.     line = next(file)
4.     print ("Line No %d - %s" % (index, line))
5. file.close()
```

Output:

Line No 0 - Computer Networks

Line No 1 - Discrete Structures

Line No 2 - Functional English

File readline() method

The **readline()** method reads a single line from the file. The string contains a trailing newline character.

Syntax:

```
fileObject.readline( size )
```

Parameters:

- **size** - This is the number of bytes to be read from the file.

Return value:

Lines can be read from a file with this method. A non-negative size argument represents the maximum number of bytes, including the trailing newline. An incomplete line may be returned if the size argument is not given. An empty string is returned when EOF is encountered immediately.

Example:

Let's assume that a **test.txt** file contains the following lines:

Computer Networks

Discrete Structures

Functional English

Consider that we write the following program:

```
1. file = open("test.txt", "r+")
2. line = file.readline()
3. print ("Read Line: %s" % (line))
4. line = file.readline(3)
```

Output:

Read Line - Computer Networks

Read Line - Discrete Structures

Read Line - Functional English

File readlines() method

The **readlines()** method reads until EOF using **readline()** and returns a list containing the lines.

Syntax:

```
fileObject.readlines( sizehint )
```

Parameters:

- **sizehint** - The number of bytes to read from the file

Return value:

The method returns a list of lines. An optional sizehint argument causes whole lines (possibly rounding up to an internal buffer size) to be read instead of reading up to EOF. An empty string is returned when EOF is encountered immediately.

Example:

Let's assume that a **test.txt** file contains the following lines:

Computer Networks

Discrete Structures

Functional English

Consider that we write the following program:

```
1. file = open("test.txt", "r+")
2. line = file.readlines()
3. print ("Read Line: %s" % (line))
```

```
4. line = file.readlines(2)
5. print ("Read Line: %s" % (line))
6. file.close()
```

Output:

```
Read Line: [Computer Networks\n', Discrete Structures\n',
           Functional English\n']
```

Read Line:

File seek() method

The **seek()** method sets the file's current position at the offset. There are two values for the whence argument: 0, which means absolute file positioning, and 1, which means seeking relative to the current position and 2 means seeking relative to the file's end. There is no return value.

Note:

- Using either 'a' or 'a+' for opening the file for appending will undo any **seek()** operation at the next write.
- This method is essentially no-op when an append mode file is opened, but it remains useful for apps opened in reading mode (mode 'a+').
- If the file is opened in the text mode with 't', only offsets returned by **tell()** are valid. A different offset causes undefined behavior.

Note that not all objects are seekable.

Syntax:

```
fileObject.seek(offset[, whence])
```

Parameters:

- **offset**- This is the position of the read/write pointer within the file.
- **whence**- By default, 0 indicates that the position of the file is absolute, 1 means that the position is relative to the current position, and 2 means that the position is relative to the file's end.

Return value:

It does not return any value.

Example:

Let's assume that a **test.txt** file contains the following lines:

```
Computer Networks
```

Discrete Structures

Functional English

Consider that we write the following program:

```
1. file = open("test.txt", "rw+")
2. line = file.readlines()
3. print ("Read Line: %s" % (line))
4. file.seek(0, 0)
5. line = file.readline()
6. print ("Read Line: %s" % (line))
7. file.close()
```

Output:

```
Read Line: [Computer Networks\n', Discrete Structures\n',
'Functional English\n']
```

```
Read Line: Computer Networks
```

File tell() method

The `tell()` method returns the current position of the file read/write pointer within the file.

Syntax:

```
fileObject.tell()
```

Parameters:

NA

Return value:

The method returns the current position of the file read/write pointer.

Example:

Let's assume that a `test.txt` file contains the following lines:

Computer Networks

Discrete Structures

Functional English

Consider that we write the following program:

```
1. file = open("test.txt", "r+")
2. line = file.readline()
```

```
3. print ("Read Line: %s" % (line))
4. pos=file.tell()
5. print ("current position : ",pos)
6. file.close()
```

Output:

Read Line: Computer Networks

Current Position: 18

Programs

1. Program to write content in a file:

```
1. fo = open("file.txt", "w")
2. fo.write( "A real man spends time with his family.\n")
3. print("Data added to the file.")
4. fo.close()
```

Output:

Data added to the file.

2. Program to read the contents of a file:

```
1. fo = open("file.txt", "w")
2. fo.write( "A real man spends time with his family.\n")
3. print("Data added to the file.")
4. fo.close()
```

Output:

Enter the name of the file with .txt extension: file.txt

A real man spends time with his family.

3. Program to count the number of words in a text file:

```
1. fname=input("Enter file name: ")
2. num_words=0
3. with open(fname,"r") as f:
4.     for line in f:
5.         words=line.split()
6.     num_words+=len(words)
```

```
7. print("Number of words: ")
8. print(num_words)
```

Output:

```
Enter file name: file.txt
Number of words:
8
```

4. Program to count the number of lines in a text file:

```
1. fname=input("Enter file name: ")
2. num_lines=0
3. with open(fname,"r") as f:
4.     for line in f:
5.         num_lines+=1
6. print("Number of lines: ")
7. print(num_lines)
```

Output:

```
Enter file name: file.txt
Number of lines:
1
```

5. Program to read a string from the user and append it into a file:

```
1. fname=input("Enter a file: ")
2. file3=open(fname,"a")
3. c=input("Enter string to append: \n")
4. file3.write("\n")
5. file3.write(c)
6. file3.close()
7. print("Contents of append file: ")
8. file4=open(fname,"r")
9. line1=file4.readline()
10. while(line1!=""):
11.     print(line1)
12.     line1=file4.readline()
```

Output:

A real man spends time with his family.

6. Program to count the number of occurrences of a word in a text file:

```

1. fname=input("Enter file name: ")
2. word=input("Enter word to be searched: ")
3. k=0
4. with open(fname,"r") as f:
5.     for line in f:
6.         words=line.split()
7.         for i in words:
8.             if(i==word):
9.                 k=k+1
10. print("Occurrences of the words: ")
11. print(k)

```

Output:

Enter file name: file.txt

Enter word to be searched: a

Occurrences of the words:

4

7. Program to copy the content of one file into another:

```

1. with open("test.txt") as f:
2.     with open("out.txt","w") as f1:
3.         for line in f:
4.             f1.write(line)

```

8. Program that reads a text file and counts the number of times a certain letter appears in it:

```

1. fname=input("Enter file name: ")
2. l=input("Enter letter to be searched: ")
3. k=0
4. with open(fname,'r') as f:

```

```
5.         for line in f:
6.             words=line.split()
7.             for i in words:
8.                 for letter in i:
9.                     if(letter==l):
10.                         k=k+1
11. print("Occurrences of the letter: ")
12. print(k)
```

Output:

```
Enter file name: file.txt
Enter letter to be searched: t
Occurrences of the letter:
6
```

9. Program to read a text file and print all the numbers present in it:

```
1. fname=input("Enter file name: ")
2. with open(fname,'r') as f:
3.     for line in f:
4.         words=line.split()
5.         for i in words:
6.             for letter in i:
7.                 if(letter.isdigit()):
8.                     print(letter)
9.                     k=1
10. if(k==0):
11.     print("No digit.")
```

Output:

```
Enter file name: file.txt
No digit.
```

10. Program to append the contents of one file to another:

```
11.name1=input("Enter file to be read from: ")
12.name2=input("Enter file to be appended to: ")
```

```

13. fin=open(name1,"r")
14. Data2=fin.read()
15. fin.close()
16. fout=open(name2,"a")
17. fout.write(data2)
18. fout.close()

```

11. Program to count the number of blank spaces in a text file:

```

1.  fname=input("Enter file name: ")
2.  k=0
3.  with open(fname,"r") as f:
4.      for line in f:
5.          words=line.split()
6.          for i in words:
7.              for letter in i:
8.                  if(letter.isspace):
9.                      k=k+1
10. print("Occurrences of the letter: ")
11. print(k)

```

Output:

```

Enter file name: file.txt
Occurrences of the letter:
96

```

Conclusion

In this chapter, we learned about different I/O functions in Python. We also learned about the opening and closing of files in Python and understood different file methods with the help of examples. Additionally, we covered reading, writing, renaming and deleting files with examples.

In the next chapter, we will learn about exception handling in Python.

Key terms

- `open()`
- `close()`
- `rename()`
- `read()`
- `write()`
- `remove()`

CHAPTER 15

Exception Handling

We might have encountered several kinds of run-time errors while writing programs, such as integer division by zero, accessing a list with an out-of-range index, using an object reference set to None, and attempting to convert a non-number to an integer. To this point, all of our run-time errors have resulted in the program's termination. Python provides an exception handling mechanism that allows programmers to deal with run-time errors and much more. Rather than permanently terminating execution, a program can detect the problem and execute code to correct the issue or manage it in other ways.

Instead, it raises an exception, indicating that something exceptional has happened. Then, Python stops what it's doing and displays an error message detailing the exception if nothing is done about it.

Structure

We will cover the following topics in this chapter:

- Standard exceptions
- Python assertion
- Handling an exception
- The except clause with else clause

- With no exceptions except close
- The except clause with multiple exceptions
- Try-finally clause
- With statement
- Argument of an exception
- Raising an exception
- User-defined exceptions

Objectives

The main objective of this chapter is to help the learner understand the use of exceptions in Python. It explains the concept of handling exceptions in Python with examples and walks learners through the idea of raising an exception in Python. Several examples are included in the chapter for a practical understanding of the concepts.

Let's take a simple example of Python raising an exception:

1. `s="hello"`
2. `int(s)`
3. Traceback (most recent call last):
4. `File "<pyshell#1>", line 1, in <module>`
5. `int(s)`
6. `ValueError: invalid literal for int() with base 10: 'hello'`

Python tries to convert the string "Hello" to an integer. Since it cannot do this, Python raises an exception and displays the details.

Using Python's exception handling functionality, we can intercept and handle exceptions so that our program doesn't end abruptly (even if a user enters "Hello" when we ask for a number). At the very least, we can have our program exit gracefully instead of crashing awkwardly.

Python provides two important features to handle any unexpected error in your program and add debugging capabilities in them:

- Exception handling
- Assertions

Standard exceptions

Here's the list of standard exceptions available in Python:

Sr.No	Exception Name	Description
1	Exception	Base class for all exceptions.
2	StopIteration	Raised when the next() method of an iterator does not point to any objects.
3	SystemExit	Raised by sys.exit() .
4	StandardError	This class contains all built-in exceptions except for StopIteration and SystemExit .
5	ArithmeticError	Error class for all numeric calculation errors.
6	OverflowError	This error occurs when a calculation exceeds the maximum limit for a numeric type.
7	FloatingPointError	Raised when a floating-point calculation fails.
8	ZeroDivisonError	Division or modulo by zero is raised for all numeric types.
9	AssertionError	Raised in case of failure of the Assert statement.
10	AttributeError	Raised if the attribute reference or assignment fails.
11	EOFError	The end file is reached when there is no input from either raw_input() or input() .
12	ImportError	Raised when an import statement fails.
13	KeyboardInterrupt	A program execution interrupt is raised when the user presses Ctrl+c to interrupt it.
14	LookupError	The base class for all lookup will throw an error.
15	IndexError	An index cannot be found in a sequence.
16	KeyError	This error occurs when the specified key cannot be found in the dictionary.
17	NameError	Error raised when an identifier cannot be found in the local or global namespace.

18	UnboundLocalError	This error occurs when attempting to access a local variable in a function or method with no value assigned to it.
19	EnvironmentError	All exceptions occurring outside the Python environment belong to this class.
20	IOError	When input or output operations fail, such as print, or when opening a non-existent file with the open() method.
21	OSError	Operating system(OS)-related error will occur.
22	SyntaxError	Python syntax error will occur.
23	IndentationError	Error is raised when indentation is not properly specified.
24	SystemError	An internal error is raised when the Python interpreter encounters it, but the interpreter does not exit when the error occurs.
25	SystemExit	Raised whenever the Python interpreter is terminated using the sys.exit() function. If not handled in the code, the interpreter will exit.
26	TypeError	An exception is raised when a data type is not supported by the operation or function.
27	ValueError	The error occurs when a built-in function for a data type has valid arguments but their values are invalid.
28	RuntimeError	A generated error that does not fit into any of the categories.
29	NotImplemented Error	A raised exception when an abstract method that must be implemented in an inherited class is not implemented.

Table 15.1: Standard exceptions available in Python

Python assertions

The **assert** statement performs debugging checks. If it encounters an **assert** statement, Python evaluates the accompanying expression, which is hopefully true.

If the expression is false, it raises an Assertion Error and passes the message as its constructor argument, if provided.

Syntax

```
assert Expression[, Arguments]
```

Python uses **ArgumentExpression** as an argument for the **AssertionError** when the assertion fails. The try-except statement can be used to catch and handle AssertionError exceptions. The program will terminate and a traceback will appear if they are not handled.

Example

```
1. def avg(list1):
2.     length=len(list1)
3.     assert length!=0,"List is empty"
4.     sum=0
5.     for i in list1:
6.         sum+=i
7.     return sum/length
8. l1=[10,20,30,40,50]
9. print("average of list is ",avg(l1))
10. l2=[]
11. print("average of list is ",avg(l2))
```

Output

```
1. average of list is  30.0
2. Traceback (most recent call last):
3.   File " assert.py", line 11, in <module>
4.     print("average of list is ",avg(l2))
5.   File " assert.py", line 3, in avg
6.     assert length!=0,"List is empty"
7. AssertionError: List is empty
```

Handling an exception

We can write programs that handle selected exceptions. If you have any code written in your program that can raise an exception, you can prevent your program from

crashing by placing the suspicious code in a **try:** block. Put an **except:** statement after the try: block, followed by a block of code that handles the problem as elegantly as possible.

Syntax

```
try:
    Suspicious code
except ExceptionI:
    Statements
except ExceptionII:
    Statements
    .....
else:
    If there is no exception then execute this block.
```

Here are a few important points about the preceding syntax:

- There can be more than one except statement in a try statement. The try block contains statements that throw exceptions, which is useful.
- We can also provide a generic except clause that handles any exception.
- We can also add a else clause that executes if no exception is raised.

Example

```
1. def test():
2.     try:
3.         x = int(input("Please enter a small positive integer: "))
4.         print("you entered ",x)
5.     except ValueError:
6.         print("This is not an integer value ")
7. test()
8. test()
```

Output

```
Please enter a small positive integer: 23
you entered  23
Please enter a small positive integer: 'hello'
This is not an integer value
```

Note: If the try block raises an exception at any statement , the code following that statement will not be executed. The control will shift to the except block.

The except clause with else clause

In a try statement, you can add a single else clause after all the except clauses. The else block executes if the code in the try block does not raise any exceptions.

The else block is a good place for code that does not require the try: block's protection.

Syntax

```
try:
    Suspicious code
except ExceptionI:
    Statements
except ExceptionII:
    Statements
    .....
else:
    If there is no exception then execute this block.
```

Example

```
1. def test():
2.     try:
3.         a=int(input("Enter an integer: "))
4.     except ValueError:
5.         print("This is not an integer value")
6.     else:
7.         print("Square of %d is %d" %(a,a*a))
8. test()
9. test()
```

Output

```
Enter an integer: 34.4
This is not an integer value
Enter an integer: 22
Square of 22 is 484
```

With no exceptions except close

Use the except statement as follows if no exceptions are defined:

Syntax

try:

 Suspicious code

except :

 Statements

else:

 If there is no exception then execute this block.

You will catch every exception that occurs if you use this type of statement. Since this practice will handle all exceptions in the same way, it is not a good programming practice. An except clause can specify which exceptions it will catch. There can be any number of except clauses within a try clause, but only one will be executed when an exception occurs.

Example

```
1. try:
2.     a=int(input("enter 1st number: "))
3.     b=int(input("enter 2nd number: "))
4.     ans=a/b
5.     print("After dividing first by second we get: ",ans)
6. except:
7.     print("Something went wrong")
```

Output

```
enter 1st number: 2
enter 2nd number: ab
Something went wrong
```

The except clause with multiple exceptions

A single line of code can result in different types of exceptions. For this, we can use the same except statement to handle multiple exceptions. We can use a tuple of values to specify multiple exceptions in a single **except** clause in the form of a comma-separated group enclosed in a set of parentheses.

Syntax

```
try:
```

```
    Suspicious code
```

```
    .....
```

```
except(Exception1[, Exception2[,...ExceptionN]]):
```

```
    If there is any exception from the given exception list,
    then execute this block
```

Example

```
1. while True:
2.     try:
3.         a=int(input("enter an integer "))
4.     ans=float(a)
5.         print("value of ",a,"in float is ",ans)
6.         break
7.     except(ValueError,TypeError):
8.         print("something went wrong")
9.         print("enter again\n")
```

Output

```
enter an integer 'e'
something went wrong
enter again
```

```
enter an integer None
something went wrong
enter again
```

```
enter an integer 34
value of  34 in float is  34.0
```

Multiple except clauses can also be used to catch multiple exceptions. Following a single try statement, you can list as many except clauses as you like:

Syntax

```
try:
```

```
    Suspicious code
```

```
.....  
except ExceptionI:  
    Statements  
except ExceptionII:  
    Statements  
.....
```

Example

```
1. def test():  
2.     try:  
3.         a=int(input("enter an integer: "))  
4.     except ValueError:  
5.         print("You have not entered an integer")  
6.     except KeyboardInterrupt:  
7.         print("You have interrupted the keyboard")  
8.     except EOFError:  
9.         print("You have done end of file operation")  
10.    else:  
11.        print("You entered ",a)  
12. test()  
13. test()  
14. test()
```

Output

```
enter an integer:  
You have interrupted the keyboard  
enter an integer: a  
You have not entered an integer  
enter an integer: 23  
You entered  23
```

Note: This method for handling multiple exceptions is better than the other one because we can have different blocks for each except clause so that we can print different statements for each one. On the other hand, we can only print a generalized statement for all the exceptions in the other method.

The Try-finally clause

You can use a finally: block with a try: block. Whether or not the try: block occurs as an exception, the finally: block is a place to add any code that must be executed.

Syntax

```
try:
    Suspicious code
    .....
except ExceptionI:
    Statements
finally:
    This code always executes
```

Example

```
1. try:
2.   fh = open("testfile", "w")
3.   fh.write("This is my test file for exception handling!!")
4. finally:
5.     print ("Error: can't find file or read data")
6. fh.close()
```

Output

Error: can't find file or read data

The same example can be written more clearly as follows:

```
1. try:
2.   fh = open("testfile", "w")
3.   try:
4.     fh.write("This is my test file for exception handling!!")
5.   finally:
6.       print ("Going to close the file")
7.   fh.close()
8. except IOError:    ##Exception
9.     print ("Msg: can't find file or read data")
```

When an exception is thrown in the try block, it immediately passes to the finally block. After all the statements in the finally block are executed, the exception is raised again and is handled in the except statements if present in the next higher layer of the try-except statement.

With statement

Try blocks are commonly used to acquire resources, and finally blocks are used to release the resources. This is achieved with the help of the **with** statement.

Consider the following example with the finally clause:

```
1.  f = None
2.  try:
3.      f = open("f:\\first.txt", 'r')
4.      while True:
5.          line = f.readline()
6.          if len(line) == 0:
7.              break
8.          print(line)
9.  except IOError:
10.     print ("Could not find file my_file.txt")
11. except KeyboardInterrupt:
12.     print ("!! You cancelled the reading from the file.")
13. f finally:
14.     if f:
15.         f.close()
16.     print ("Cleaning up: Closed the file")
```

Output

hello

i am learning Python

today is thursday

Cleaning up: Closed the file

Now, the difference here is that we are using the open function with the **with** statement; we leave the closing of the file to be done automatically with open.

Argument of an exception

When an exception occurs, it may have an associated value, i.e., the Exception's argument. The argument is usually an official message from Python describing the **Exception**. You can receive the argument if you specify a variable after the exception type, preceded by the **as** keyword. When we specify `except Exception as e`, the variable `e` is not an argument but an instance of the Exception itself.

Syntax

try:

You do your operations here

.....

except ExceptionType as Argument:

You can print value of Argument here...

In an **except** command, you can have a variable follow the name of the Exception if you are handling a single exception. On the other hand, a variable can track the exception tuple if you are trapping more than one Exception.

The value of this variable is the value of the Exception, mainly containing the Exception clause. Variables can accept one or more values. The tuple usually contains an error number, an error string, and the location of the error.

Example

```
1. try:
2.     a=int(input("Enter a number: "))
3.     print("You entered ")
4. except ValueError as Argument:
5.     print("The argument does not contain numbers\n",Argument)
```

Output

Enter a number: abc

The argument does not contain numbers

invalid literal for int() with base 10: 'abc'

Raising an exception

We can forcefully raise exceptions using the **raise** keyword in Python programming when related errors occur at runtime. Exceptions can also, optionally, be populated with value to clarify why they occurred.

Syntax

```
raise [Exception [, args [, traceback]]]
```

Arguments is a value for the exception argument, and Exception describes the type of Exception that occurred (for example, NameError). Exception arguments are optional, and NoException is returned if they are not provided.

The final argument, traceback, is optional (and seldom used in practice), and it is the traceback object used in the Exception if present.

Note: To catch an exception, the "except" clause must refer to the same Exception thrown as a class object or as a simple string.

Example

```
1. def test():
2.     try:
3.         a=int(input("enter a number between 25 and 40: "))
4.         if a>40 or a<25:
5.             raise ValueError("Out of range")
6.         print("you entered ",a)
7.     except ValueError as ve:
8.         print(ve)
9.     test()
10. test()
```

Output

```
enter a number between 25 and 40: 22
Out of range
enter a number between 25 and 40: 40
you entered  40
```

User-defined exceptions

Users can define their own Exception by creating a new class in Python. This exception class has to be derived, either directly or indirectly, from the Exception class. Most built-in exceptions are also derived from this class.

Example

```
1. class Error(Exception):
2.     pass
```

```
3. class ValSmallError(Error):
4.     pass
5. class ValLargeError(Error):
6.     pass
7. ll=10
8. up=20
9. while True:
10.    try:
11.        n=int(input("enter any number"))
12.        if n<ll:
13.            raise ValSmallError
14.    elif n>up:
15.        raise ValLargeError
16.        break
17.    except ValSmallError:
18.        print("this value is very small")
19.        print("-----")
20.    except ValLargeError:
21.        print("this value is very large")
22.        print("-----")
23. print("you guessed number between the range")
```

Output

enter any number1

this value is very small

enter any number56

this value is very large

enter any number34

this value is very large

enter any number22

this value is very large

enter any number16
you guessed number between the range

Conclusion

Exception handling is an essential concept in Python. In this chapter, we learned about exceptions and their handling in Python. We also looked at examples for easy understanding of the concepts.

In the next chapter, we will learn about object-oriented programming.

Key terms

- Exception
- ZeroDivisionError
- Exception handling
- Except clause
- With statement
- Raising an exception

CHAPTER 16

Object-Oriented Programming

We have designed our program around functions in all the programs we wrote, i.e., blocks of statements manipulating data; this is called procedure-oriented programming. Another way of organizing your program is to combine data and functionality and wrap it inside something called an object; this is called object-oriented programming. If you're writing large programs or have a problem better suited to object-oriented programming, you can use procedural programming most of the time.

Structure

We will cover the following topics in this chapter:

- An overview of OOP terminology
- Creating classes
- Creating instance objects
- Accessing attributes
- Built-in class attributes
- Destroying objects
- Class inheritance

- Types of inheritance
- Super() keyword
- Functions to check a relationship of two classes and instances
- Overriding methods

Objectives

The main objective of this chapter is to help learners understand the concept of OOP from scratch. It also focuses on classes and objects and looks at accessing attributes and built-in class attributes. The chapter uses examples to help learners understand the concepts of inheritance. We will also look at overriding methods at the end of the chapter.

An overview of OOP terminology

Object: Instances of a data structure that are unique which are defined by its class. Object is simply a collection of data (variables) and methods (functions) that act on that data.

Class: Class is a blueprint for the object or a user-defined prototype for a thing that defines a set of attributes that characterize any entity/object of the **Class**.

Class variable: Each instance of a Class has access to the class variables. Class variables exist only once, and any change to a class variable will be seen by all the instances of the Class.

Object variable (data member): Every object/instance of a Class has access to the object variables. Hence, each object has its own copy of the field, and they are not related to the field with the same name in another instance. This will be easier to understand with an example.

Function overloading: A function may be assigned more than one behavior. Different operations are performed depending on the type of object or argument.

Instance variable: These are the variables defined inside a method and belonging only to the instance of a class.

Inheritance: This means that the characteristics of one Class are transferred to another class derived from it.

Instance: This is a single object of a class. Instances of a class Circle include objects that belong to the Class, for example.

Instantiation: It refers to the creation of an instance/object of a class.

Method: This is a function that is defined in a class definition.

Object: Class-defined data structures are unique instances. Objects consist of both data members (class variables and instance variables) and methods.

Operator overloading: This is the assignment of more than one function to a particular operator.

Creating classes

Create a class using the `"""class"""` keyword. docstring is the first string and contains a brief description of the Class. It is recommended, but it is not compulsory.

1. `class MyNewClass:`
2. `'''This is a docstring. I have created a new class'''`
3. `pass`

All the attributes of a class are defined in the class namespace. Data and functions can be attributes, and special attributes begin with double underscores (`__`). A docstring for a class can be found by looking up its `__doc__` attribute. Each time we define a class, a new class object is created with the same name. The object of this class allows us to access its attributes and instantiate new objects from it.

Syntax

```
class ClassName:
<statement-1>
<statement-2>
...
<statement-N>
```

The self

Unlike ordinary functions, class methods have only one difference—an extra first name must be added to the beginning of the parameter list. However, you don't need to provide a value for this parameter when calling the method; Python will do it for you. Self refers to the object itself, and it is called self by convention.

You can give this parameter any name, but self is strongly recommended; any other name would be highly discouraged. Standardizing the name of your program has many advantages, including immediate recognition by any reader.

The `__init__` method

Functions in the class that begin with double underscore (`__`) are called special functions since they have special meaning. The `__init__()` function is of particular interest. This special function is called if a new object of that class is created. In Object Oriented Programming (OOP), this type of function is called a constructor. The initialization of all variables is usually done with it.

Example

```
1. class student:
2.     'This is my first program using classes'
3.     def __init__(self,name,age):
4.         self.name=name
5.         self.age=age
6.     def display(self):
7.         print("name = ",self.name)
8.         print("age = ",self.age)
```

Creating instance objects

You can create instances of a class by calling its `__init__` method with the class name and the arguments it accepts.

```
1. #creating objects
2. ob1=student("Rohit",21)
3. ob2=student("Divanshi",20)
```

Accessing attributes

You can access an object's attributes using the dot operator and access class variables by using the class name, as follows:

```
1. ob1.display()
2. ob2.display()
```

Now, we get the following by putting them together:

```
1. class student:
2.     'This is my first program using classes'
3.     def __init__(self,name,age):
```

```

4.         self.name=name
5.     self.age=age
6.     def display(self):
7.         print("name = ",self.name)
8.         print("age = ",self.age)
9.     #creating an object
10. ob1=student("Rohit",21)
11. ob2=student("Divanshi",20)
12. ob1.display()
13. ob2.display()

```

Output

```

name = Rohit
age = 21
name = Divanshi
age = 20

```

The following functions are alternative methods to the regular statements for accessing attributes:

- **getattr(obj, name[, default])**: to access an attribute of an object
- **hasattr(obj,name)**: to check whether an attribute is present
- Use the **setattr(obj,name,value)** method to set an attribute; the attribute will be created if it does not exist.
- The **delattr(obj, name)** method deletes an attribute.

Take a look at these examples:

```

1. hasattr(ob1, 'age')    # Returns true if 'age' attribute exists
2. getattr(ob1, 'name')  # Returns value of 'age' attribute
3. setattr(ob1, 'age', 17) # Set attribute 'age' at 17 of ob1
4. delattr(ob2, 'age')    # Delete attribute 'age' of ob2

```

Built-in class attributes

Every Python class keeps the following built-in attributes, and they can be accessed using the dot operator, like any other attribute:

- `__dict__`: Namespaces for the classes in a dictionary
- `__doc__`: If undefined, the documentation string will be used
- `__name__`: Name of the class
- `__module__`: The name of the module where the class resides; this attribute is "`__main__`" in interactive mode
- `__bases__`: A possible empty tuple containing the base classes in the order in which they appear in the base class list

Example

```
1. class Student:
2.     'This is my first program using classes'
3.     def __init__(self,name,age):
4.         self.name=name
5.     self.age=age
6.     def display(self):
7.         print("name = ",self.name)
8.         print("age = ",self.age)
9.     #creating an object
10. ob1=Student("Rohit",21)
11. ob2=Student("Divanshi",20)
12. print ("Student.__doc__:", Student.__doc__)
13. print ("Student.__name__:", Student.__name__)
14. print ("Student.__module__:", Student.__module__)
15. print ("Student.__bases__:", Student.__bases__)
16. print ("Student.__dict__:",Student.__dict__ )
```

Output

```
1. Student.__doc__: This is my first program using classes
2. Student.__name__: Student
3. Student.__module__: __main__
4. Student.__bases__: (<class 'object'>,)
5. Student.__dict__: {'__module__': '__main__', '__doc__': 'This is my first program using classes', '__
```

```

init__': <function Student.__
init__ at 0x00EB22B8>, 'display': <function Student.
display at 0x02F48810>, '__dict__': <attribute '__
dict__' of 'Student' objects>, '__weakref__': <attribute '__
weakref__' of 'Student' objects>}

```

Destroying objects (garbage collection)

Python automatically deletes unnecessary objects (built-in types or class instances) to free up memory. During garbage collection, Python periodically reclaims blocks of memory that are no longer in use. This occurs when an object's reference count reaches 0 in Python's garbage collector. The reference count of an object changes as the number of aliases pointing to it changes.

An object's reference count increases when it is assigned a new name or placed in a container (list, tuple, or dictionary), and its reference count decreases when it is deleted with `del`, reassigned, or when it goes out of scope. Python automatically collects reference counts when they reach zero.

```

x = 50      # Create object <40>
y = x       # Increase ref. count  of <40>
z = [y]     # Increase ref. count  of <40>
del x       # Decrease ref. count  of <40>
y = 100     # Decrease ref. count  of <40>
z[0] = -1   # Decrease ref. count  of <40>

```

You will normally not notice when the garbage collector destroys an orphaned instance and reclaims its space. An instance of a class may implement a special `__del__` () method, called a destructor, that is invoked when the instance is about to be destroyed. An instance might use this method to clean up any non-memory resources it has used.

Class inheritance

We can create a class by deriving it from an existing class by listing the parent class in parentheses after the new class name.

A child class inherits the attributes of its parent class, and you can use these attributes as if they were defined in the child class. A child class can also override data members and methods from its parent.

Syntax

```

class SubClassName (ParentClass1[, ParentClass2, ...]):
    'Optional class documentation string'

```

class_suite

Example

```
1. class Parent:
2.     parentAttr = 100
3.     def __init__(self):
4.         print ("Calling parent constructor")
5.     defparentMethod(self):
6.         print ('Calling parent method')
7. defsetAttr(self, attr):
8.     Parent.parentAttr = attr
9. defgetAttr(self):
10.        print ("Parent attribute :", Parent.parentAttr)
11. class Child(Parent): # define child class
12.     def __init__(self):
13.         print ("Calling child constructor")
14. defchildMethod(self):
15.         print ('Calling child method')
16. c = Child()
17. c.childMethod()
18. c.parentMethod()
19. c.setAttr(200)
20. c.getAttr()
```

Output

```
Calling child constructor
Calling child method
Calling parent method
Parent attribute : 200
```

Types of inheritance

There are two types of inheritance:

- Multilevel inheritance

- Multiple inheritance

Multilevel inheritance

Like other object oriented programming languages, Python supports multilevel inheritance. This type of inheritance occurs when a derived class inherits from another derived class. There is no maximum depth for multilevel inheritance in Python.

Example

```
1. class Animal:
2.     def eat(self):
3.         print('Eating...')
4. class Dog(Animal):
5.     def bark(self):
6.         print ('Barking...')
7. class BabyDog(Dog):
8.     def weep(self):
9.         print ('Weeping...')
10. d=BabyDog()
    11.d.eat()
12. d.bark()
13. d.weep()
```

Output

Eating...

Barking...

Weeping...

Multiple inheritance

Multiple inheritance is supported in Python, so we can inherit from multiple parents. A child class can be derived from more than one base (parent) class.

Example

```
1. class First(object):
2.     def __init__(self):
```

```
3.     super(First, self).__init__()
4.     print("first")
5.
6.     class Second(object):
7.     def __init__(self):
8.         super(Second, self).__init__()
9.         print("second")
10.
11.    class Third(Second, First):
12.    def __init__(self):
13.        super(Third, self).__init__()
14.        print("third")
```

Output

```
first
second
third
```

The super () keyword

The **super()** method is most commonly used with the **__init__** function in the base class. This is usually the only place where we need to do some things in a child class and then complete the initialization in the parent.

Example

```
1.  class omnivores:
2.  def __init__(self, omni):
3.      print(omni, "are omnivores")
4.  class human(omnivores):
5.  def __init__(self):
6.      print('omnivores are those whole eat both animals
    and plants')
7.      super().__init__("humans")
8.  obj=human()
```

Output

omnivores are those whole eat both animals and plants
 humans are omnivores

Functions to check the relationship between two classes and instances

- **issubclass(sub, sup) boolean function:** It returns True if the given subclass sub is indeed a subclass of the superclass sup.
- **isinstance(obj, class) boolean function:** It returns True if obj is an instance of class or if it is an instance of a subclass of class.

Overriding methods

When overriding a parent method, you may want to add unique or different functionality to your subclass.

Example

```

1. class Parent:
2.     defmyMethod(self):
3.         print ('Calling parent method')
4. class Child(Parent):
5.     defmyMethod(self):
6.         print ('Calling child method')
7. c = Child()
8. c.myMethod()
```

Output

Calling child method

Base overloading methods

- **__init__ (self [,args...]):** Constructor (with any optional arguments)
Sample Call: obj= className(args)
- **__del__(self):** Destructor, deletes an object
Sample Call: del obj

- **`__repr__( self )`**: Evaluatable string representation
Sample Call: `repr(obj)`
- **`__str__( self )`**: Printable string representation
Sample Call: `str(obj)`
- **`__cmp__ ( self, x )`**: Object comparison
Sample Call: `cmp(obj, x)`

Programs

1. Program to find the area of a rectangle using classes:

```
1. class rectangle():
2.     def __init__(self,breadth,length):
3.         self.breadth=breathth
4.         self.length=length
5.     def area(self):
6.         return self.breadth*self.length
7. a=int(input("Enter length of rectangle: "))
8. b=int(input("Enter breadth of rectangle: "))
9. obj=rectangle(a,b)
10. print("Area of rectangle:",obj.area())
11. print()
```

Output

```
Enter length of rectangle: 3
Enter breadth of rectangle: 4
Area of rectangle: 12
```

2. Program to create a class in which one method accepts the string and the other displays it:

```
1. class print1():
2.     def __init__(self):
3.         self.string=""
4.     def get(self):
5.         self.string=input("Enter string: ")
```

```
6. def put(self):
7.     print("String is:")
8.     print(self.string)
9. obj=print1()
10. obj.get()
11. obj.put()
```

Output

```
Enter string: hello
String is:
hello
```

3. Program to append, delete, and display the elements of a list using classes:

```
1. Class perform:
2. def __init__(self,l1):
3.     self.n=l1
4. def add(self,a):
5.     return self.n.append(a)
6. def remove(self,b):
7.     return self.n.remove(b)
8. def display(self):
9.     return (self.n)
10. l1=[2,4,66,56,34]
11. obj=perform(l1)
12. n=int(input("enter number to append: "))
13. obj.add(n)
14. print(obj.display())
15. n=int(input("enter element to be removed: "))
16. obj.remove(n)
17. print(obj.display())
```

Output

```
enter number to append: 25
[2, 4, 66, 56, 34, 25]
```

```
enter element to be removed: 56  
[2, 4, 66, 34, 25]
```

4. Program to overload the '*' operator:

```
1. class A:  
2.     def __init__(self,x):  
3.         self.x=x  
4.     def __mul__(self,other):  
5.         print("First number ",self.x)  
6.         print("Second number ",other.x)  
7.         print("Multiplication is ",self.x*other.x)  
8. a=int(input("Enter first number: "))  
9. ob1=A(a)  
10. b=int(input("Enter second number: "))  
11. ob2=A(b)  
12. ob3=ob1*ob2
```

Output

```
Enter first number: 12  
Enter second number: 5  
First number  12  
Second number 5  
Multiplication is  60
```

5. Program to find the area of a rectangle using classes:

```
a. class rect:  
b.     def __init__(self,breadth,length):  
c.         self.breadth=breathth  
d.         self.length=length  
e.     def area(self):  
f.         return self.breadth*self.length  
g. a=int(input("Enter length: "))  
h. b=int(input("Enter breadth: "))  
i. obj=rect(a,b)
```

```
j. print("Area is ",obj.area())
```

Output

Enter length: 5

Enter breadth: 6

Area is: 30

6. Program to check whether a class is a subclass of another class:

```
1. classBase(object):
2.     pass    # Empty Class
3.
4. classDerived(Base):
5.     pass    # Empty Class
6.
7. # Driver Code
8. print(issubclass(Derived, Base))
9. print(issubclass(Base, Derived))
10.
11.d =Derived()
12.b =Base()
13.
14.# b is not an instance of Derived
15.print(isinstance(b, Derived))
16.
17.# But d is an instance of Base
18.print(isinstance(d, Base))
```

Output

True

False

False

True

7. Program to implement inheritance:

```
1. # Base or Super class
2. class Person(object):
3.     def __init__(self, name):
4.         self.name = name
5.
6.     def getName(self):
7.         return self.name
8.
9.     def isEmployee(self):
10.        return False
11.
12. # Inherited or Subclass (Note Person in bracket)
13. class Employee(Person):
14.     def __init__(self, name, eid):
15.
16.         ''' In Python 3.0+, "super().__init__(name)"
17.            also works'''
18.         super(Employee, self).__init__(name)
19.         self.empID = eid
20.
21.     def isEmployee(self):
22.         return True
23.
24.     def getID(self):
25.         return self.empID
26.
27. # Driver code
28. emp = Employee("Geek1", "E101")
29. print(emp.getName(), emp.isEmployee(), emp.getID())
```

Output

```
('Geek1', True, 'E101')
```

Conclusion

In this chapter, we learned about the OOP concept in Python from scratch. We also looked at creating classes and accessing attributes, understood different inheritances, and learned how to override methods. Examples are included in the chapter help learners understand the concepts effectively.

Key terms

- Function overloading
- Operator overloading
- Inheritance
- Multilevel inheritance
- Multiple inheritance
- The `super()` keyword
- Base overloading methods

Index

